

Natural language processing

Who am I ?

Purpose

The purpose of this course is to get you kicking with NLP , transformers, LLM's, Gen AI

This is an applied course

Our hammers and nails is PYTORCH

Note : everything is going to be on my github profile

<https://github.com/Amr-Abdellatif>

Do you know GIT ?

Language

<https://github.com/Amr-Abdellatif>

Household names like Echo (Alexa), Siri, and Google Translate have at least one thing in common. They are all products derived from the application of natural language processing (NLP), NLP refers to a set of techniques involving the application of statistical methods, to understand text for the sake of solving real-world tasks. This “understanding” of text is mainly derived by transforming texts to useable computational representations

<https://github.com/Amr-Abdellatif>

language is ambiguous

Ambiguity in Language

- I saw a girl with a telescope. ✓

- I saw a girl with a bicycle. ✓



- I saw a bus with a telescope. ✓



Who had the telescope?



OR



No ambiguity!

Latest Technologies

1. LLM's
2. Agents
3. Rag

<https://github.com/Amr-Abdellatif>

Text Preprocessing

<https://github.com/Amr-Abdellatif>

Techniques

1. **Lowercasing, removing URLs, punctuations, and stopwords.**
2. **Stemming vs. Lemmatization.**
3. **Regular Expressions (RegEx).**
4. **Text Normalization.**
5. **More ...**

<https://github.com/Amr-Abdellatif>

lets take a look at each one of these

<https://github.com/Amr-Abdellatif>

Lowercase Conversion

Raw	Lowercased
Canada CanadA CANADA	canada
TOMCAT Tomcat toMcat	tomcat

<https://github.com/Amr-Abdellatif>

Stop Word Removal

Stop words are a set of commonly used words in a language.

Stop Words

These words include:

- a
- I
- the
- in
- of
- for
- at
- to
- on
- with
- from

<https://github.com/Amr-Abdellatif>

Stop Word Removal

Sample text with Stop Words	Without Stop Words
GeeksforGeeks – A Computer Science Portal for Geeks	GeeksforGeeks , Computer Science, Portal ,Geeks
Can listening be exhausting?	Listening, Exhausting
I like reading, so I read	Like, Reading, read

<https://github.com/Amr-Abdellatif>

Stemming

Stemming is the process of reducing inflection in words (e.g. troubled, troubles) to their root form (e.g. trouble). The “root” in this case may not be a real root word, but just a canonical form of the original word.

	original_word	stemmed_words
0	connect	connect
1	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

	original_word	stemmed_word
0	trouble	troubl
1	troubled	troubl
2	troubles	troubl
3	troublesome	troublesom

<https://github.com/Amr-Abdellatif>

Stemming

Stemming is useful for dealing with sparsity issues as well as standardizing vocabulary. if say you search for “deep learning classes”, you also want to surface documents that mention “deep learning class” as well as “deep learn classes”, although the latter doesn’t sound right. But you get where we are going with this. You want to match all variations of a word to bring up the most relevant documents.

<https://github.com/Amr-Abdellatif>

Lemmatization

Lemmatization on the surface is very similar to stemming, where the goal is to remove inflections and map a word to its root form. The only difference is that lemmatization tries to do it the proper way. It doesn't just chop things off, it actually transforms words to the actual root, but comes with a slower performance cost

	original_word	lemmatized_word
0	trouble	trouble
1	troubling	trouble
2	troubled	trouble
3	troubles	trouble

	original_word	lemmatized_word
0	goose	goose
1	geese	goose

<https://github.com/Amr-Abdellatif>

Normalization

Text normalization is the process of transforming a text into a canonical (standard) form. For example, the word “goooooood” and “gud” can be transformed to “good”

Raw	Normalized
2moro 2mrrw 2morrow 2mrw tomrw	tomorrow
b4	before
otw	on the way
:) :-) ;-)	smile

<https://github.com/Amr-Abdellatif>

Tokenization

Tokenization is process of dividing corpus into smaller pieces which might be eg. words , letter , subwords.



<https://github.com/Amr-Abdellatif>

Tokenization

[https://tiktokenizer.vercel.app/?
model=gpt-4-1106-preview](https://tiktokenizer.vercel.app/?model=gpt-4-1106-preview)

```
<|im_start|>system<|im_sep|>You are a helpful assistant<|im  
_end|><|im_start|>user<|im_sep|><|im_end|><|im_start|>assis  
tant<|im_sep|>
```

<https://github.com/Amr-Abdellatif>

General Rule of Thumb

Not all tasks need the same level of preprocessing. For some tasks, you can get away with the minimum. However, for others, the dataset is so noisy that, if you don't preprocess enough, **it's going to be garbage-in-garbage-out.**

<https://github.com/Amr-Abdellatif>

Traditional Text Representations

<https://github.com/Amr-Abdellatif>

**we preprocessed text ,the next question is:
how can we represent it in a suitable way ?**

<https://github.com/Amr-Abdellatif>

Sparse vs Dense Vectors

<https://github.com/Amr-Abdellatif>

Bag-of-Words (BoW) & its limitations

The Bag-of-Words model represents text as an unordered collection of words, disregarding grammar and word order. It creates a vocabulary of all unique words in the corpus and represents each document as a vector of word counts.

<https://github.com/Amr-Abdellatif>

Bag-of-Words (BoW)

we think of the whole document as a “bag” of words, rather than a sequence. We represent the document simply by the frequency of each word. For example, if we have a vocabulary of 1,000 words, then the whole document will be represented by a 1,000-dimensional vector, where the vector's i^{th} entry represents the frequency of the i^{th} vocabulary word in the document.

<https://github.com/Amr-Abdellatif>

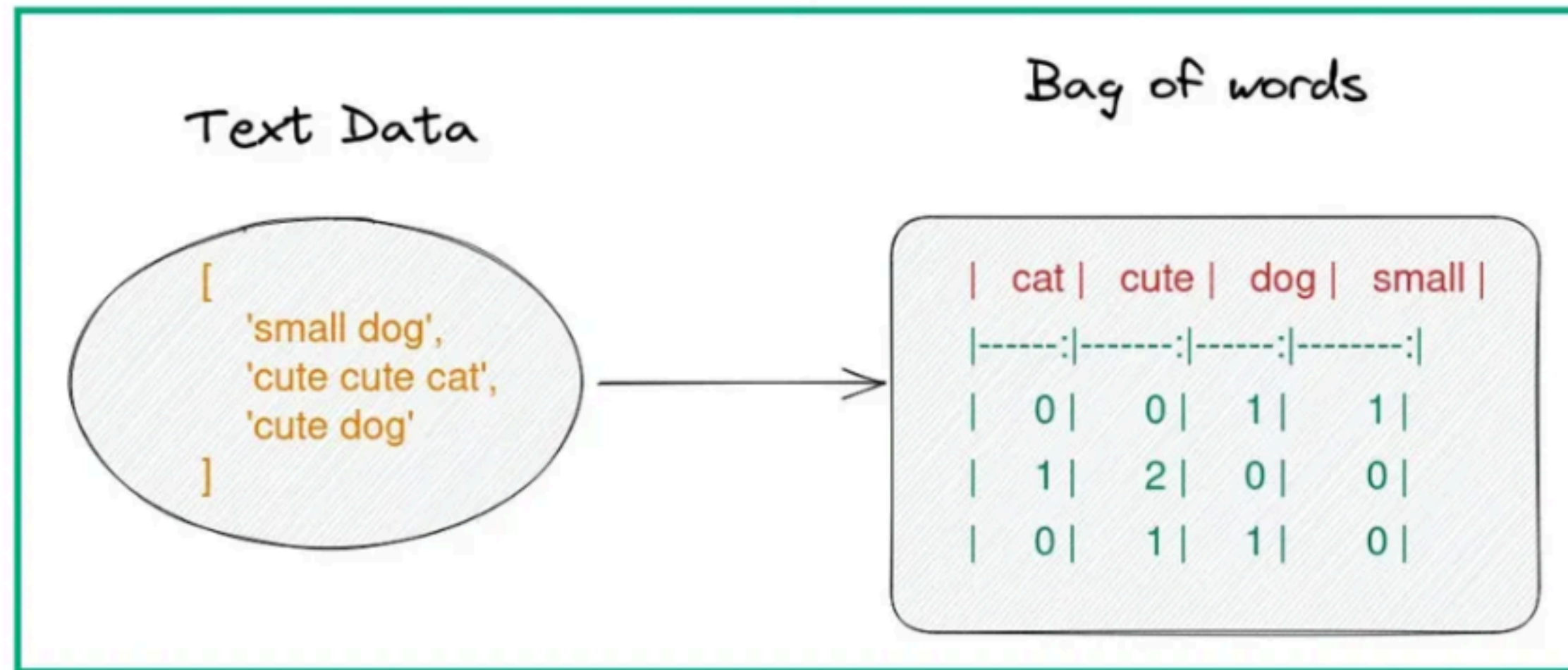
Bag-of-Words (BoW) - Example

we have a document with 50 unique words and its 3 pages only document, how can we represent this 3 pages as 3 vectors each of them has 50 values ?

the answer is BOW

<https://github.com/Amr-Abdellatif>

Bag-of-Words (BoW) - Example



<https://github.com/Amr-Abdellatif>

N-Grams for context capture

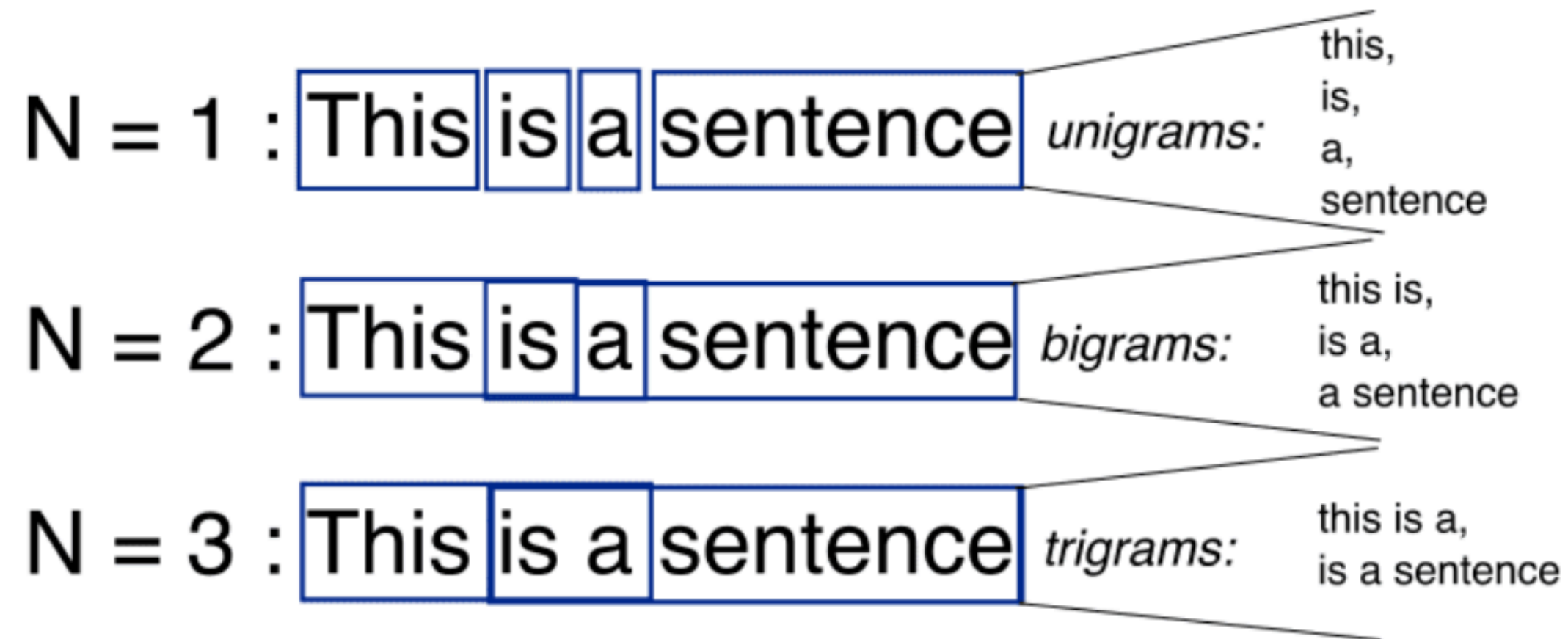


Figure 2 Uni-gram, Bi-gram, and Tri-gram Model

N-grams are sequences of n items from a given text. They help capture some context and word order information.

TF-IDF

is a numerical statistic that reflects how important a word is to a document in a collection of documents (corpus). It's one of the most popular text representation techniques in information retrieval and natural language processing.

<https://github.com/Amr-Abdellatif>

TF-IDF

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

TF measures the frequency of a term within a document

$$IDF(t, D) = \log \left(\frac{\text{Total number of documents in the corpus } N}{\text{Number of documents containing term } t} \right)$$

IDF measures the rarity of a term across a collection of documents

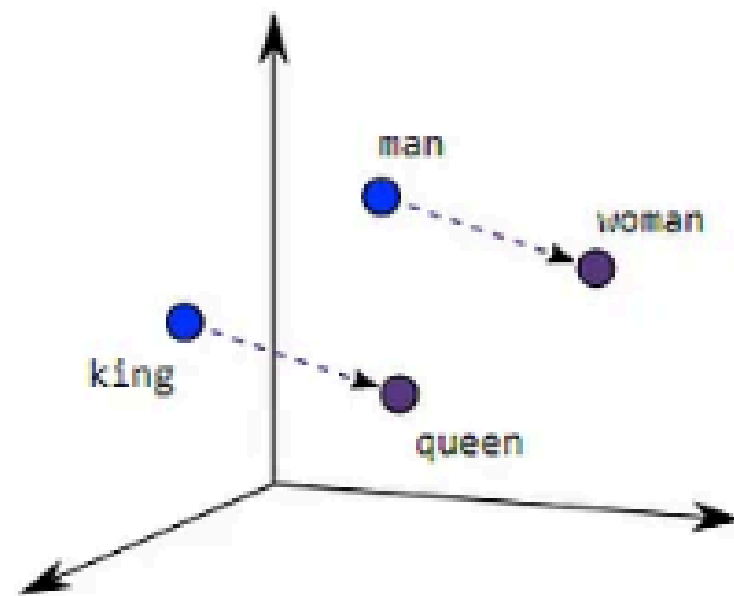
<https://github.com/Amr-Abdellatif>

Why Word Embeddings Improve Over BOW and TF-IDF ?

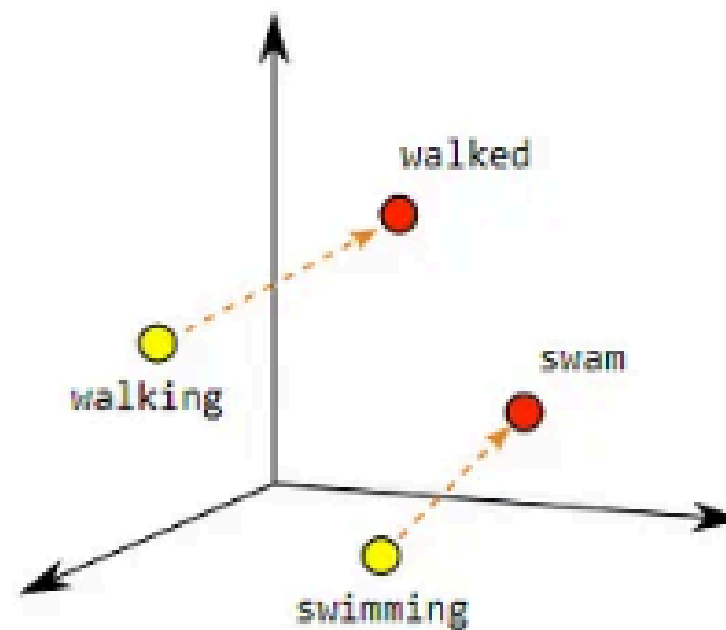
<https://github.com/Amr-Abdellatif>

Word Space Concept: Vector algebra in embeddings

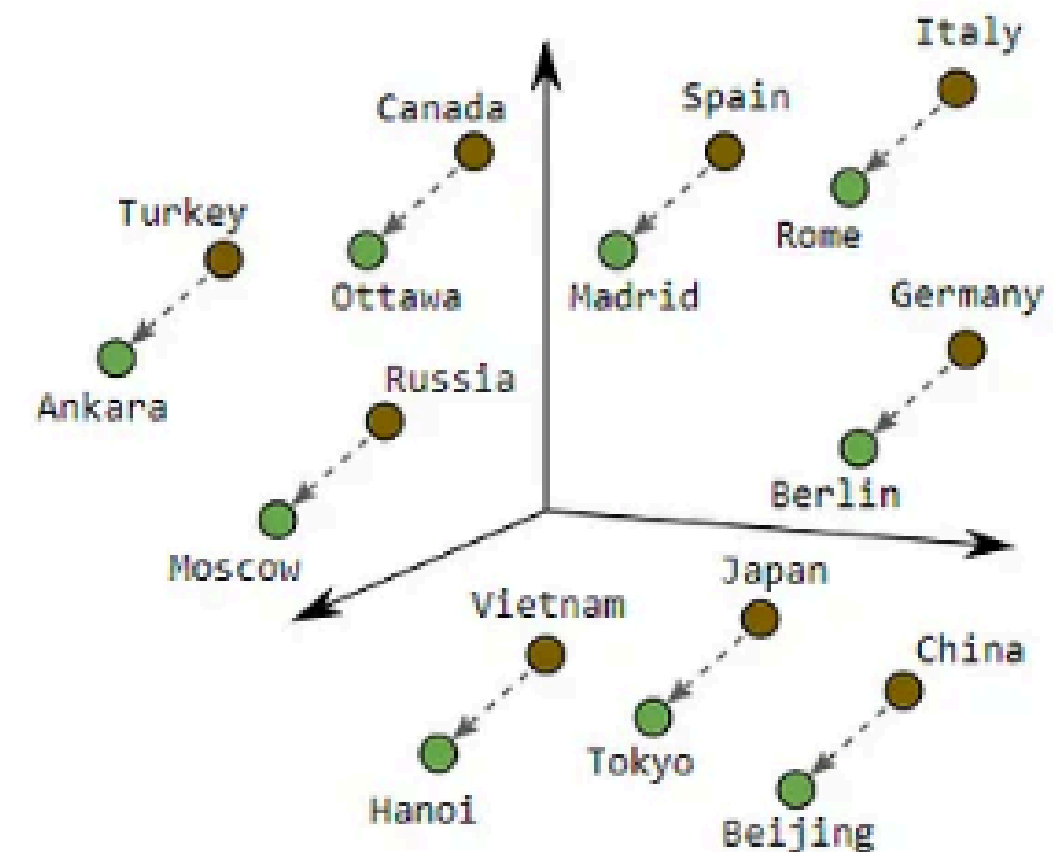
can we visualize these vectors ?



Male-Female



Verb Tense



Country-Capital

<https://github.com/Amr-Abdellatif>

word2vec is not a global vector representation

unlike GLOVE which represents global vectors

<https://github.com/Amr-Abdellatif>

Glove

Glove is an extension to the Word2Vec method. Word2Vec only captures the local context of words. During training, it only considers neighboring words to capture the context. Glove considers the entire corpus and creates a large matrix that can capture the co-occurrence of words within the corpus

<https://github.com/Amr-Abdellatif>

Question

How can we Handle Sequential Data ?

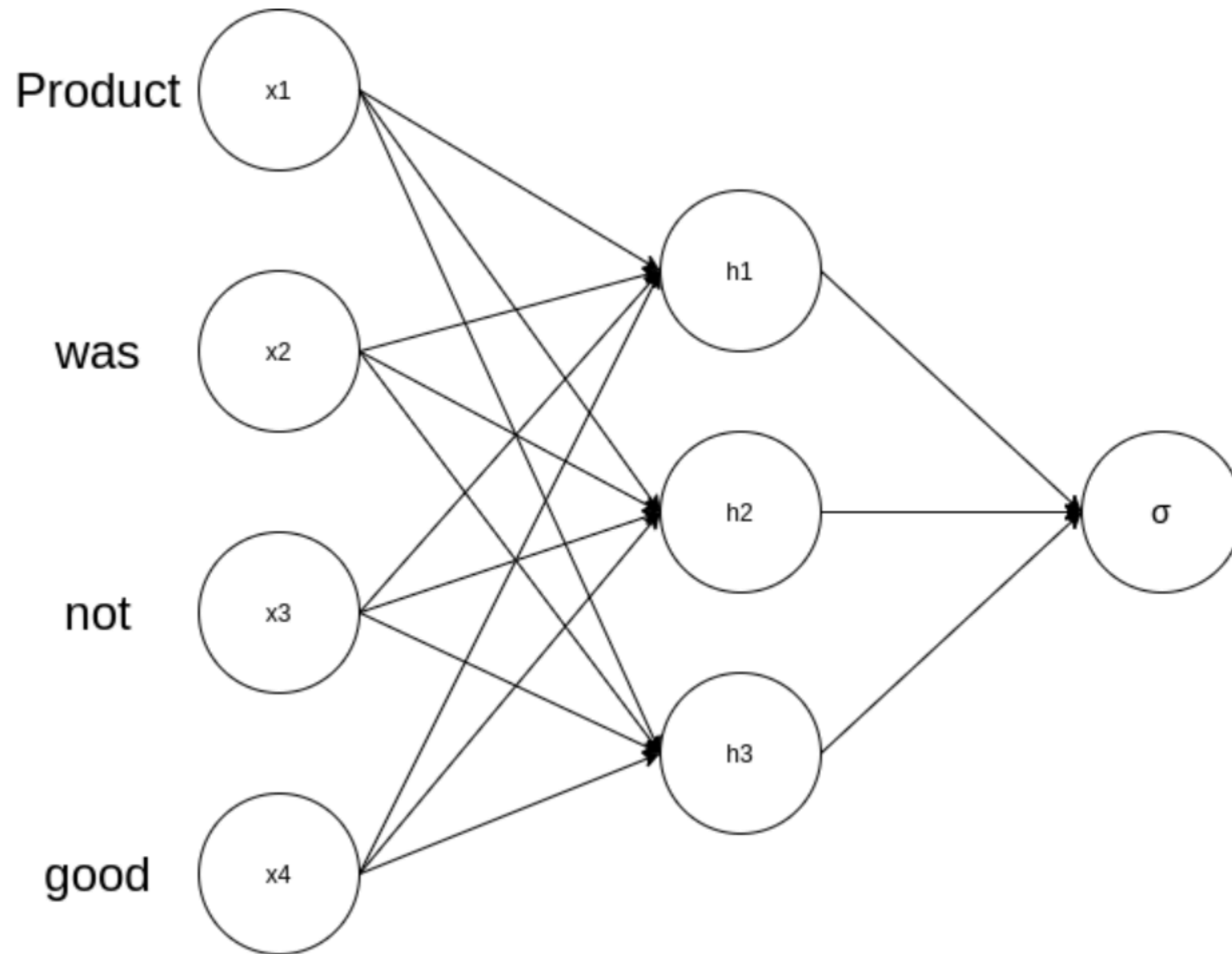
<https://github.com/Amr-Abdellatif>

CNN's

convolution operator endowed our neural network with a prior that items near each other are related but items far from each other have no relationship

<https://github.com/Amr-Abdellatif>

Text with FC layers



- Everything is connected and no relation of sequence is captured

RNN's

Now we want to develop models that can handle a new type of structure: sequences with T items that occur in a specific order. For example, the alphabet —a, b, c, d, . . .—is a sequence of 26 characters.

<https://github.com/Amr-Abdellatif>

RNN's

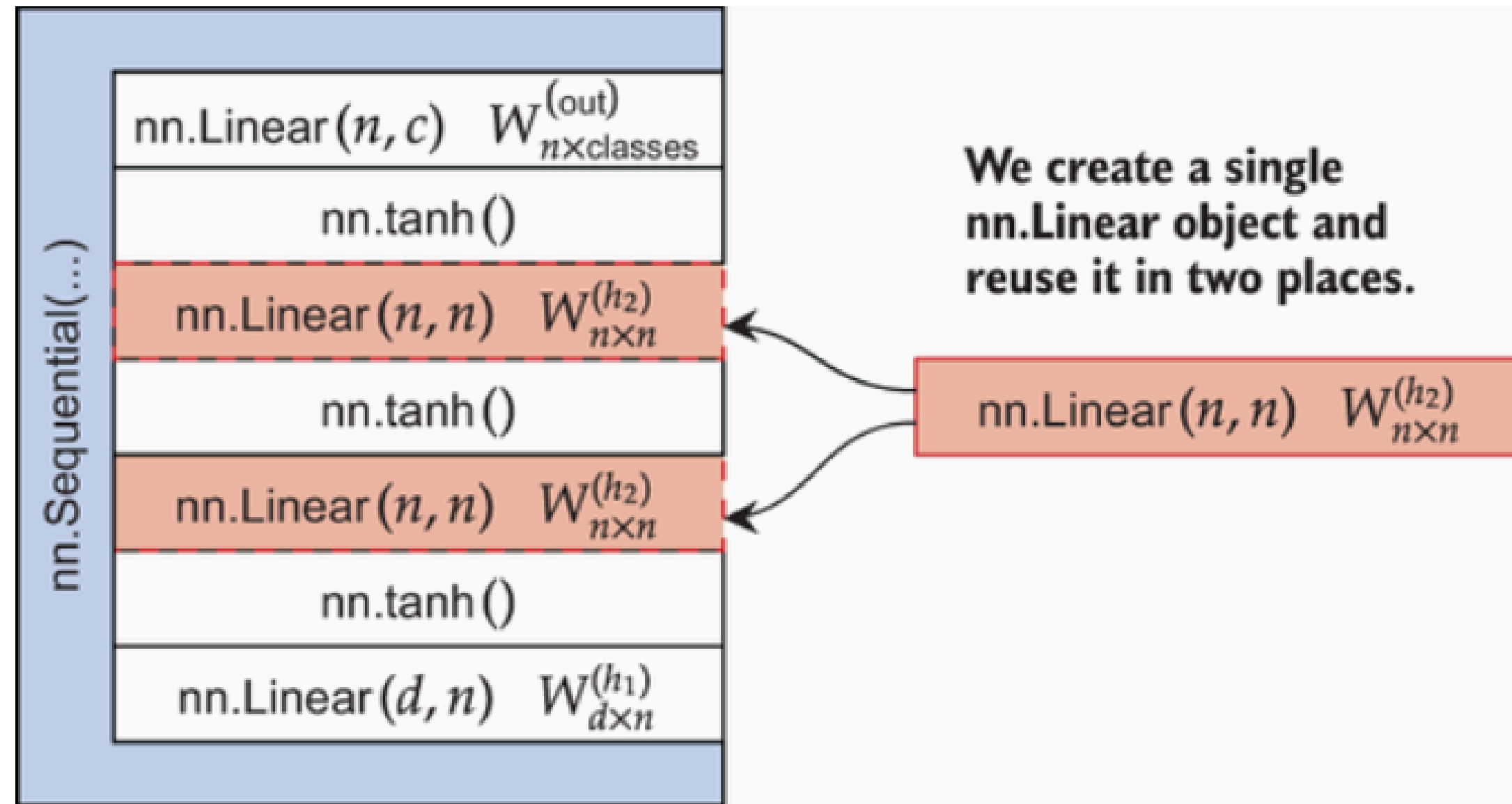
You could use the temperature every hour as a sequence to try to predict the temperature in the future.

In all these cases, the structure is uniquely different compared to the images and convolution. Sequences can have a variable number of items. Forexample, the two previous sentences have variable length

<https://github.com/Amr-Abdellatif>

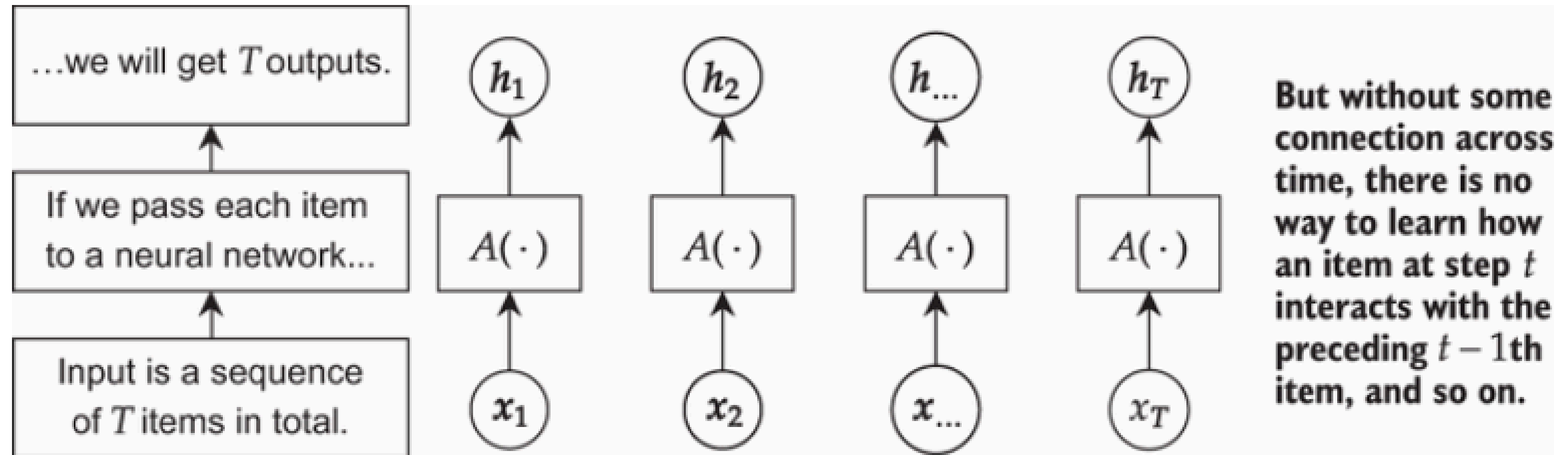
RNN's

Weight sharing methodology

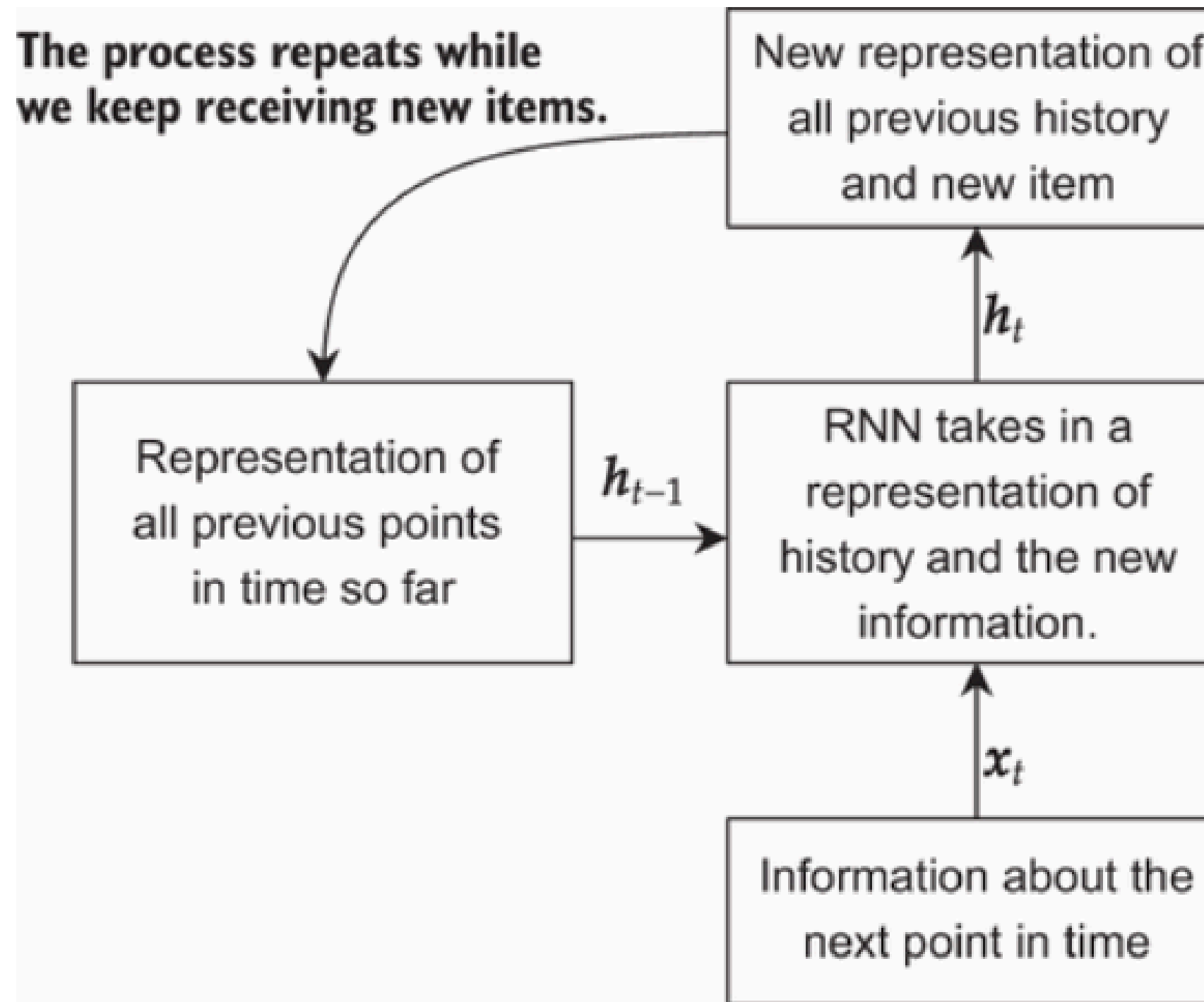


<https://github.com/Amr-Abdellatif>

RNN's



RNN's



<https://github.com/Amr-Abdellatif>

RNN's

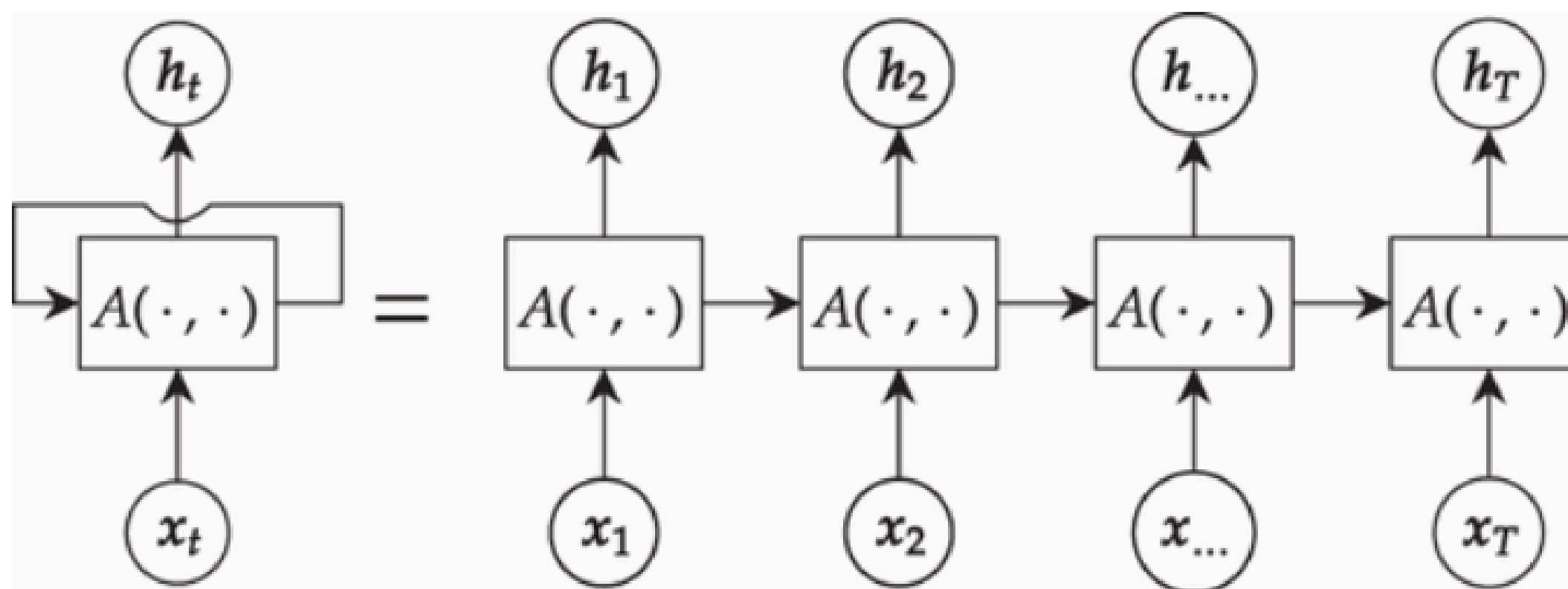


Figure 4.6 The network A is defined to take in two inputs: the previous hidden state h_{t-1} and the current input x_t . This allows us to unroll the network and share information across time, dealing effectively with the sequential nature of the data.

RNN's

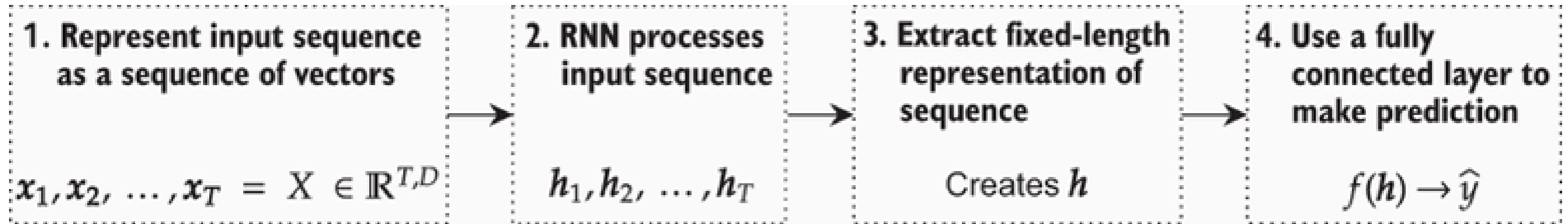
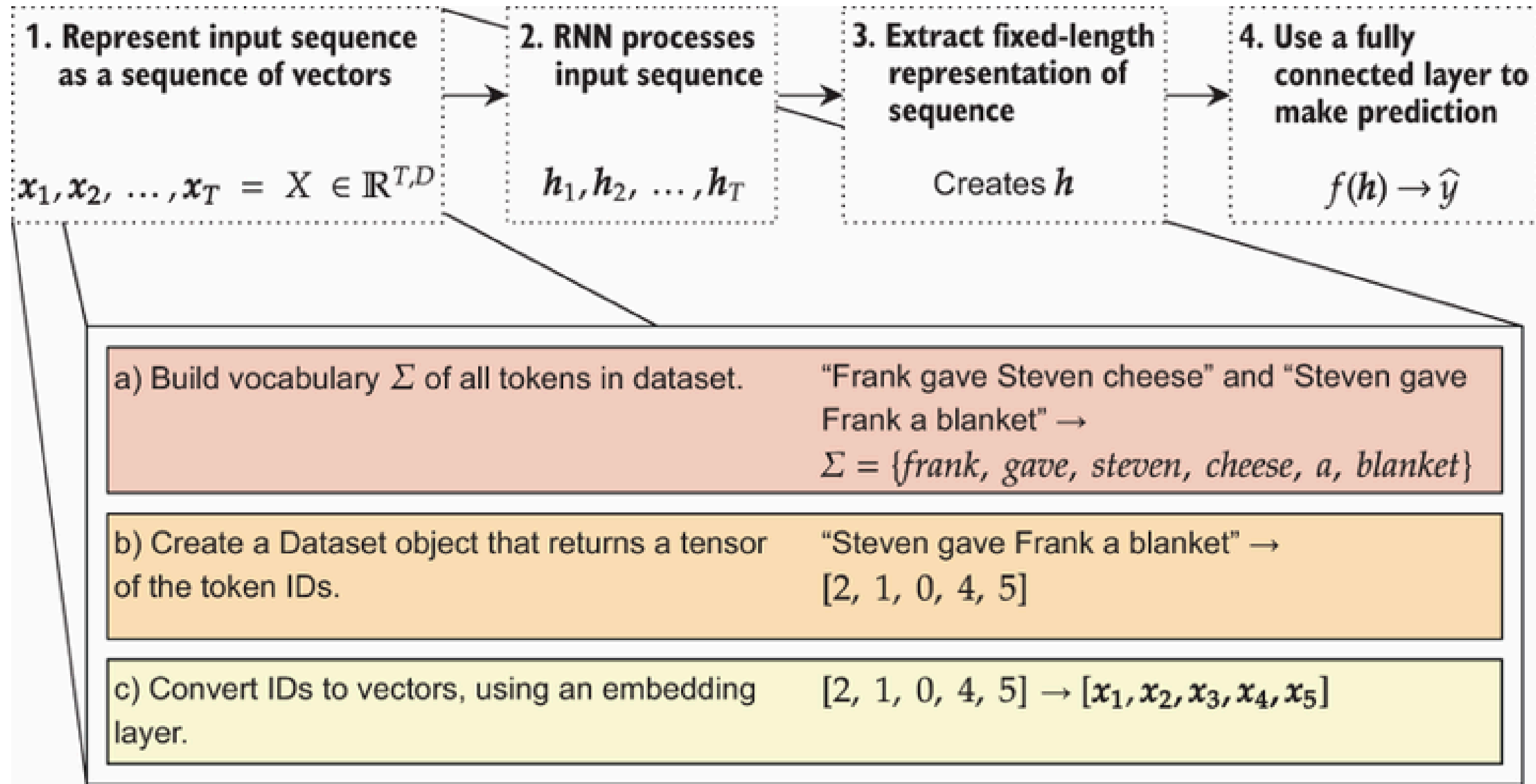
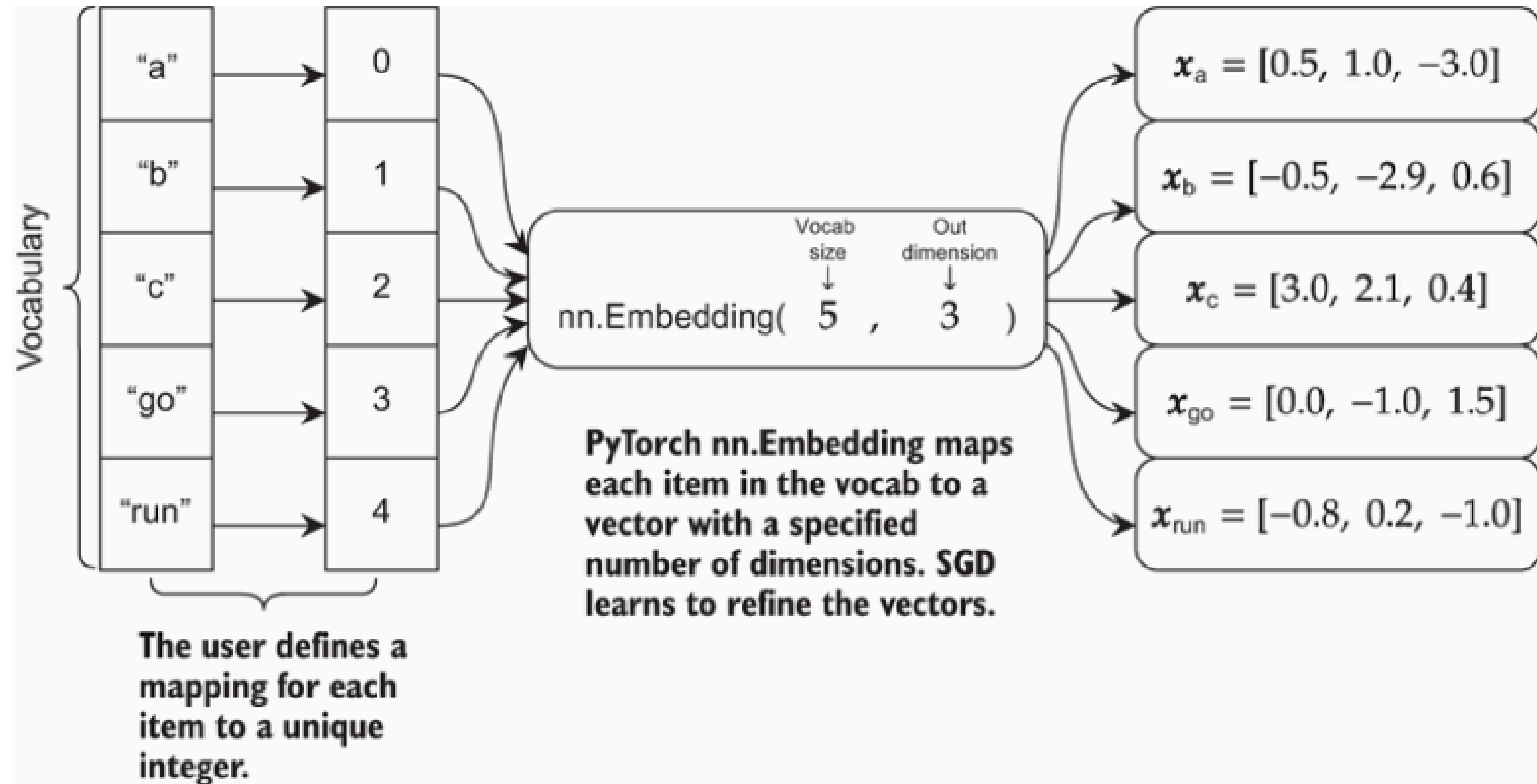


Figure 4.8 Four high-level steps for making a prediction based on an input sequence of length T . We need to create a representation of the sequence as vectors, pass that representation to an RNN to produce T hidden states, reduce to one hidden state, and then use a fully connected layer to make a prediction.

How do we think about a model building ?



How do we think about a model building ?



Look for GPT2 vocab size and embedding dim ?

<https://github.com/Amr-Abdellatif>

lets look at some emeddings

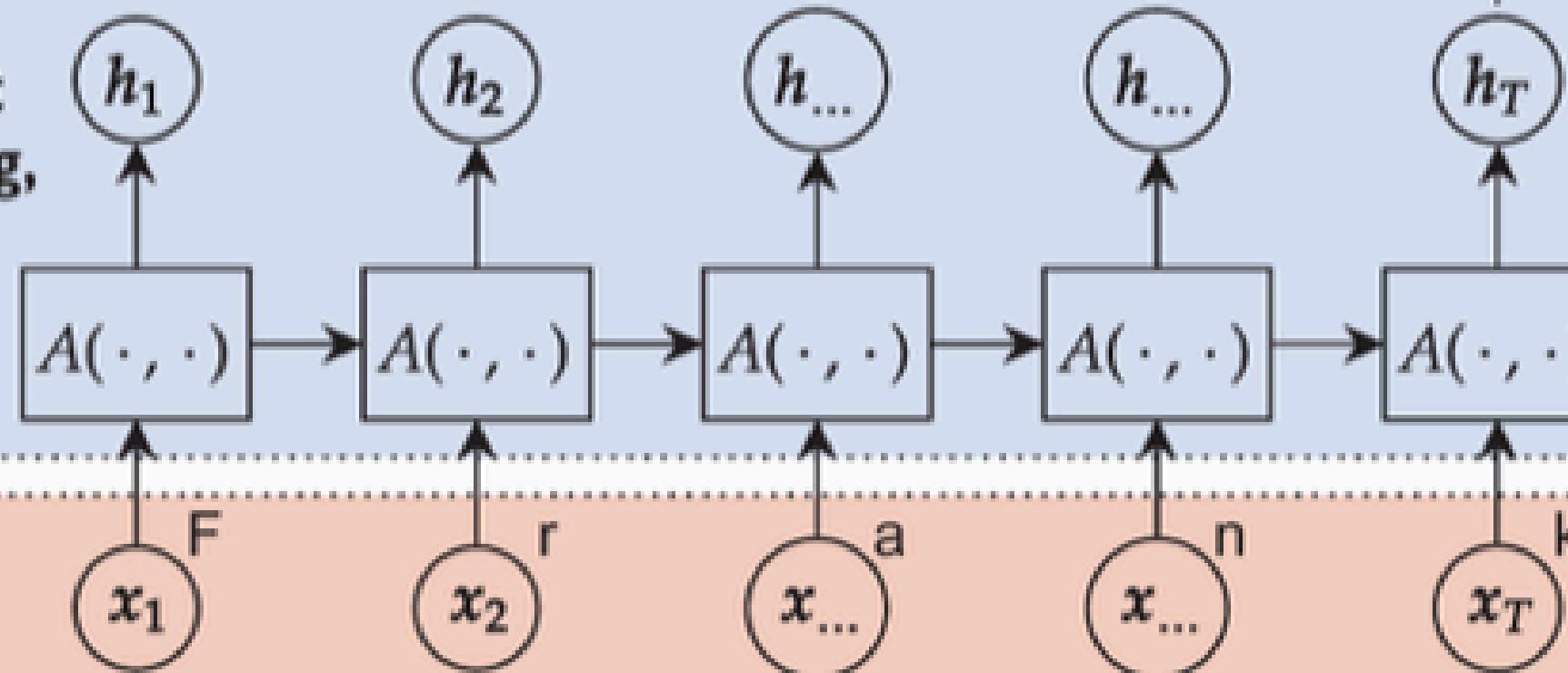
<https://umap-learn.readthedocs.io/en/latest/supervised.html>

<https://github.com/Amr-Abdellatif>

RNN's

RNN is combined with fully connected layer to make one prediction per sequence.

RNN processes input sequence one item at a time. If T items long, the last hidden state summarizes the whole sequence.



Use a fully connected sub-network to process result of sequence.

$f(\cdot)$

h_T

$\hat{y}$

$\ell(\hat{y}, y)$

loss.backward()
optimizer.update()

Incur loss for prediction, calculate gradient, then update model via optimizer.

Data is a sequence, also called “time series” of T items or “steps.” Each item is represented by a vector.

The Problem with RNN's

When processing long sequences, gradients can become extremely small as they're backpropagated through time. This means the network struggles to learn long-range dependencies because earlier inputs effectively "disappear" from memory.

<https://github.com/Amr-Abdellatif>

The Problem with RNN's

gradients can also grow exponentially,
causing unstable training and making the
model unable to converge.

<https://github.com/Amr-Abdellatif>

The Problem with RNN's

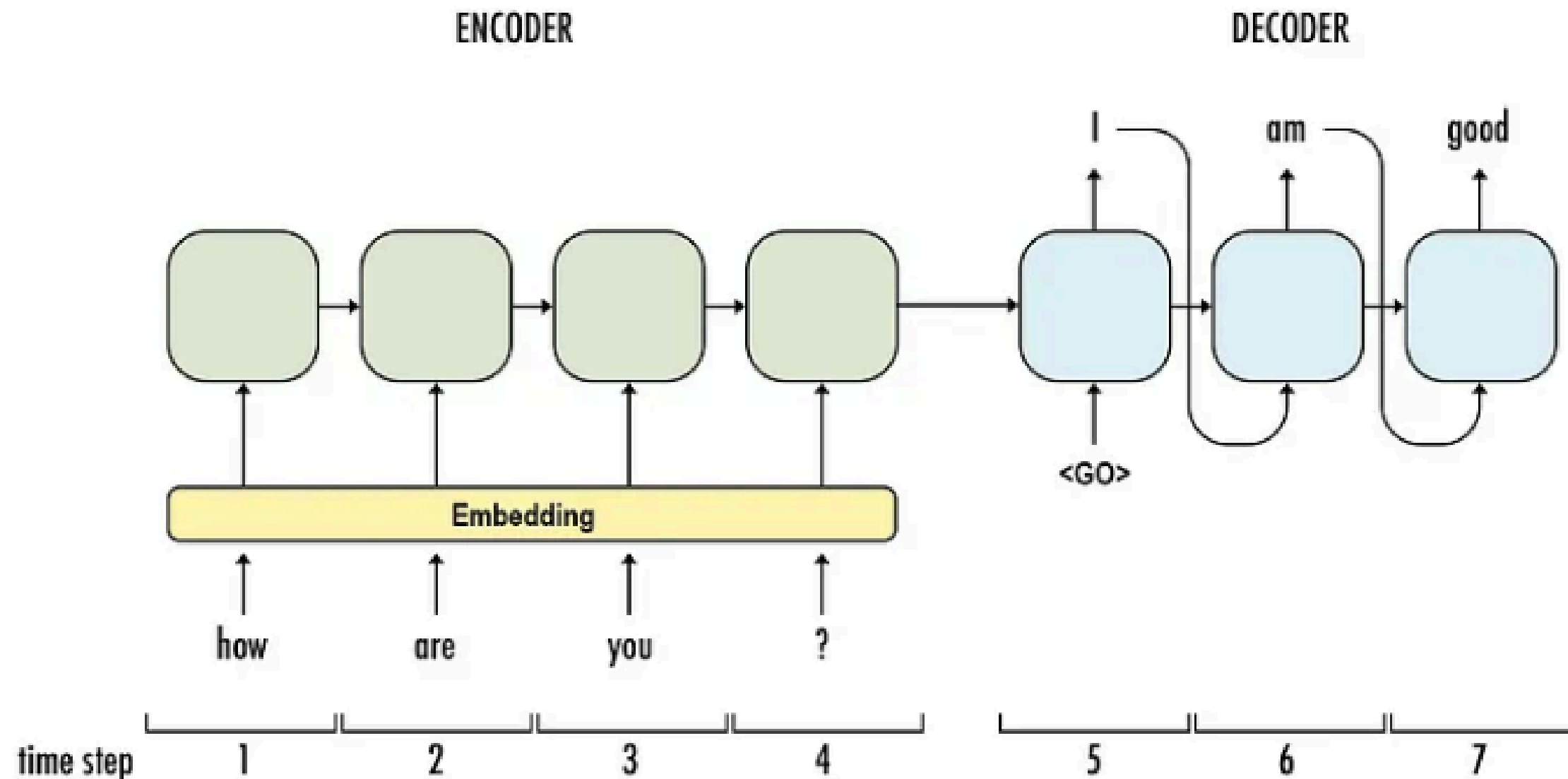
As input grows computation grows, sequential processing also we cannot parallelize the process

<https://github.com/Amr-Abdellatif>

NoteBook

<https://github.com/Amr-Abdellatif>

What we have seen so far is the basics of Seq2Seq models



<https://github.com/Amr-Abdellatif>

What we have seen so far is the basics of
Seq2Seq models

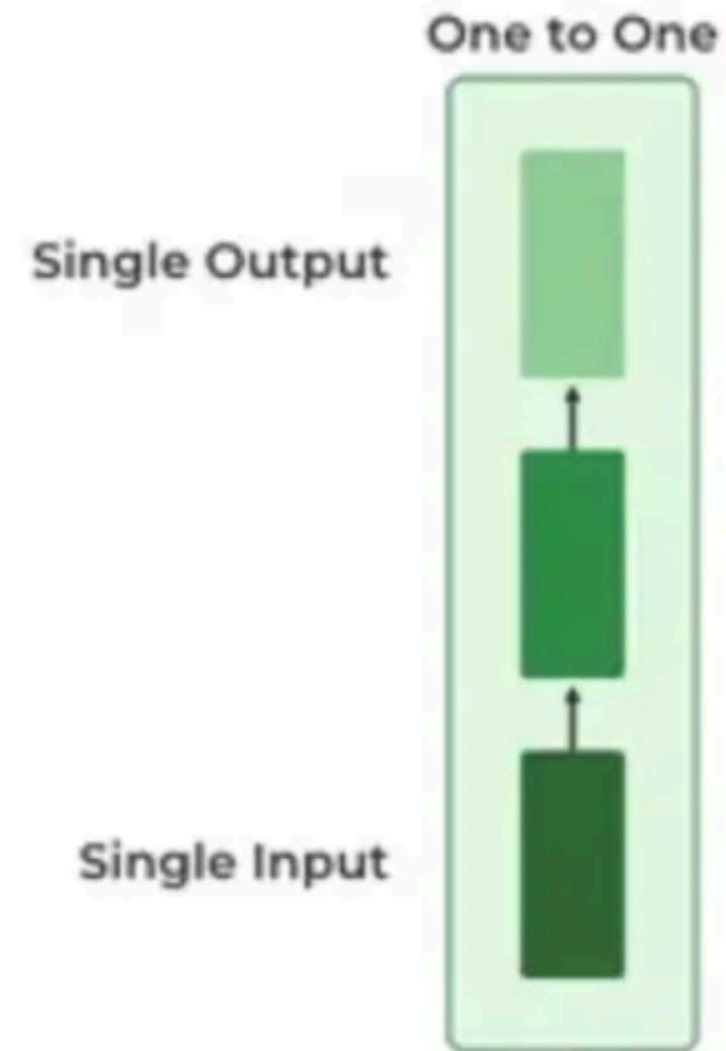
Their types as follows :

<https://github.com/Amr-Abdellatif>

What we have seen so far is the basics of Seq2Seq models

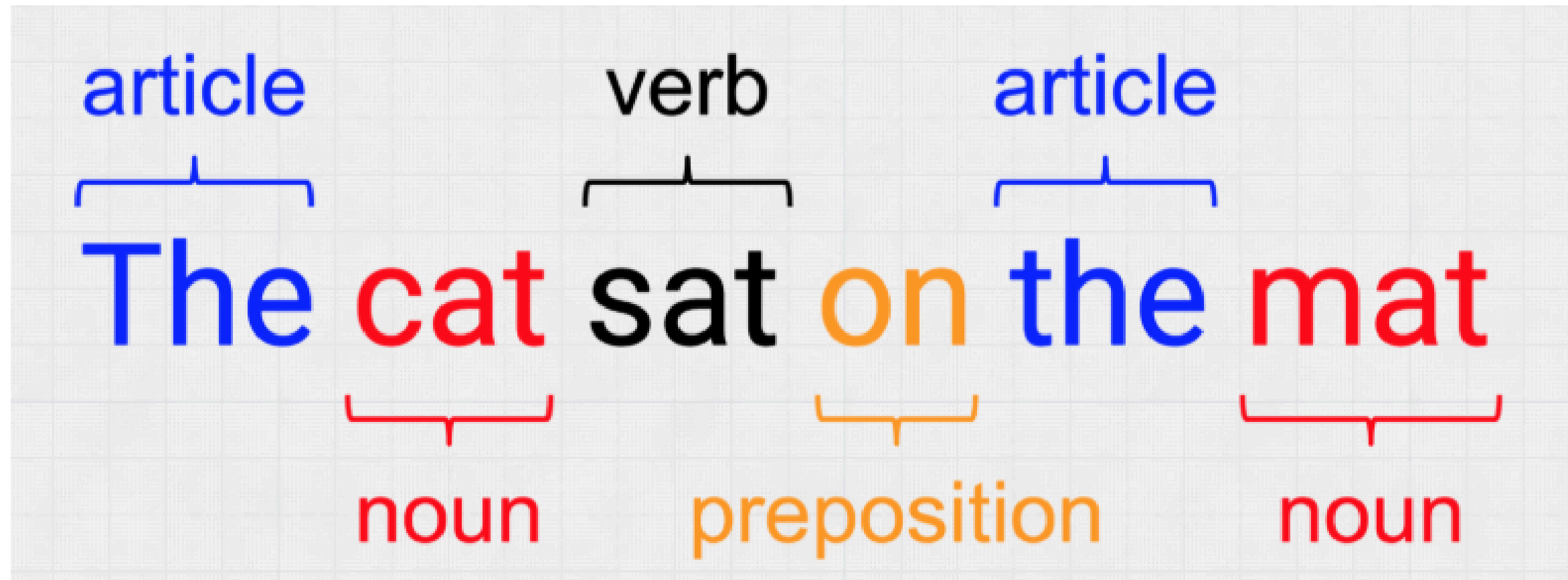
One-to-One Seq2Seq Model

ex: POS Tagging



<https://github.com/Amr-Abdellatif>

POS Tagging

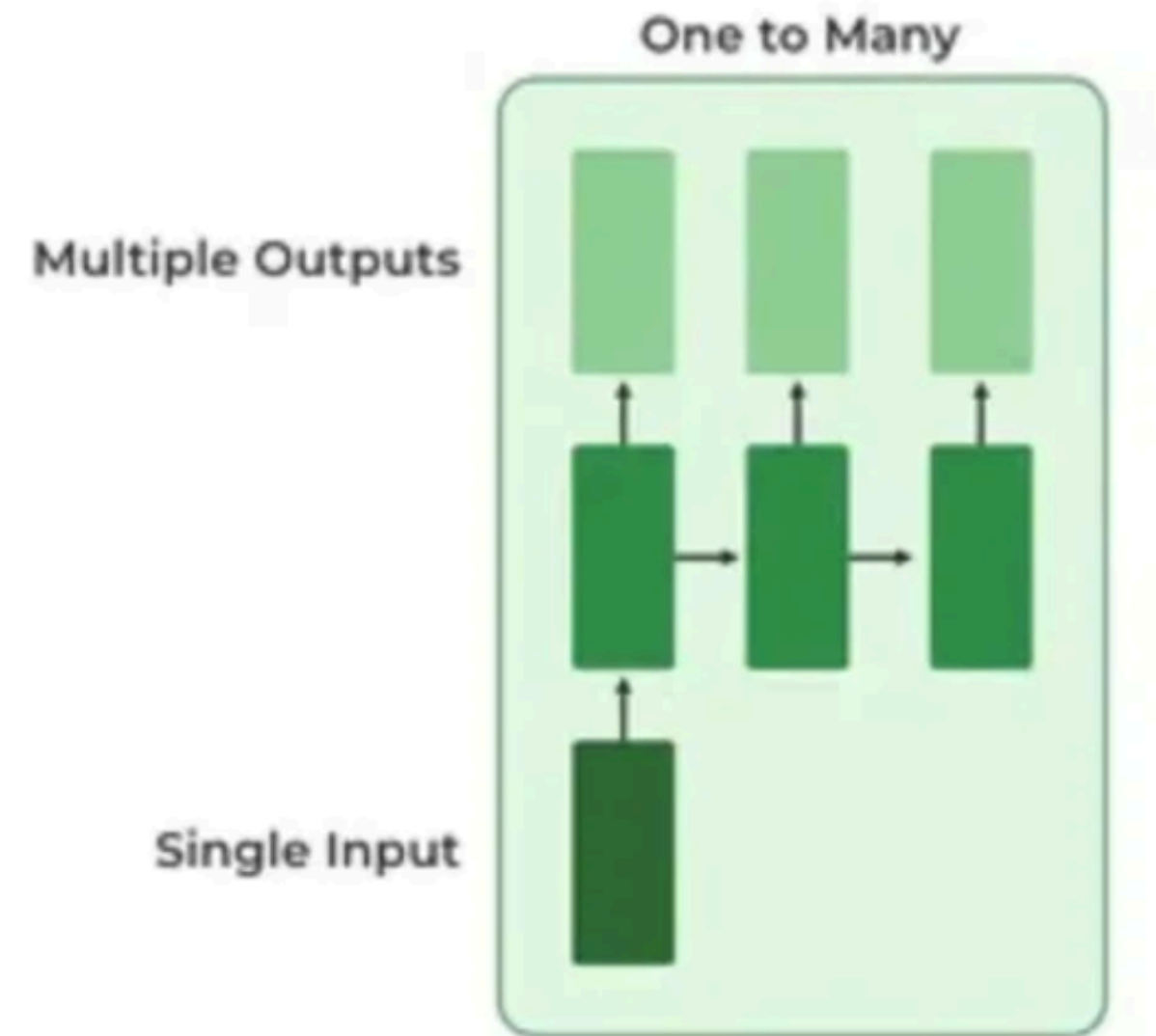


<https://github.com/Amr-Abdellatif>

What we have seen so far is the basics of Seq2Seq models

One-to-Many

ex: Image Captioning

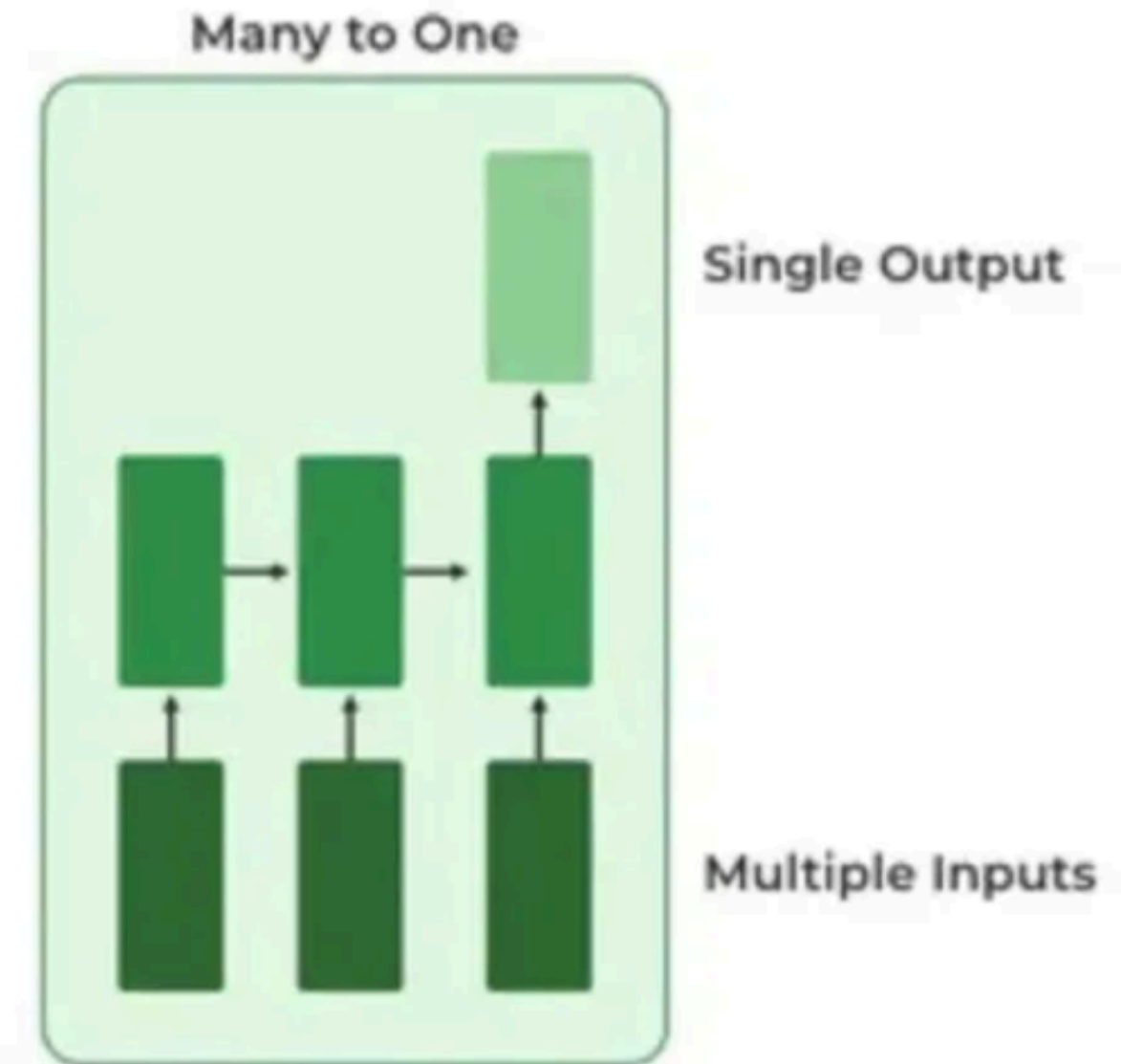


<https://github.com/Amr-Abdellatif>

What we have seen so far is the basics of Seq2Seq models

Many-to-One

ex: Sentiment Analysis

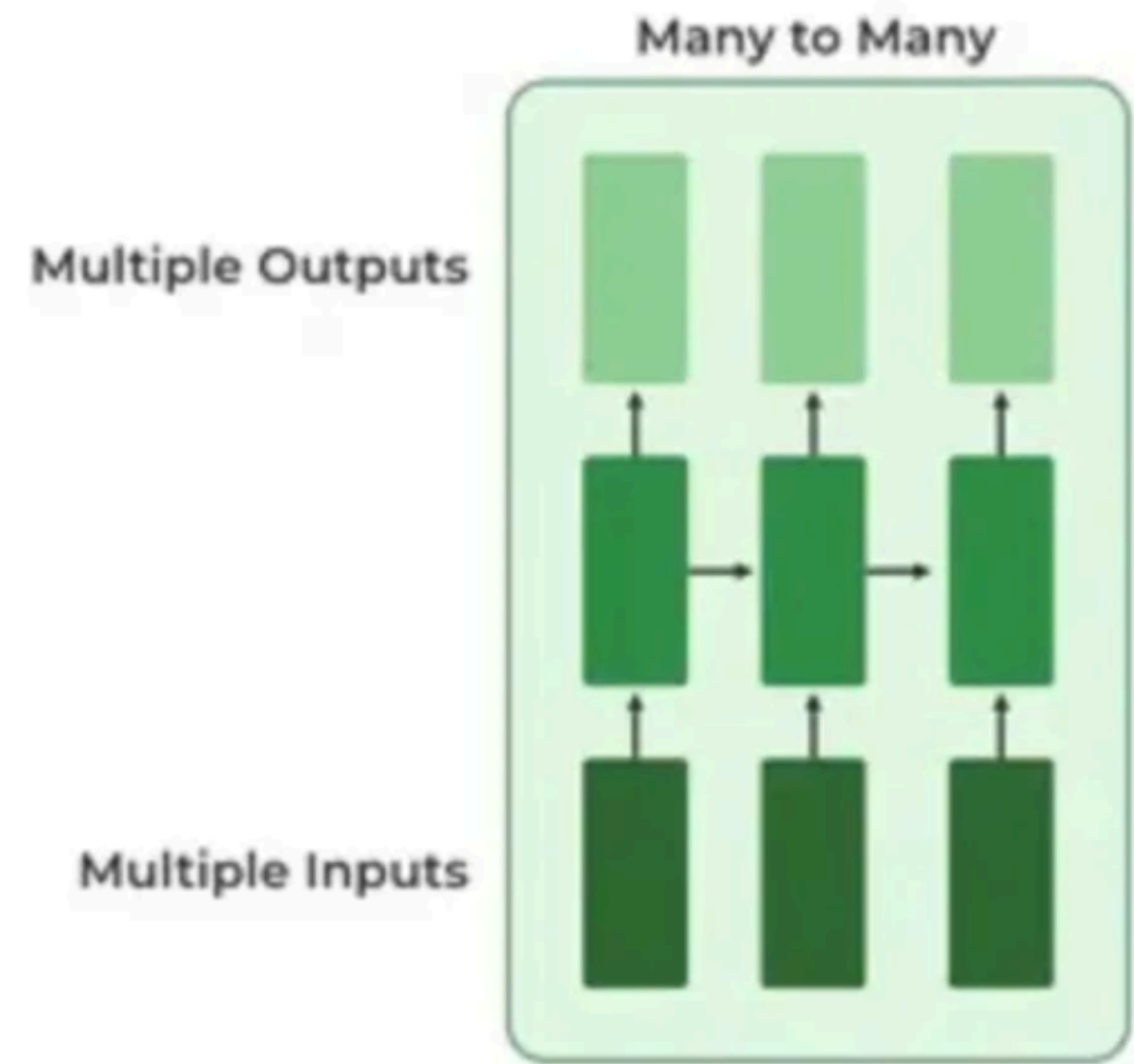


<https://github.com/Amr-Abdellatif>

What we have seen so far is the basics of Seq2Seq models

Many-to-Many

ex: Machine Translation



<https://github.com/Amr-Abdellatif>

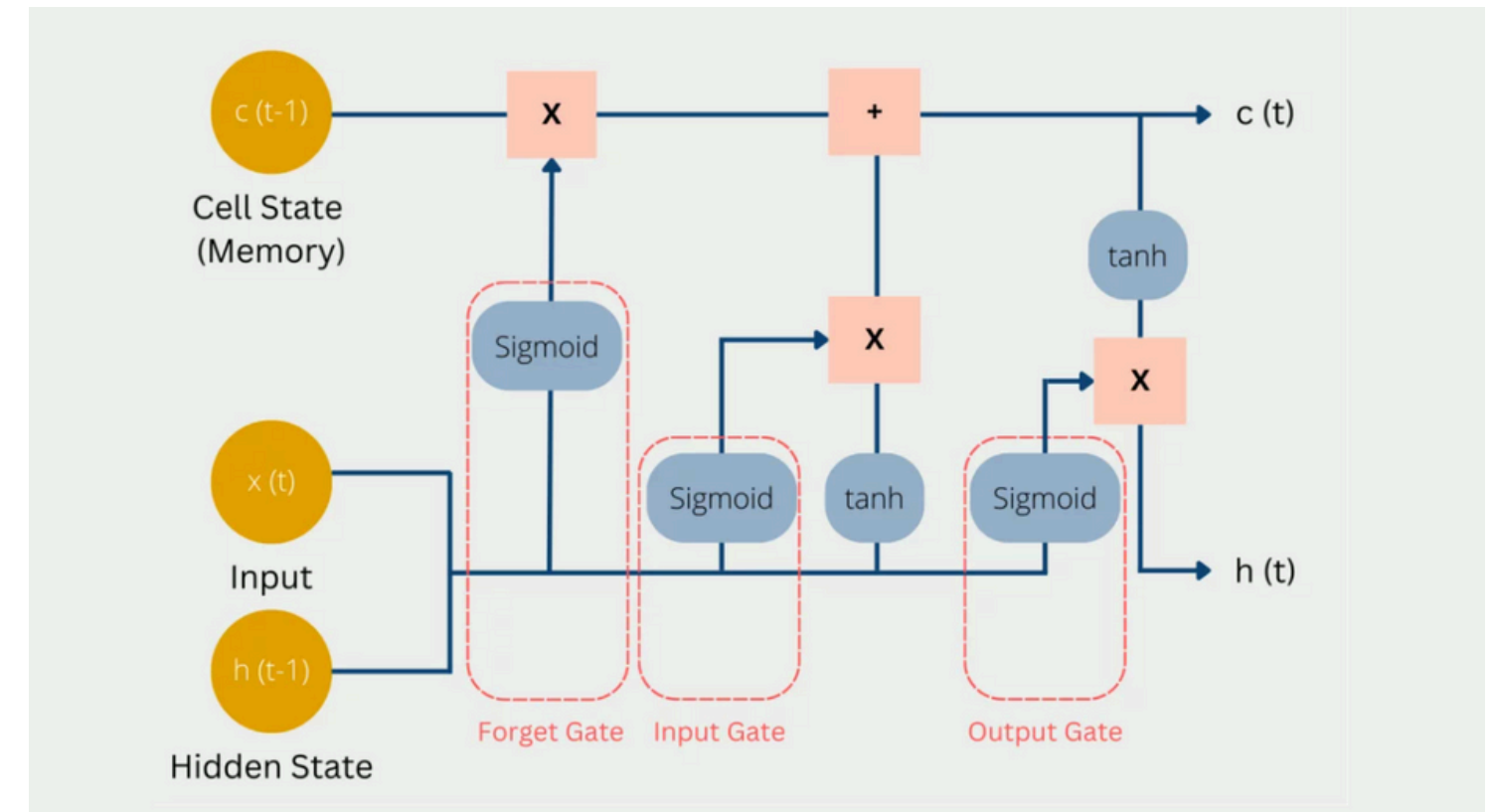
Now Let's explore more advanced RNN networks

LSTM

<https://github.com/Amr-Abdellatif>

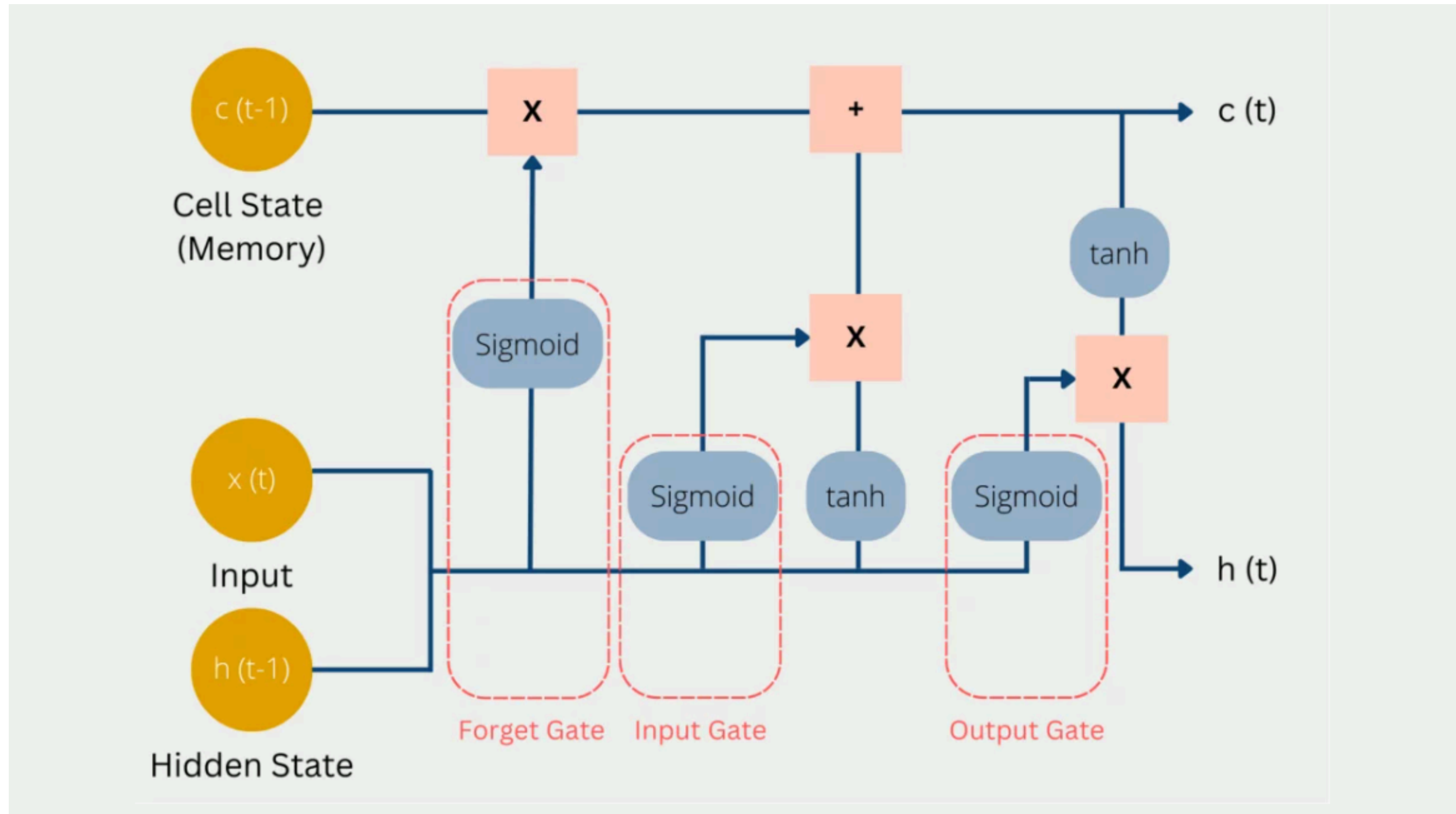
LSTM's

Long Short-Term Memory (LSTM) networks are specialized recurrent neural networks designed to overcome the vanishing gradient problem that affects standard RNNs when learning long-term dependencies.



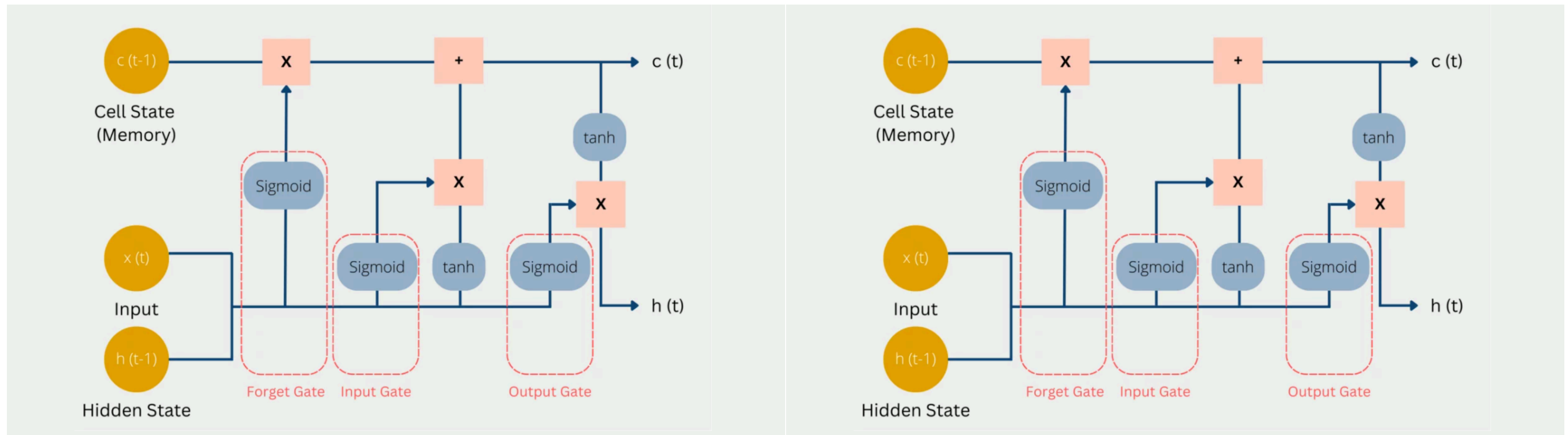
<https://github.com/Amr-Abdellatif>

LSTM's



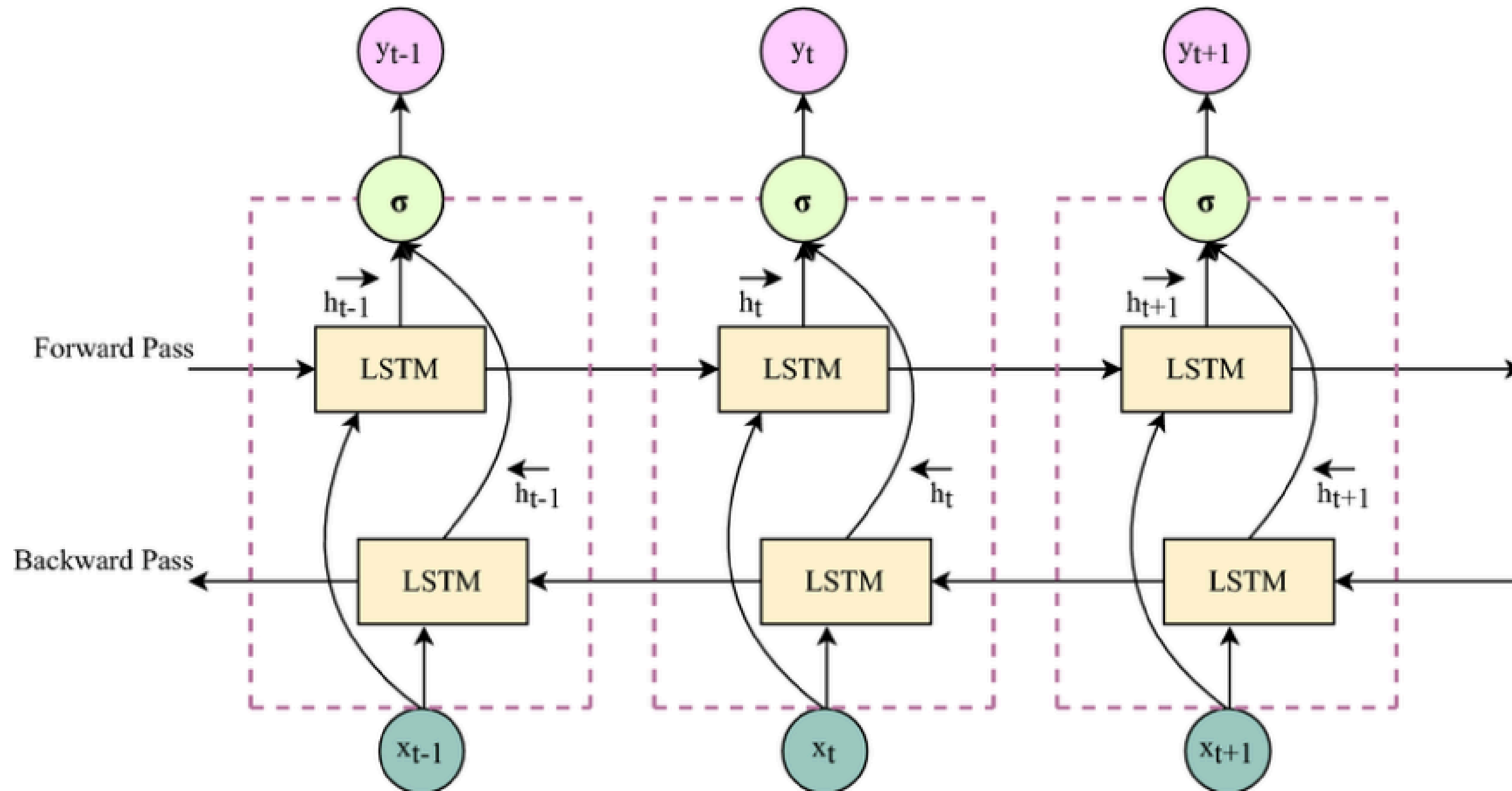
<https://github.com/Amr-Abdellatif>

LSTM's



<https://github.com/Amr-Abdellatif>

Bi Directional LSTM's

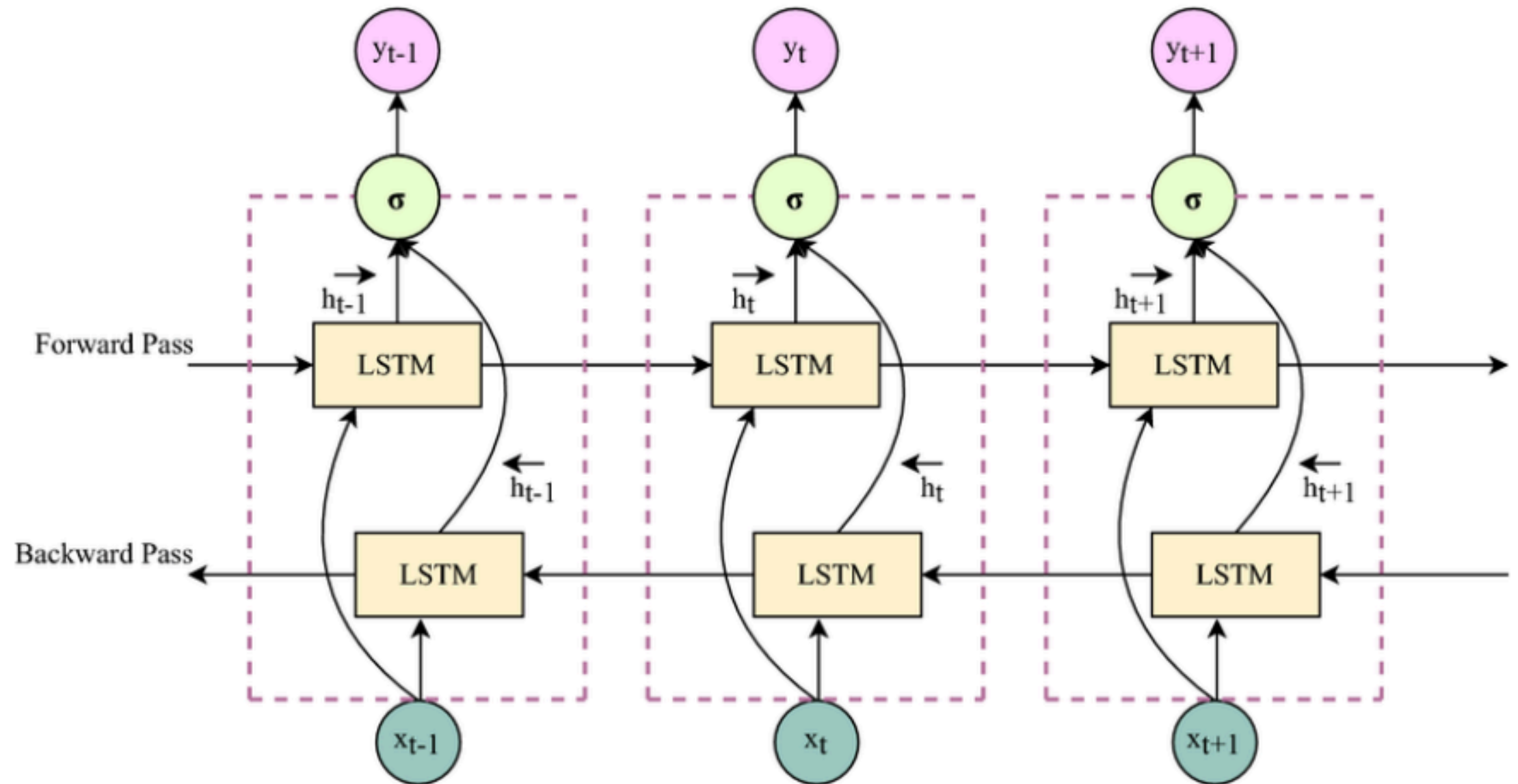


Bidirectional LSTM architecture

<https://github.com/Amr-Abdellatif>

Bi Directional LSTM's

A Bidirectional LSTM runs two separate LSTM layers in parallel



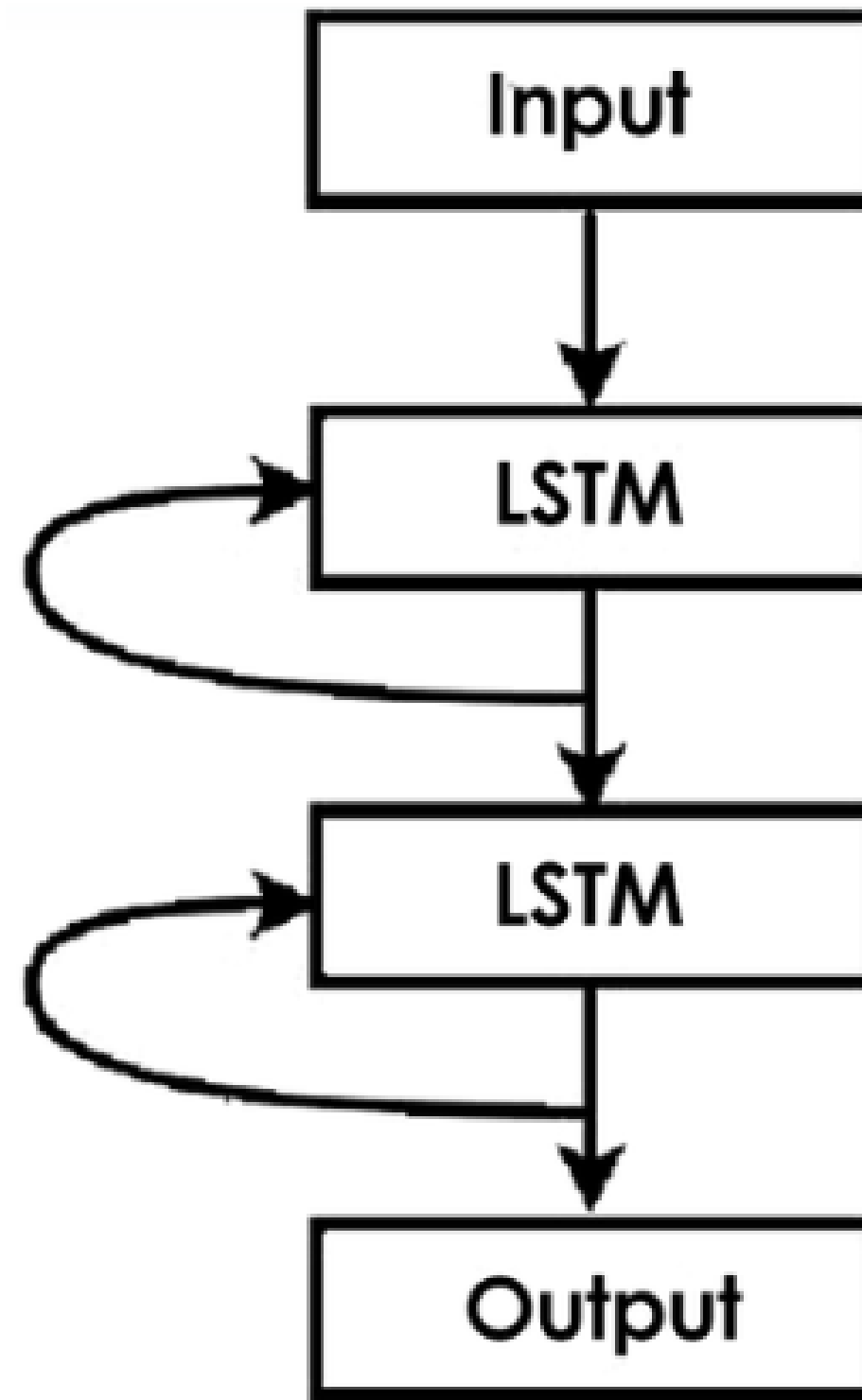
Bidirectional LSTM architecture

Can We Stack RNN Layers ?

<https://github.com/Amr-Abdellatif>

Multi Layered LSTM's

Each cell will have to wait for all previous cells in all layers, so it will get complicated pretty fast.



LSTM's

Check code

<https://github.com/Amr-Abdellatif>

GRU's

GRUs are a simplified version of LSTMs (2014)

<https://github.com/Amr-Abdellatif>

GRU's

Fewer parameters

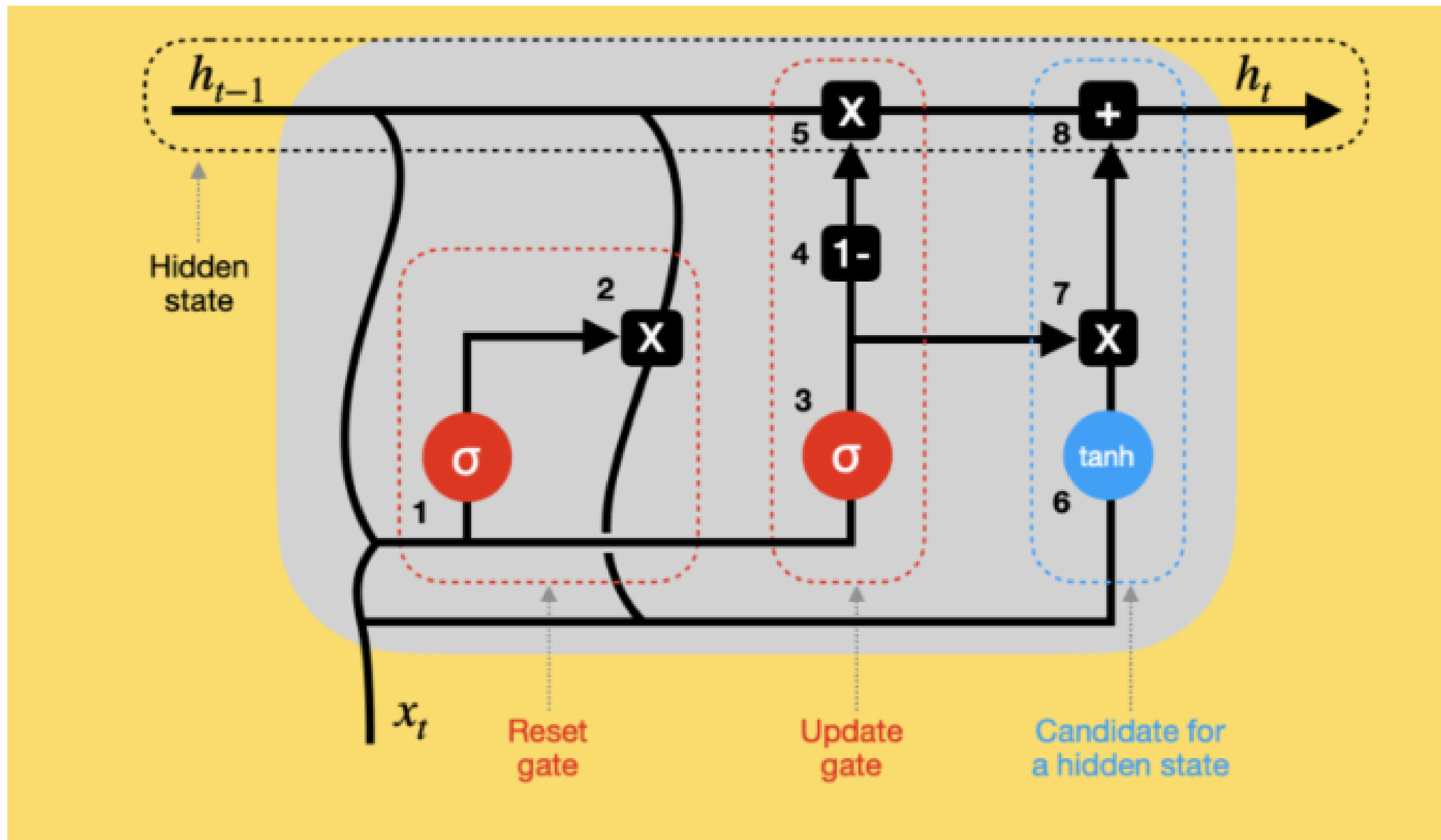
<https://github.com/Amr-Abdellatif>

GRU's

Uses reset and update gates, combines hidden state and cell state

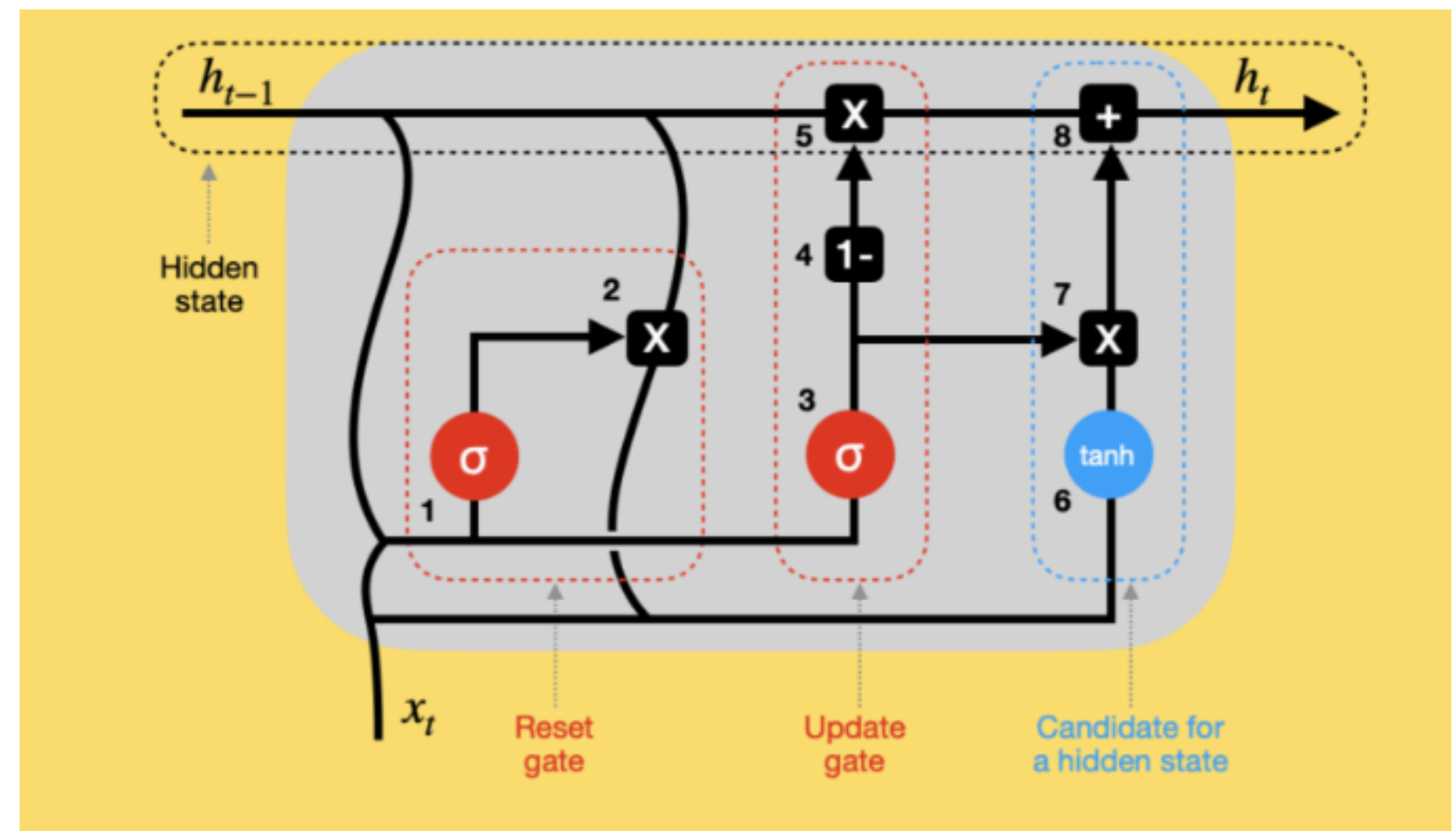
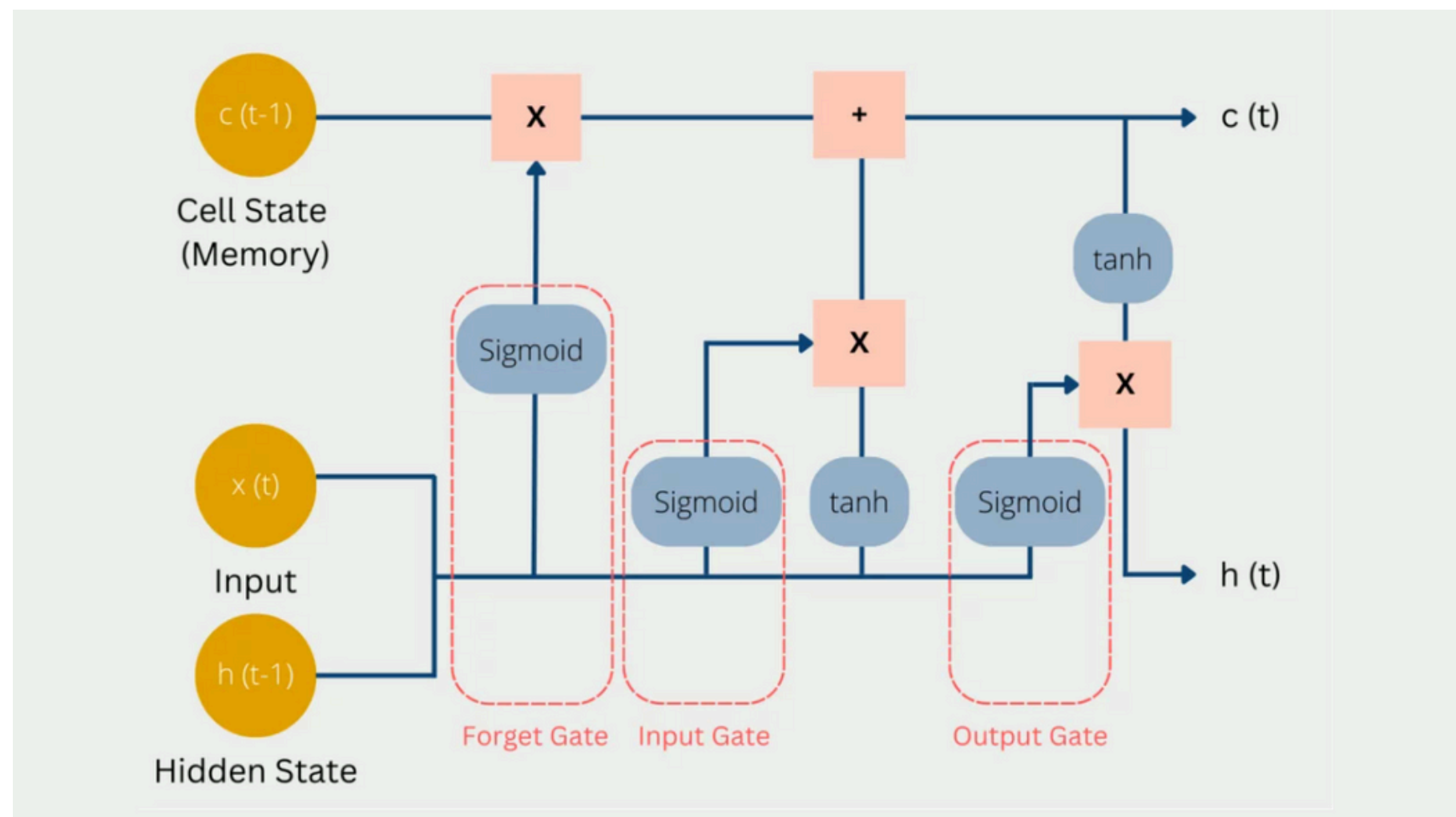
<https://github.com/Amr-Abdellatif>

GRU's



<https://github.com/Amr-Abdellatif>

GRU's



<https://github.com/Amr-Abdellatif>

GRU's

Task : Check paper

<https://arxiv.org/pdf/1412.3555>

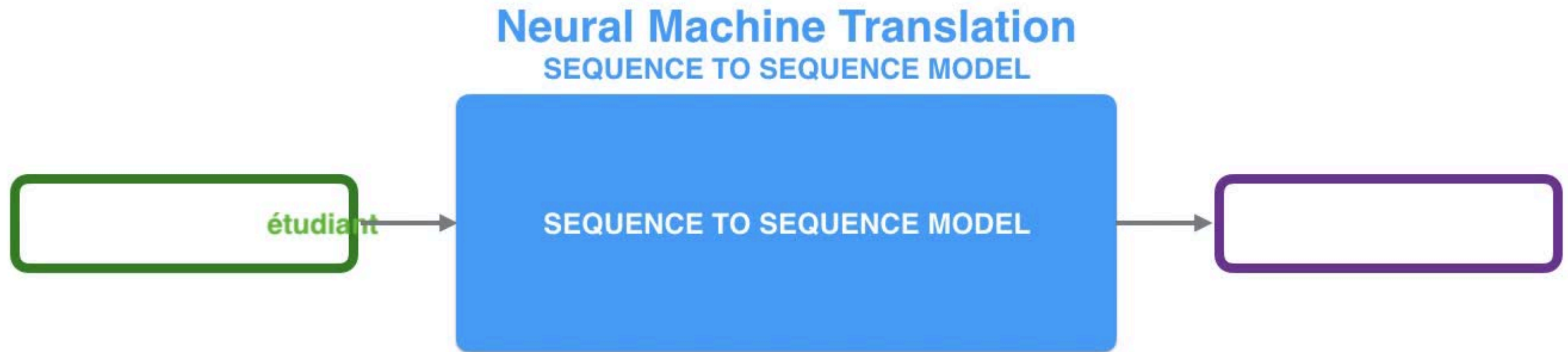
<https://github.com/Amr-Abdellatif>

Neural Machine Translation



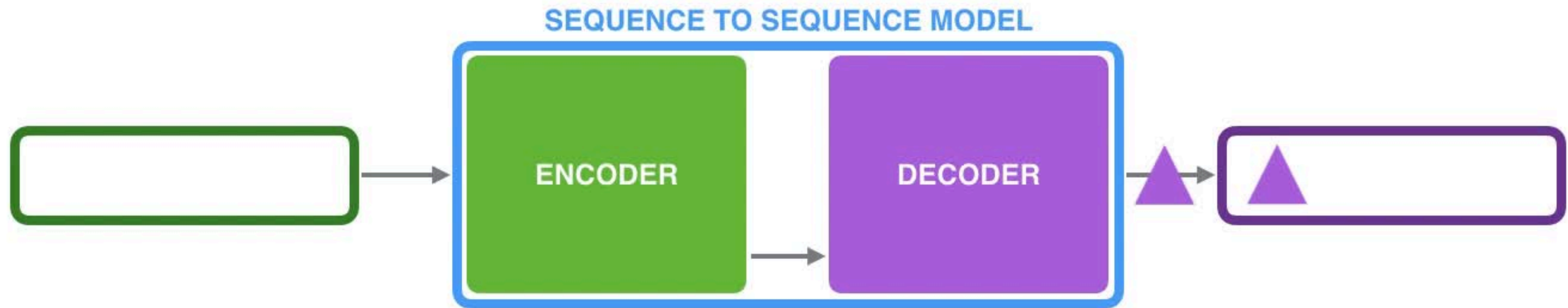
<https://github.com/Amr-Abdellatif>

NMT



<https://github.com/Amr-Abdellatif>

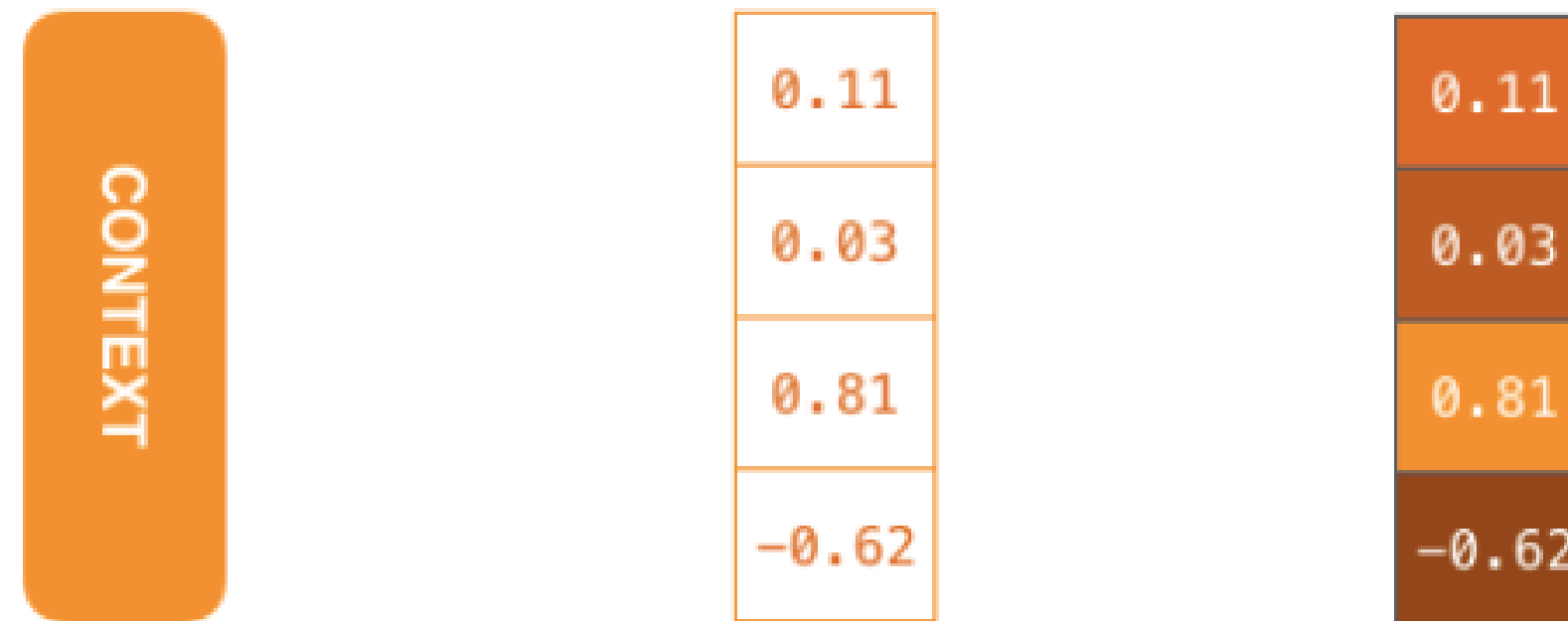
NMT



The encoder processes each item in the input sequence, it compiles the information it captures into a vector (called the context). After processing the entire input sequence, the encoder sends the context over to the decoder, which begins producing the output sequence item by item.

<https://github.com/Amr-Abdellatif>

NMT

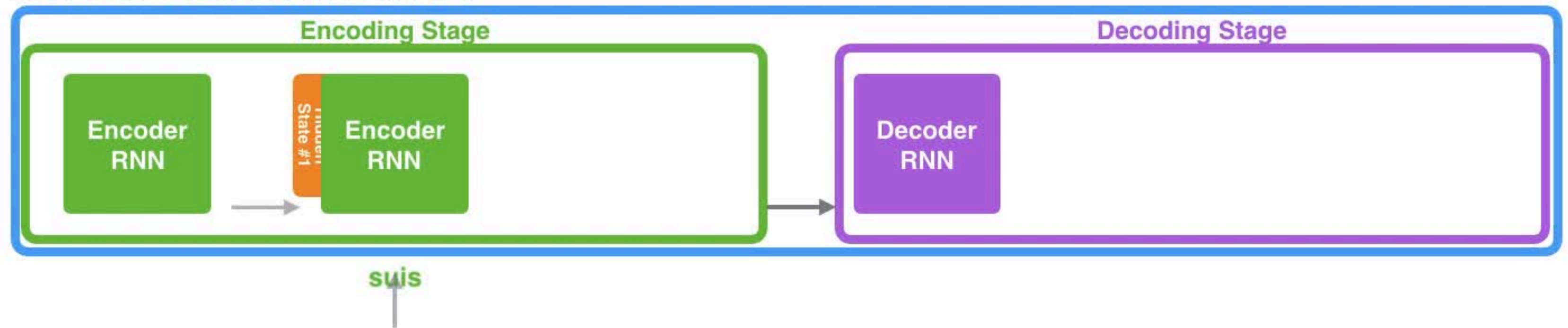


The context is a vector (an array of numbers, basically) in the case of machine translation. The encoder and decoder tend to both be recurrent neural networks

<https://github.com/Amr-Abdellatif>

NMT

Neural Machine Translation SEQUENCE TO SEQUENCE MODEL



étudiant

<https://github.com/Amr-Abdellatif>

NMT

Attention

In early sequence-to-sequence NMT models, the entire input sequence (source sentence) had to be compressed into a single fixed-length vector called the context vector. This vector was then used to generate the entire output sequence (target sentence).

<https://github.com/Amr-Abdellatif>

NMT

Attention

The context vector turned out to be a bottleneck for these types of models. It made it challenging for the models to deal with long sentences. A solution was proposed and introduced a technique called “Attention”, which highly improved the quality of machine translation systems. Attention allows the model to focus on the relevant parts of the input sequence as needed.

<https://github.com/Amr-Abdellatif>

NMT

Attention

- Information loss: Compressing an entire sentence (regardless of length) into a fixed-size vector inevitably loses information, especially for longer sequences.
- Memory constraints: The model had to somehow store all relevant information about word order, relationships, and meaning from the source sentence in this single vector.
- Declining performance with sequence length: As sentences got longer, performance degraded significantly because the fixed-size context vector couldn't adequately capture all necessary information.

<https://github.com/Amr-Abdellatif>

Key Types of Attention in NMT

1. Bahdanau Attention - The original attention mechanism introduced in 2014 that transformed NMT. It computes alignment scores between encoder hidden states and the current decoder state using a feed-forward neural network.
2. Luong Attention - Simplified attention that uses dot products between vectors instead of feed-forward networks, making it computationally more efficient.
3. Self-Attention - The breakthrough mechanism used in Transformer models, allowing the model to attend to different positions within the same sequence.

NMT

Neural Machine Translation

SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



étudiant

<https://github.com/Amr-Abdellatif>

Bahdanau Attention

Lets Translate

1. Encoding

First, we encode our input sentence:

- Input: "I love programming"
- The encoder RNN processes this word by word and generates hidden states:
 - h_1 for "I": [0.2, 0.3, 0.1, ...]
 - h_2 for "love": [0.5, 0.1, 0.4, ...]
 - h_3 for "programming": [0.3, 0.7, 0.2, ...]

<https://github.com/Amr-Abdellatif>

Bahdanau Attention

2. Decoding with Attention

When generating the first arabic word "أنا", the decoder:

- a) Takes its current state s_1 : [0.4, 0.2, 0.5, ...] generated from final hidden encoder state
- b) Computes "alignment scores" with each input word using a small neural network:
 - For "I": $\text{score}_1 = \text{NN}(s_1, h_1) = 0.8$
 - For "love": $\text{score}_2 = \text{NN}(s_1, h_2) = 0.1$
 - For "programming": $\text{score}_3 = \text{NN}(s_1, h_3) = 0.1$
- c) Converts scores to weights using softmax:
 - Weight for "I": $0.8 \rightarrow 80\%$
 - Weight for "love": $0.1 \rightarrow 10\%$
 - Weight for "programming": $0.1 \rightarrow 10\%$
- d) Creates a context vector as a weighted sum:
 - $\text{context} = 0.8h_1 + 0.1h_2 + 0.1h_3$
- e) Uses this context vector plus its current state to predict "أنا"

<https://github.com/Amr-Abdellatif>

Bahdanau Attention

Next time step

When generating "أحب" (love):

a) The decoder updates its state to s_2 as follows

- The previous decoder state (s_1)
- The previous context vector (from the attention mechanism)
- The embedding of the previously generated word ("أنا")

<https://github.com/Amr-Abdellatif>

Bahdanau Attention

b) Recomputes attention:

- For "I": $\text{score}_1 = \text{NN}(s_2, h_1) = 0.1$
- For "love": $\text{score}_2 = \text{NN}(s_2, h_2) = 0.8$
- For "programming": $\text{score}_3 = \text{NN}(s_2, h_3) = 0.1$

c) New weights:

- Weight for "I": 10%
- Weight for "love": 80%
- Weight for "programming": 10%

d) New context vector focusing mostly on "love"

e) Predicts "أحب"

<https://github.com/Amr-Abdellatif>

Luong Attention

1. Encoding

Just like before, we encode our input sentence:

- Input: "I love programming"
- The encoder RNN generates hidden states:
 - h_1 for "I": [0.2, 0.3, 0.1, ...]
 - h_2 for "love": [0.5, 0.1, 0.4, ...]
 - h_3 for "programming": [0.3, 0.7, 0.2, ...]

Luong Attention

2. Decoding with Luong Attention

When generating the first arabic word "أ", the decoder:

- a) Processes the previous output and produces its current state s_1 : [0.4, 0.2, 0.5, ...]
- b) Computes alignment scores using dot product (or one of Luong's three scoring functions):
 - For "أ": $\text{score}_1 = s_1 \cdot h_1 = 0.4 \times 0.2 + 0.2 \times 0.3 + 0.5 \times 0.1 + \dots = 0.8$
 - For "love": $\text{score}_2 = s_1 \cdot h_2 = 0.4 \times 0.5 + 0.2 \times 0.1 + 0.5 \times 0.4 + \dots = 0.1$
 - For "programming": $\text{score}_3 = s_1 \cdot h_3 = 0.4 \times 0.3 + 0.2 \times 0.7 + 0.5 \times 0.2 + \dots = 0.1$
- c) Converts scores to weights using softmax:
 - Weight for "أ": $0.8 \rightarrow 80\%$
 - Weight for "love": $0.1 \rightarrow 10\%$
 - Weight for "programming": $0.1 \rightarrow 10\%$
- d) Creates a context vector as a weighted sum:
 - $\text{context} = 0.8 \times h_1 + 0.1 \times h_2 + 0.1 \times h_3$
- e) Combines the context with the current state to form an "attentional hidden state":
 - $\tilde{h}_1 = \tanh(W[s_1; \text{context}])$
- f) Uses this attentional hidden state to predict "أ"

<https://github.com/Amr-Abdellatif>

NMT

Task

Papers

<https://arxiv.org/abs/1409.0473>

<https://arxiv.org/abs/1508.04025>

<https://github.com/Amr-Abdellatif>

Metrics

<https://github.com/Amr-Abdellatif>

BLEU (BiLingual Evaluation Understudy)

Compare predicted translation (candidate) to the reference one
Ranges from 0 to 1, 1 is the best.

<https://github.com/Amr-Abdellatif>

BLEU (BiLingual Evaluation Understudy)

Translated sentence : الشمس مشرقة

Reference: The sun is shining.

Candidate: The sun is in the sky.

$$BLEU-1 = \frac{\sum \text{red box}}{\sum \text{blue box}} = \frac{3}{6} = 0.5$$

	the	is	sun	in	shinning	sky
min(count candidate, count reference)	1	1	1	0	0	0
Count candidate	2	1	1	1	0	1

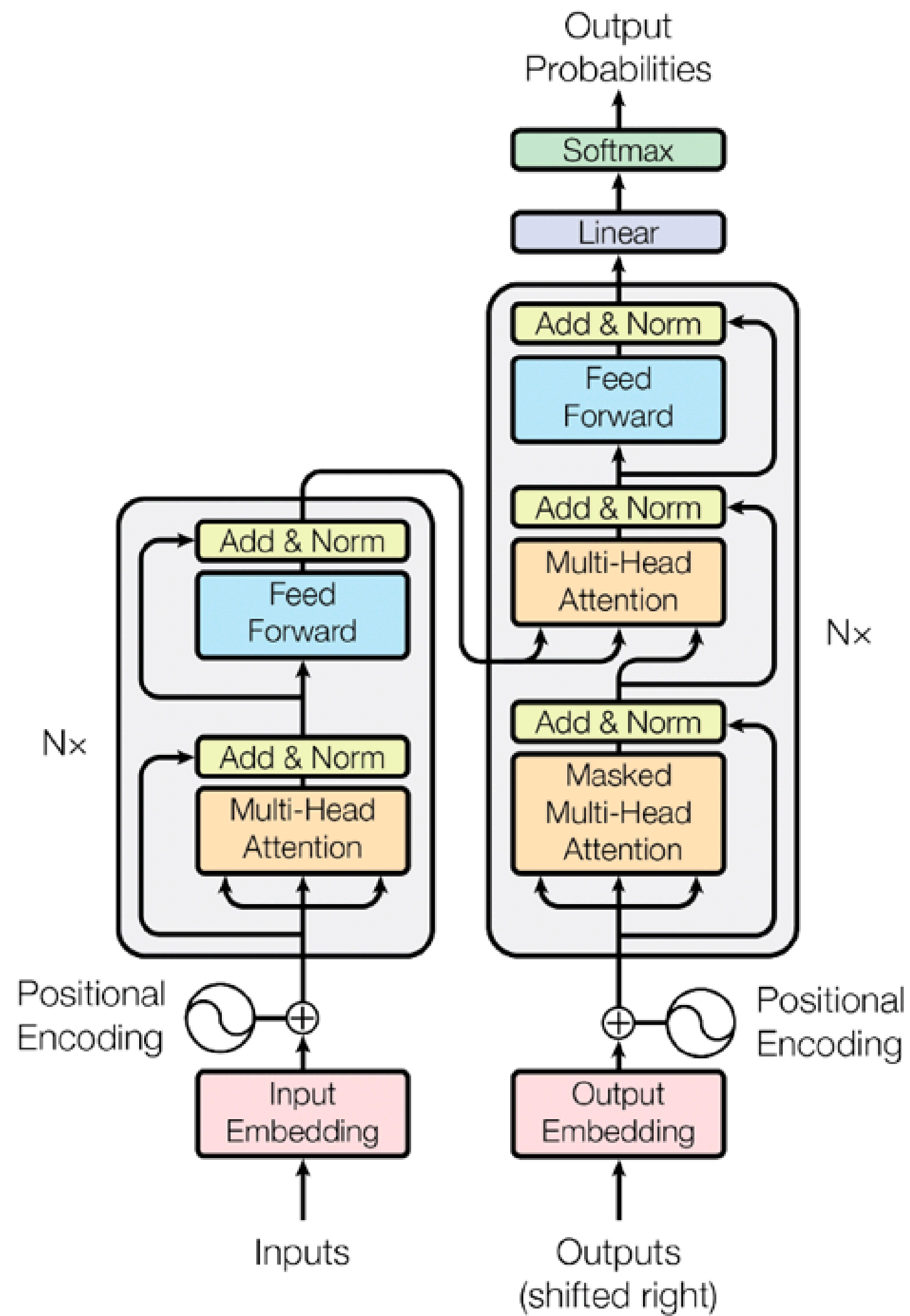
N-rouge

Task

make a search about this metric and deliver a .txt file with your findings

<https://github.com/Amr-Abdellatif>

Transformers



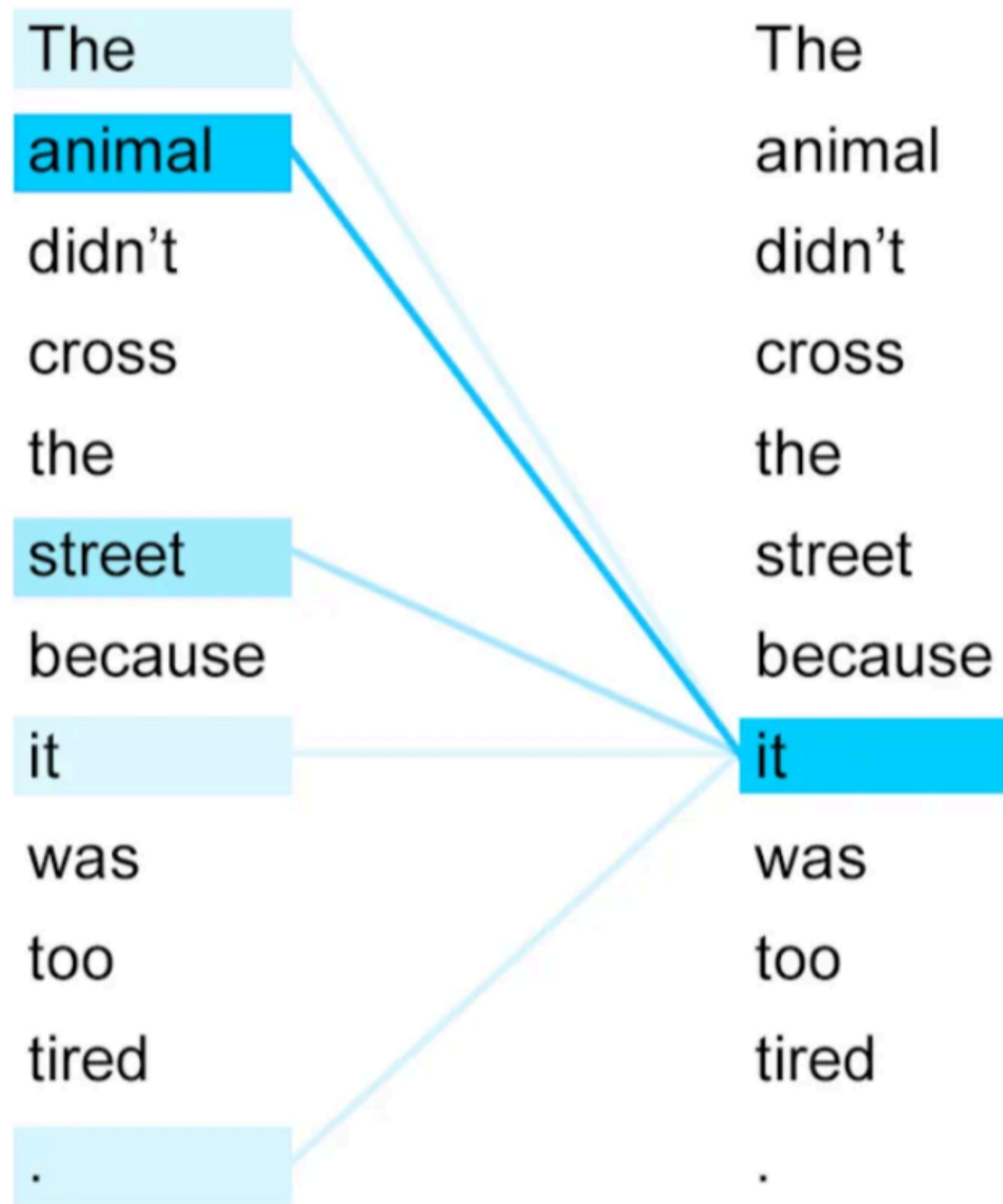
What is a transformer

- Transformer model was introduced by Google in the very famous paper Attention is all you need.
- The model is built based on attention and got rid of RNNs.
- Transformer model reshaped the NLP industry as it enabled a way to optimize large model by parallelizing model operations and making use of GPUs/TPUs.

But First, SelfAttention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Self Attention



The magic of self-attention is that it creates context-aware representations. Words can mean different things in different contexts, and self-attention helps capture these relationships dynamically.

Self Attention

The model calculates three vectors for each word:

- Query: "What am I looking for?"
- Key: "What do I contain?"
- Value: "What information do I provide?"

<https://github.com/Amr-Abdellatif>

Self Attention

Let's imagine a simple sentence: "The cat sat on the mat"

<https://github.com/Amr-Abdellatif>

Self Attention

Step 1: Word Embeddings

- "The" $\rightarrow [1, 2, 0, 1]$
- "cat" $\rightarrow [5, 3, 2, 1]$
- "sat" $\rightarrow [4, 1, 3, 2]$
- "on" $\rightarrow [2, 2, 1, 0]$
- "the" $\rightarrow [1, 2, 0, 1]$
- "mat" $\rightarrow [3, 2, 4, 1]$

Self Attention

Step 2: Create Query, Key, and Value Vectors

For each word, we create three new vectors through three different transformation matrices:

Query (Q): "What am I looking for?"

Key (K): "What information do I contain?"

Value (V): "What information do I provide?"

Let's say after these transformations:

"cat" Q-vector = [2, 1]

"cat" K-vector = [3, 2]

"cat" V-vector = [7, 8]

<https://github.com/Amr-Abdellatif>

Self Attention

Step 3: Calculate Attention Scores

Let's focus on "cat" and calculate how much it should attend to each word:

For "cat" attending to "mat":

- "cat" Q-vector = [2, 1]
- "mat" K-vector = [4, 3]
- Attention score = dot product = $2 \times 4 + 1 \times 3 = 11$

We calculate this for every word pair in the sentence.

Self Attention

Step 4: Scale and Apply Softmax

Let's say after calculating all scores for "cat" looking at each word:

- "cat" → "The": 5
- "cat" → "cat": 8
- "cat" → "sat": 9
- "cat" → "on": 3
- "cat" → "the": 5
- "cat" → "mat": 11

We scale these by (dividing by $\sqrt{d_k}$) and apply softmax to get probabilities:

- "The": 0.05
- "cat": 0.15
- "sat": 0.20
- "on": 0.02
- "the": 0.05
- "mat": 0.53

This means "cat" attends most strongly to "mat" in this context.

<https://github.com/Amr-Abdellatif>

Self Attention

Step 5: Create Weighted Values

Multiply each word's V-vector by its attention probability:

- "The" V-vector $\times$ 0.05
- "cat" V-vector $\times$ 0.15
- ...etc.

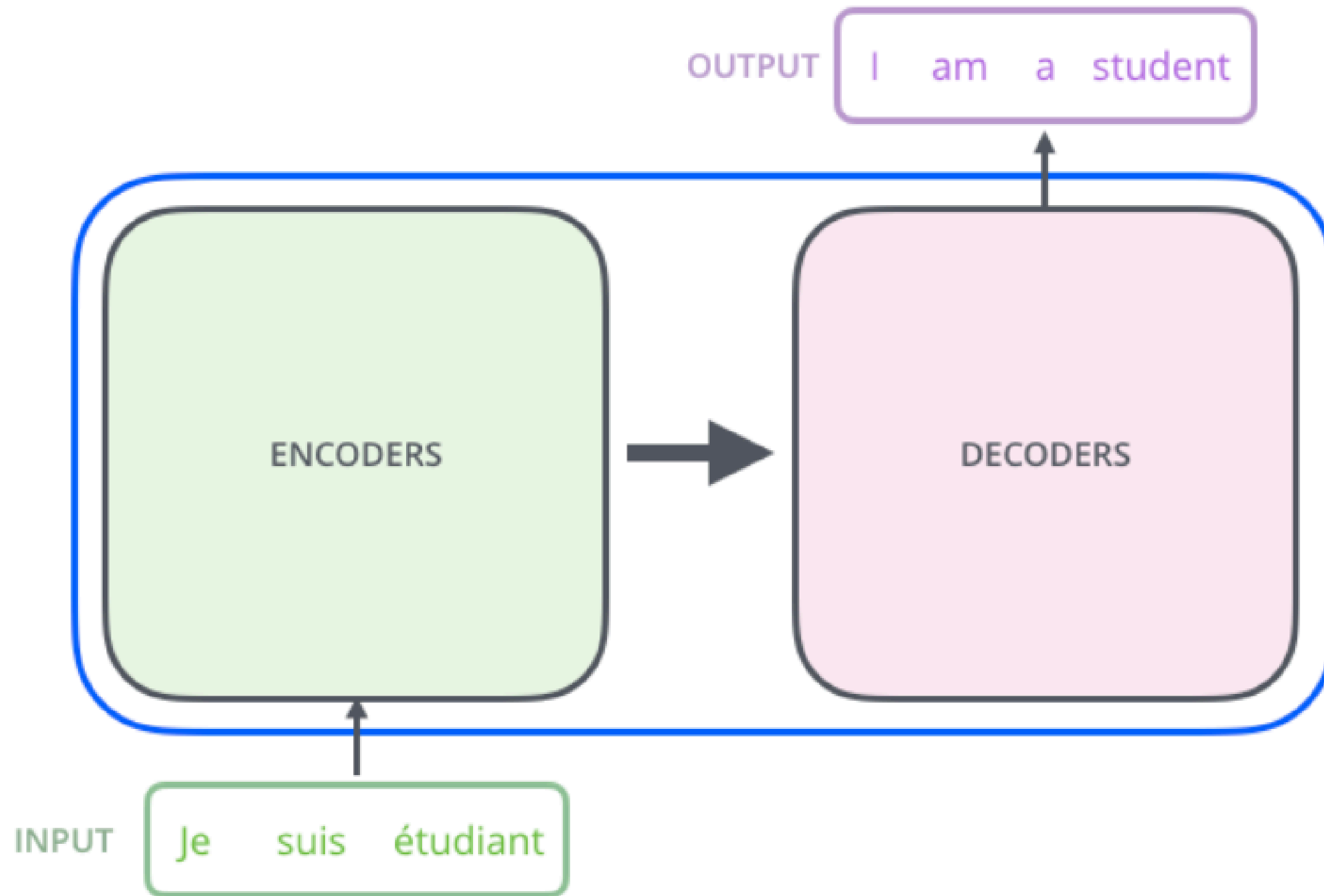
Self Attention

Step 6: Sum Them Up

Add all these weighted vectors to get the final representation for "cat"

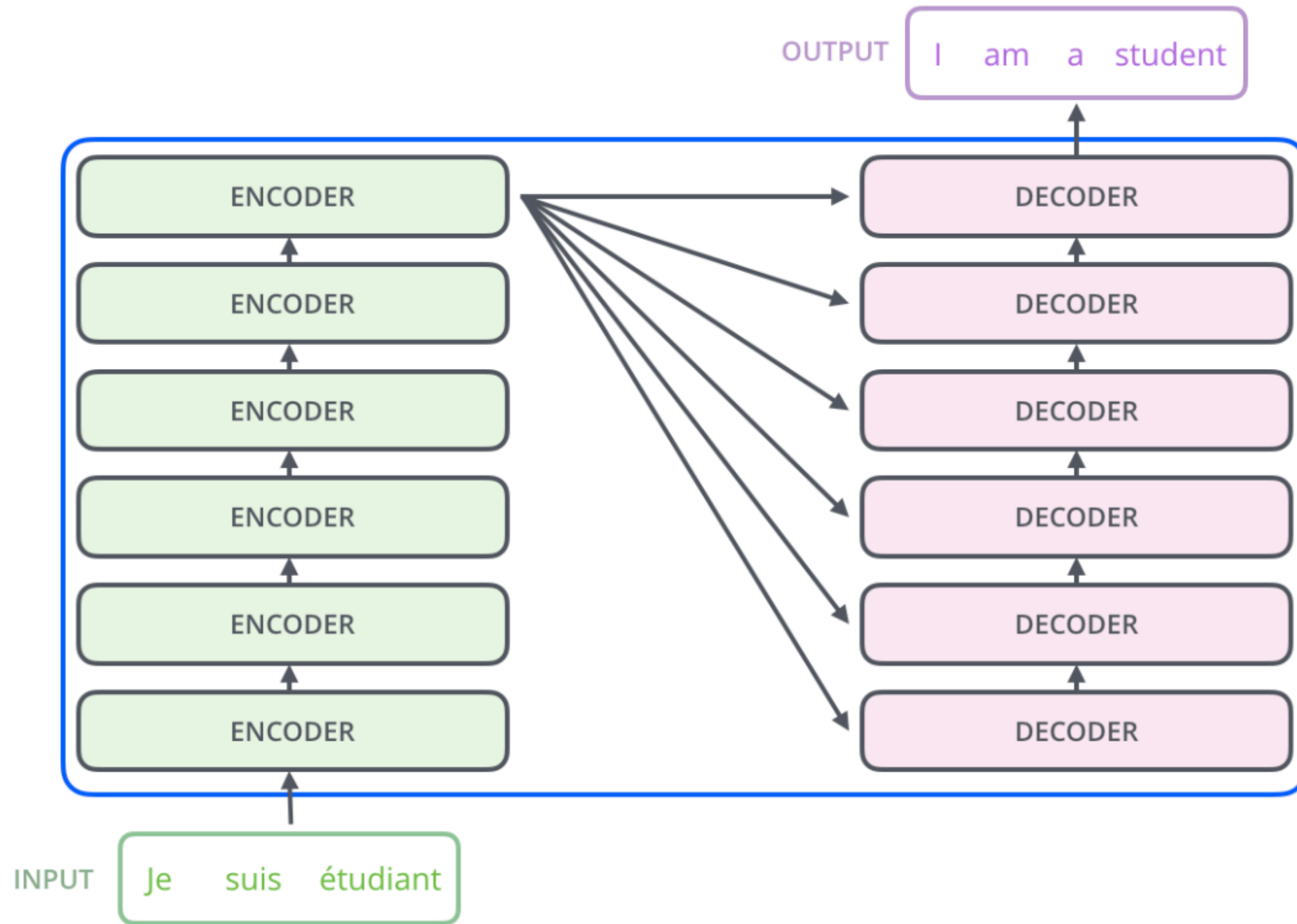
<https://github.com/Amr-Abdellatif>

Transformers



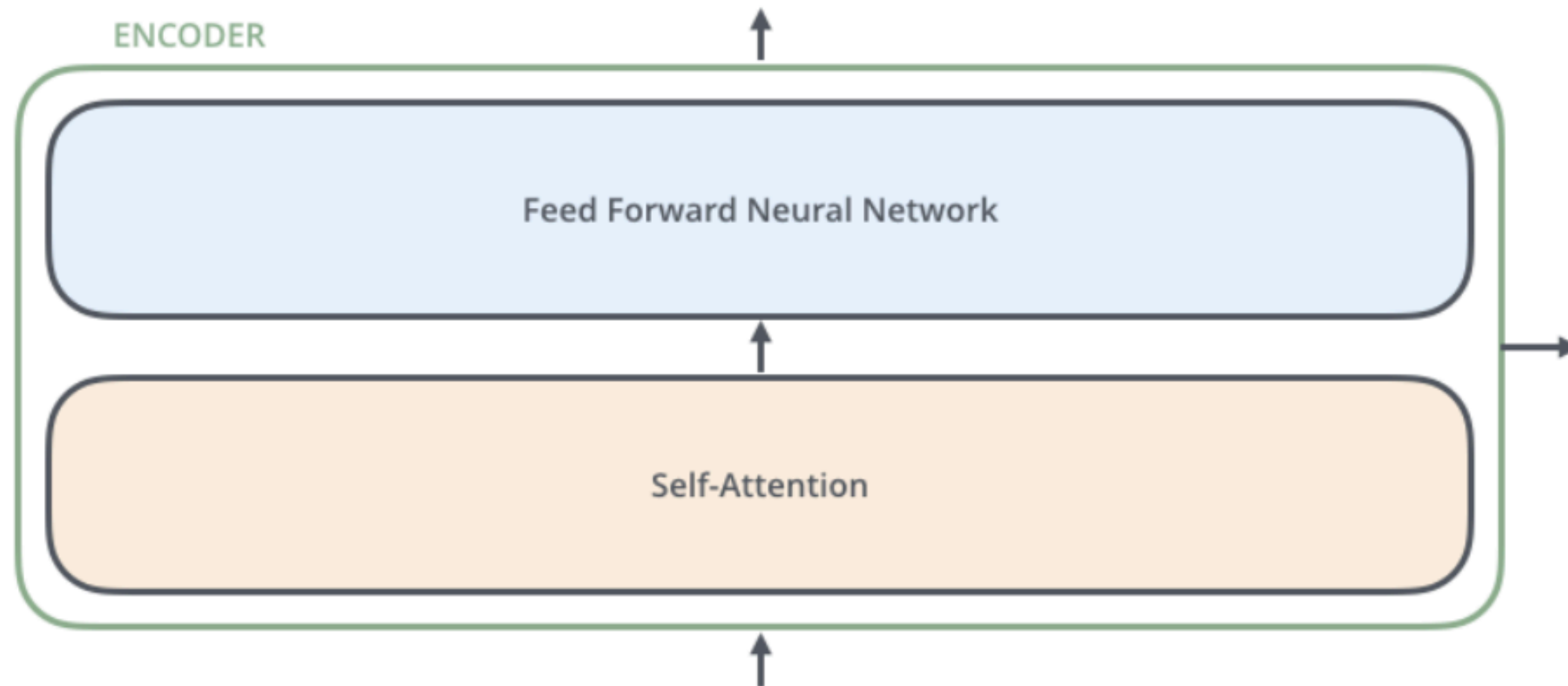
<https://github.com/Amr-Abdellatif>

Transformers



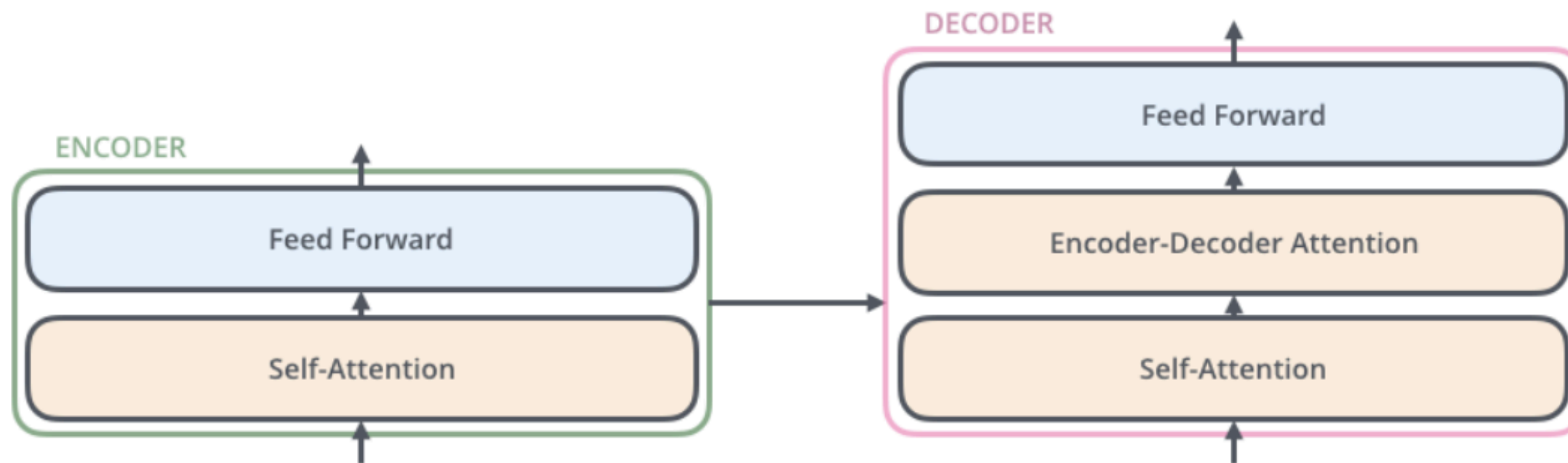
- The encoding component is a stack of encoders (the paper stacks six of them on top of each other – there’s nothing magical about the number six, one can definitely experiment with other arrangements). The decoding component is a stack of decoders of the same number.

Transformers



- The encoders are all identical in structure (yet they do not share weights). Each one is broken down into two sub-layers:

Transformers



- The encoder's inputs first flow through a self-attention layer – a layer that helps the encoder look at other words in the input sentence as it encodes a specific word. We'll look closer at self-attention later in the post.
- The outputs of the self-attention layer are fed to a feed-forward neural network. The exact same feed-forward network is independently applied to each position.

Transformers



Je



suis



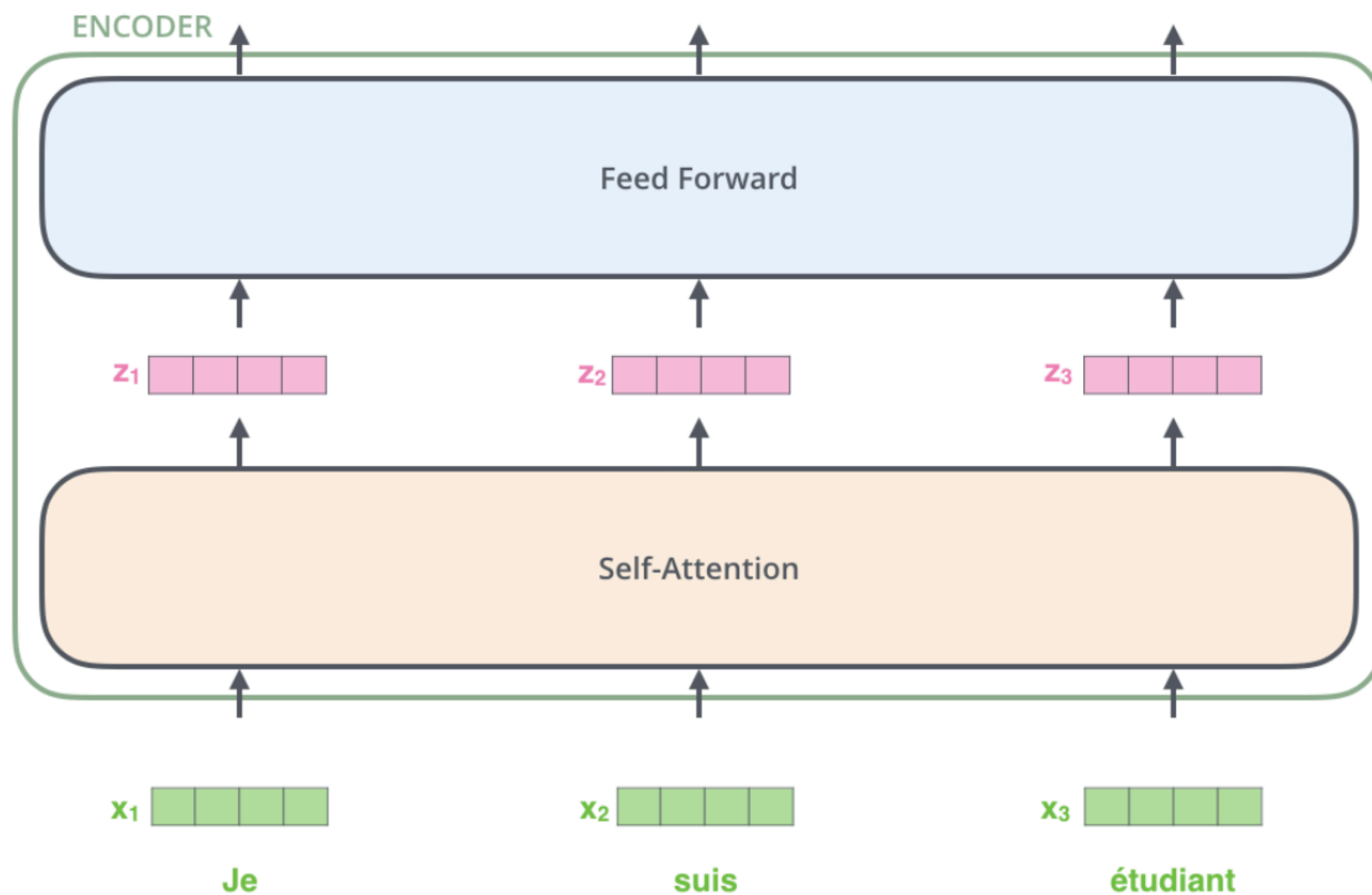
étudiant

Each word is embedded into a vector of size 512. We'll represent those vectors with these simple boxes.

- we begin by turning each input word into a vector using an Embedding layer.

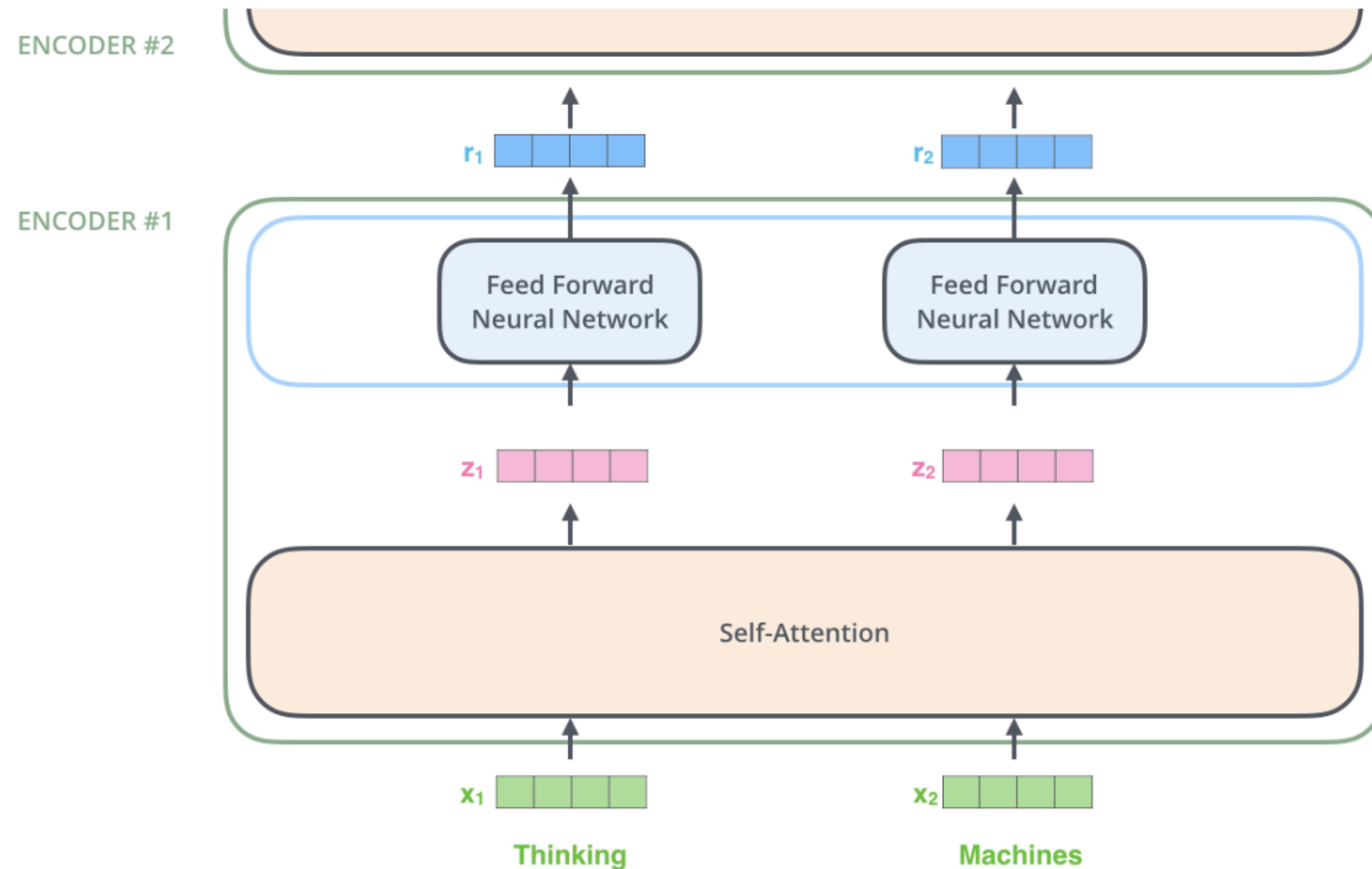
<https://github.com/Amr-Abdellatif>

Transformers



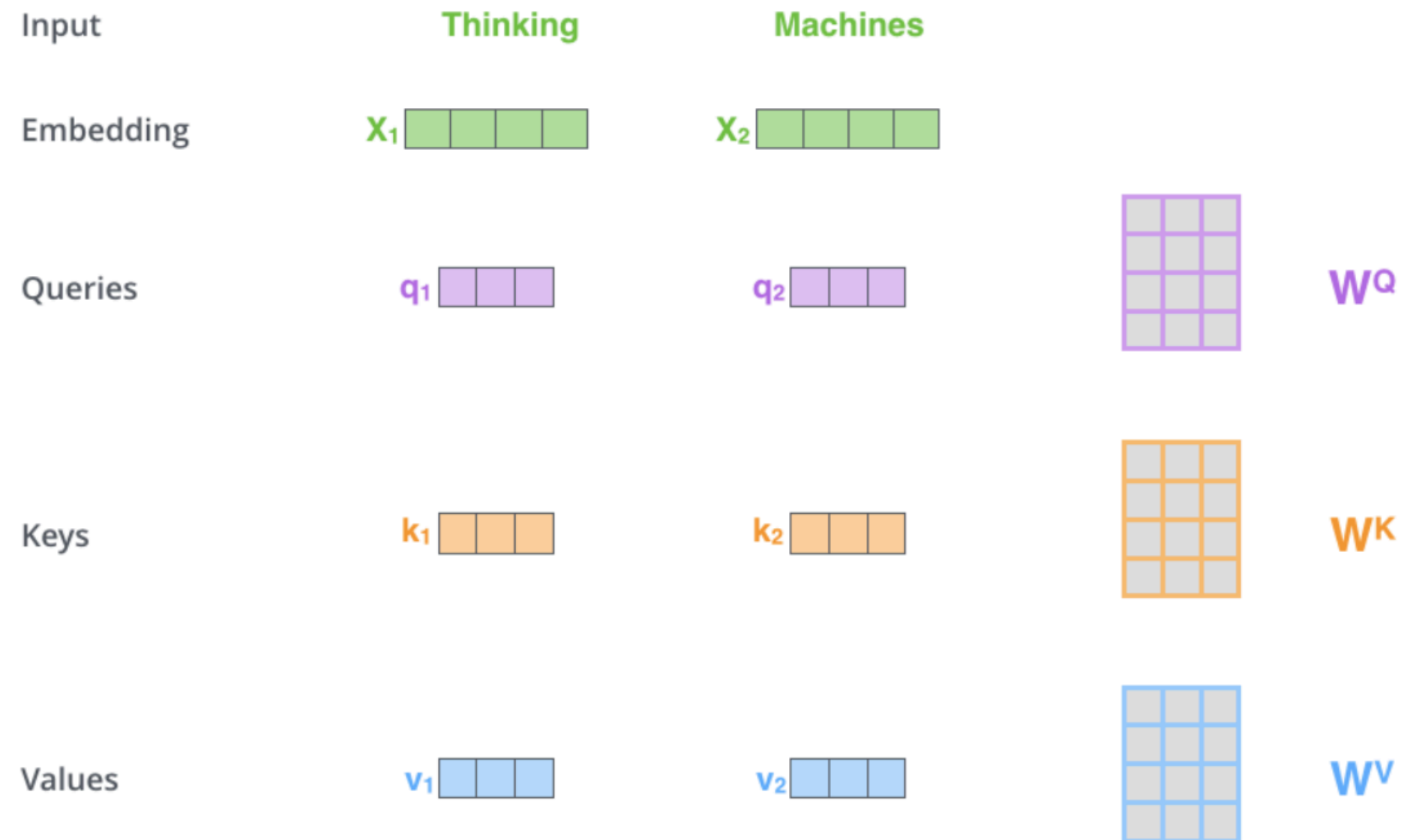
<https://github.com/Amr-Abdellatif>

Transformers



- an encoder receives a list of vectors as input. It processes this list by passing these vectors into a 'self-attention' layer, then into a feed-forward neural network, then sends out the output upwards to the next encoder.

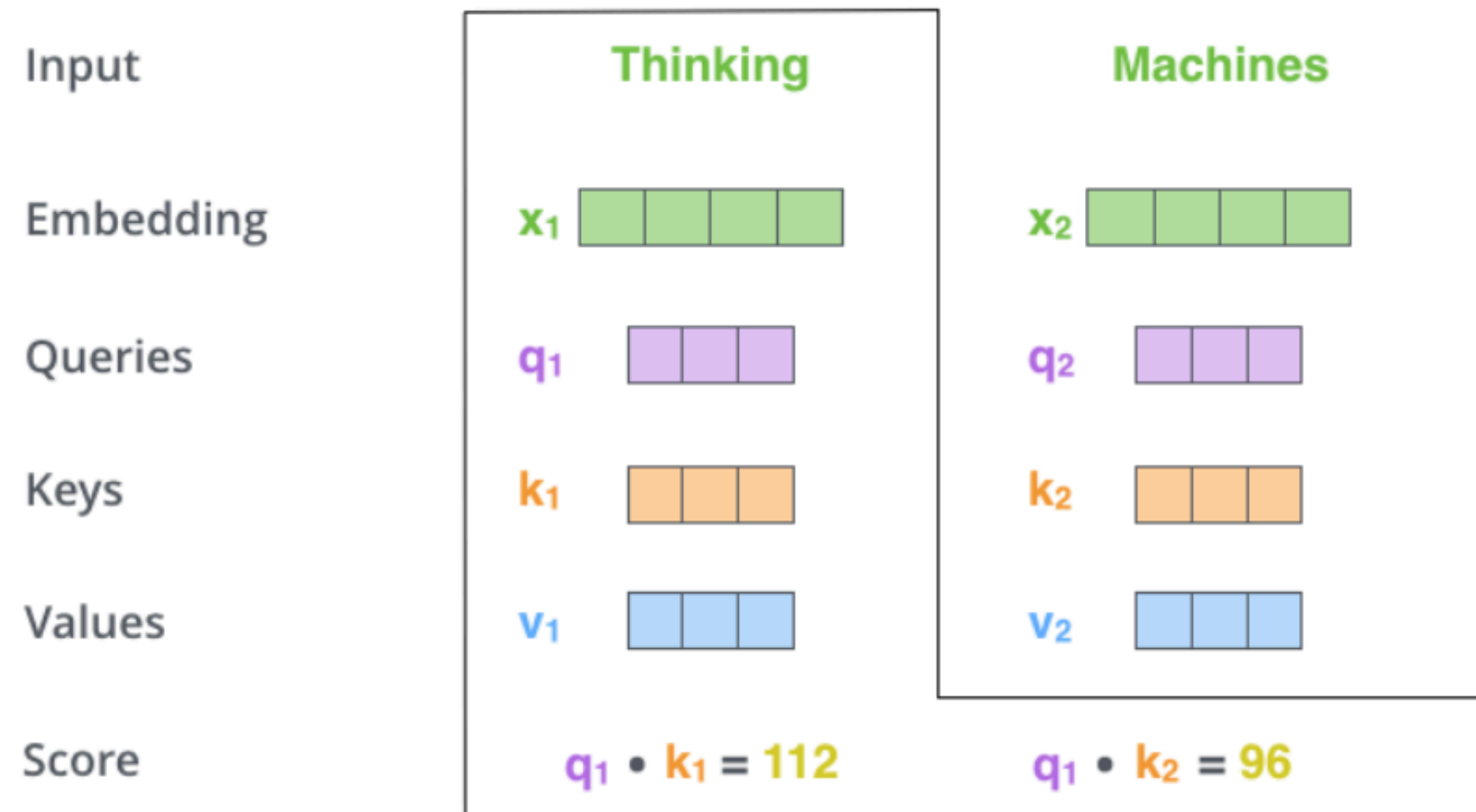
Transformers



- for each word, we create a Query vector, a Key vector, and a Value vector.
- These vectors are created by multiplying the embedding by three matrices that we trained during the training process
- Notice that these new vectors are smaller in dimension than the embedding vector. Their dimensionality is 64, while the embedding and encoder input/output vectors have dimensionality of 512. They don't HAVE to be smaller, this is an architecture choice

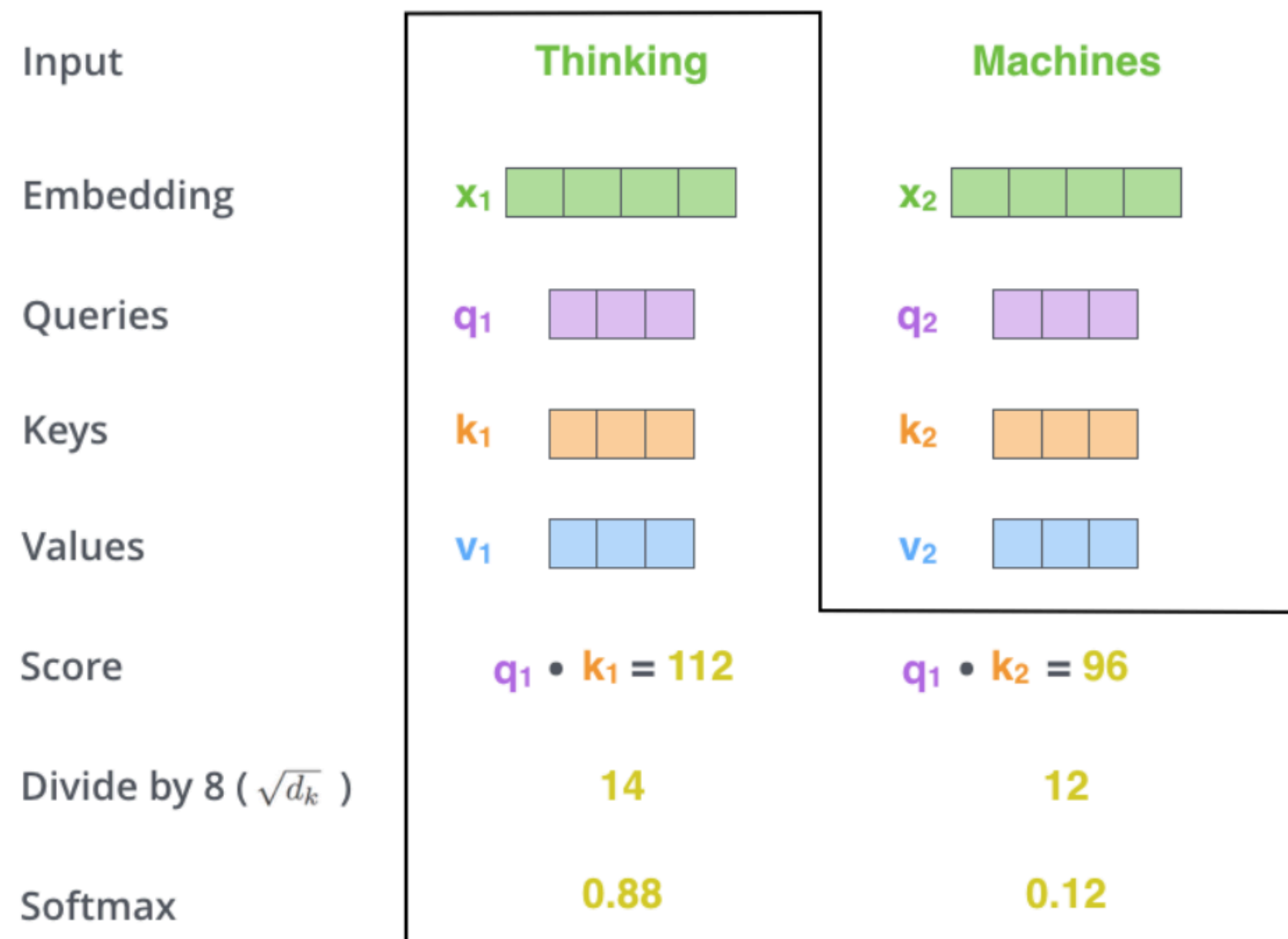
<https://github.com/Amr-Abdellatif>

Transformers



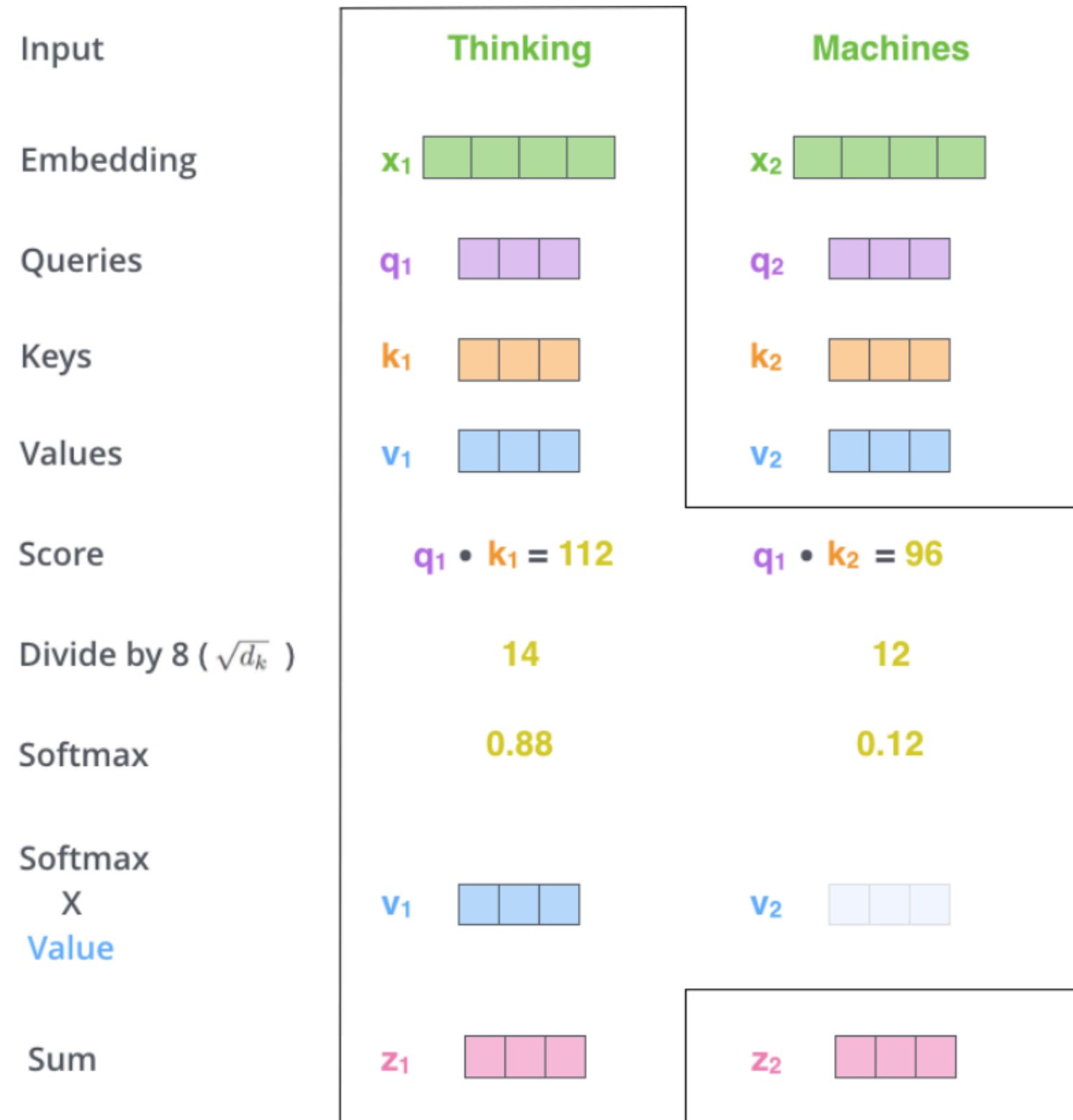
- The second step in calculating self-attention is to calculate a score. Say we're calculating the self-attention for the first word in this example, "Thinking". We need to score each word of the input sentence against this word
- The score is calculated by taking the dot product of the query vector with the key vector of the respective word we're scoring. So if we're processing the self-attention for the word in position #1, the first score would be the dot product of q1 and k1. The second score would be the dot product of q1 and k2.

Transformers



- The third and fourth steps are to divide the scores by 8 (the square root of the dimension of the key vectors used in the paper – 64)

Transformers



- The fifth step is to multiply each value vector by the softmax score (in preparation to sum them up). The intuition here is to keep intact the values of the word(s) we want to focus on, and drown-out irrelevant words (by multiplying them by tiny numbers like 0.001, for example).
- The sixth step is to sum up the weighted value vectors. This produces the output of the self-attention layer at this position (for the first word against every other word in the sequence).

Transformers

$$\text{softmax}\left(\frac{\begin{matrix} \textcolor{violet}{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \textcolor{brown}{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} \textcolor{blue}{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

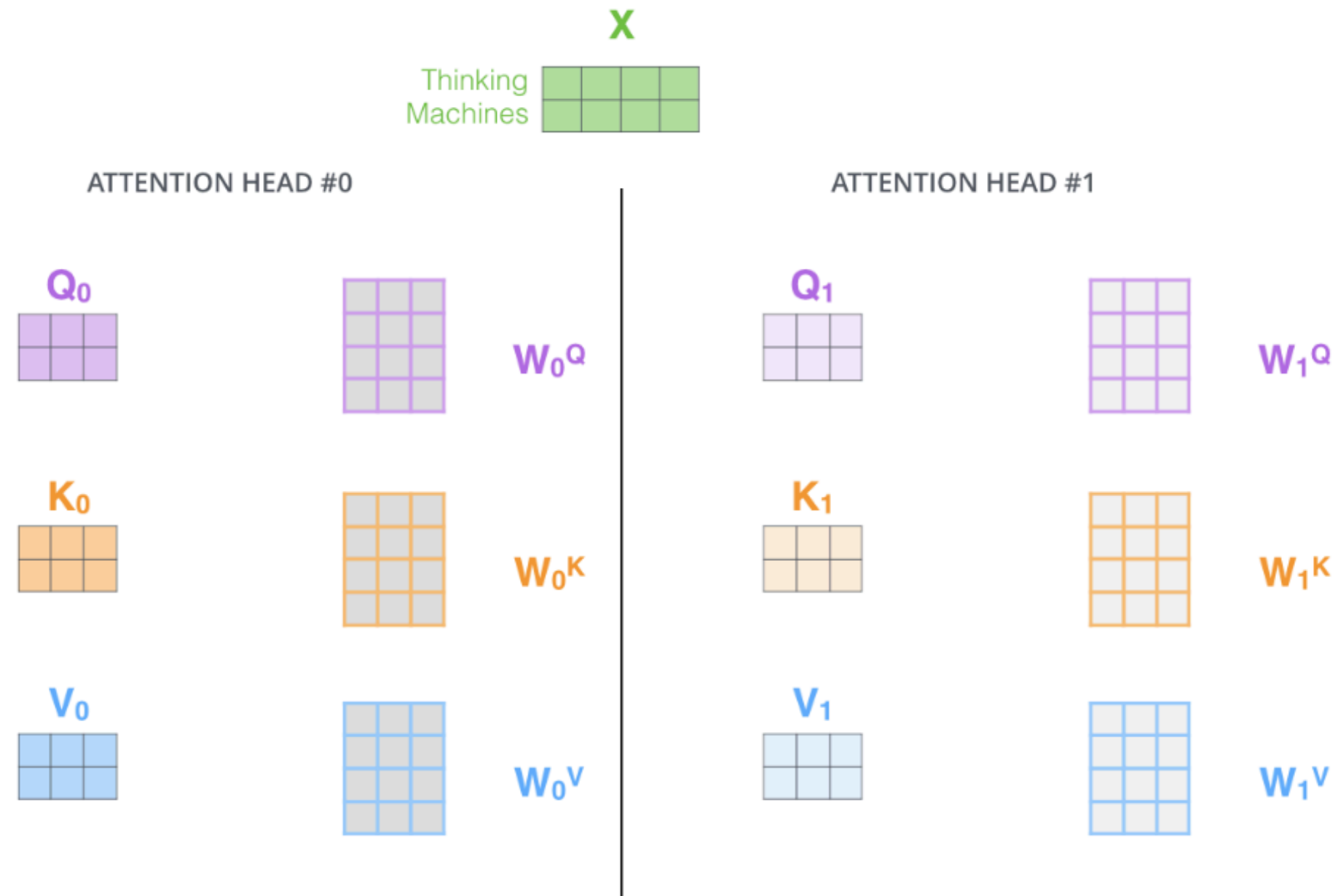
$$= \begin{matrix} \textcolor{pink}{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

The self-attention calculation in matrix form

<https://github.com/Amr-Abdellatif>

Transformers

multi-headed

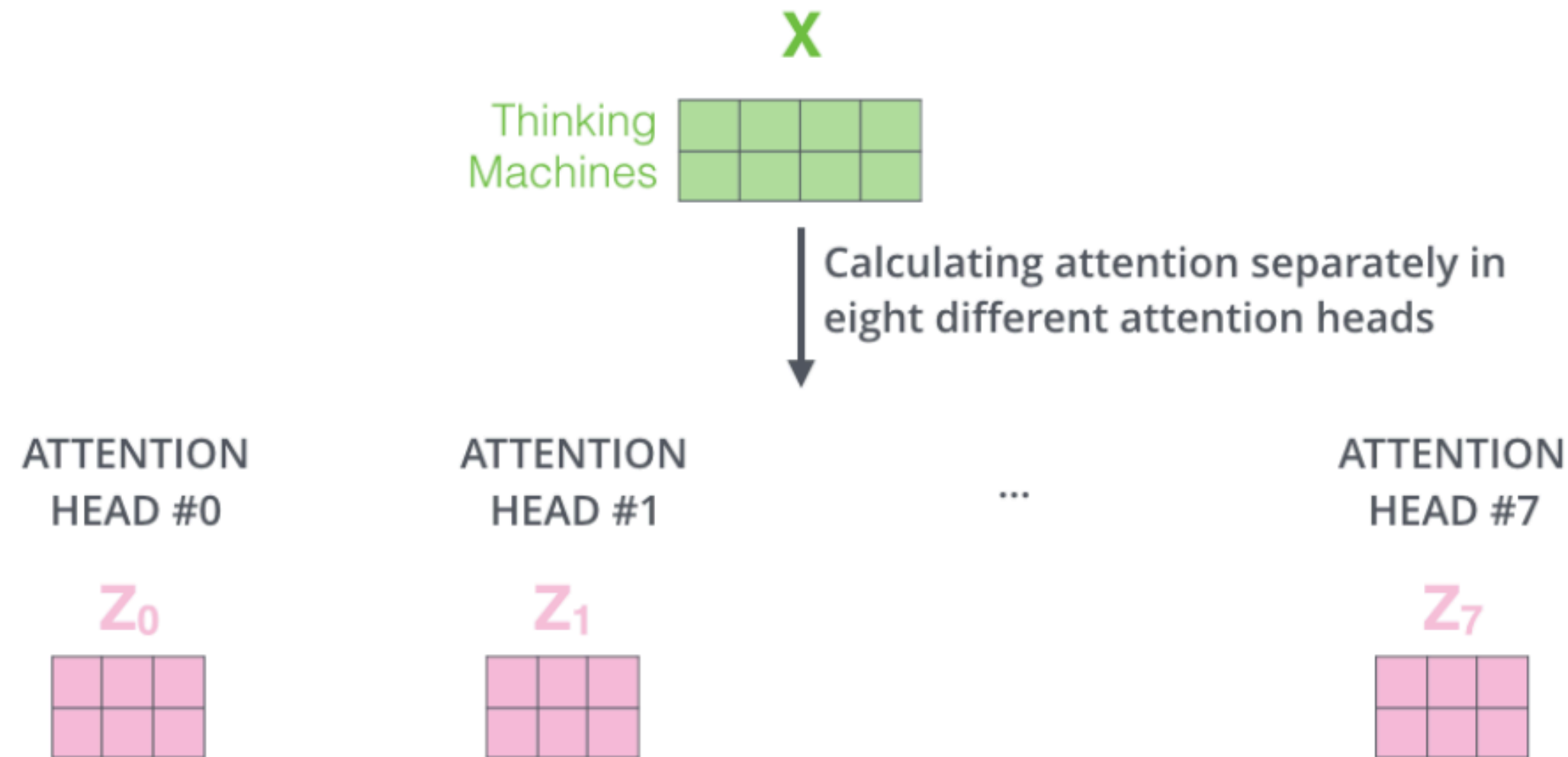


With multi-headed attention, we maintain separate Q/K/V weight matrices for each head resulting in different Q/K/V matrices. As we did before, we multiply X by the $W_Q/W_K/W_V$ matrices to produce Q/K/V matrices.

<https://github.com/Amr-Abdellatif>

Transformers

multi-headed

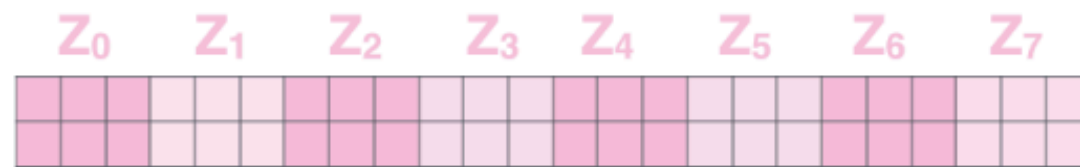


- If we do the same self-attention calculation we outlined above, just eight different times with different weight matrices, we end up with eight different Z matrices

Transformers

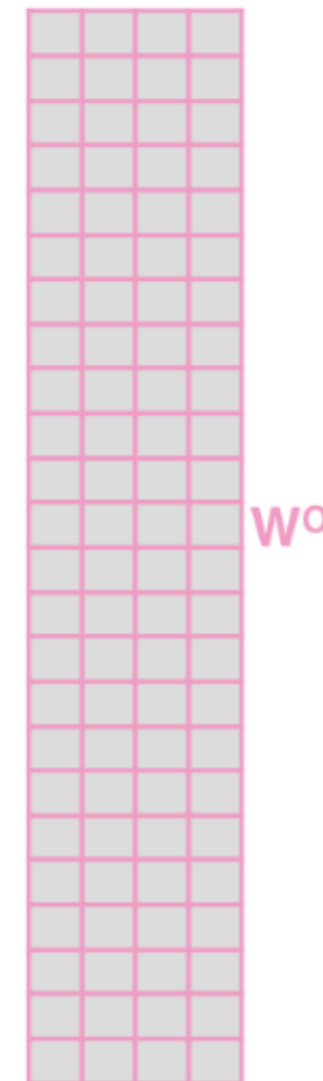
multi-headed

1) Concatenate all the attention heads

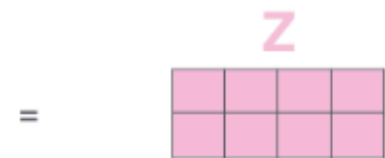


2) Multiply with a weight matrix W^O that was trained jointly with the model

x



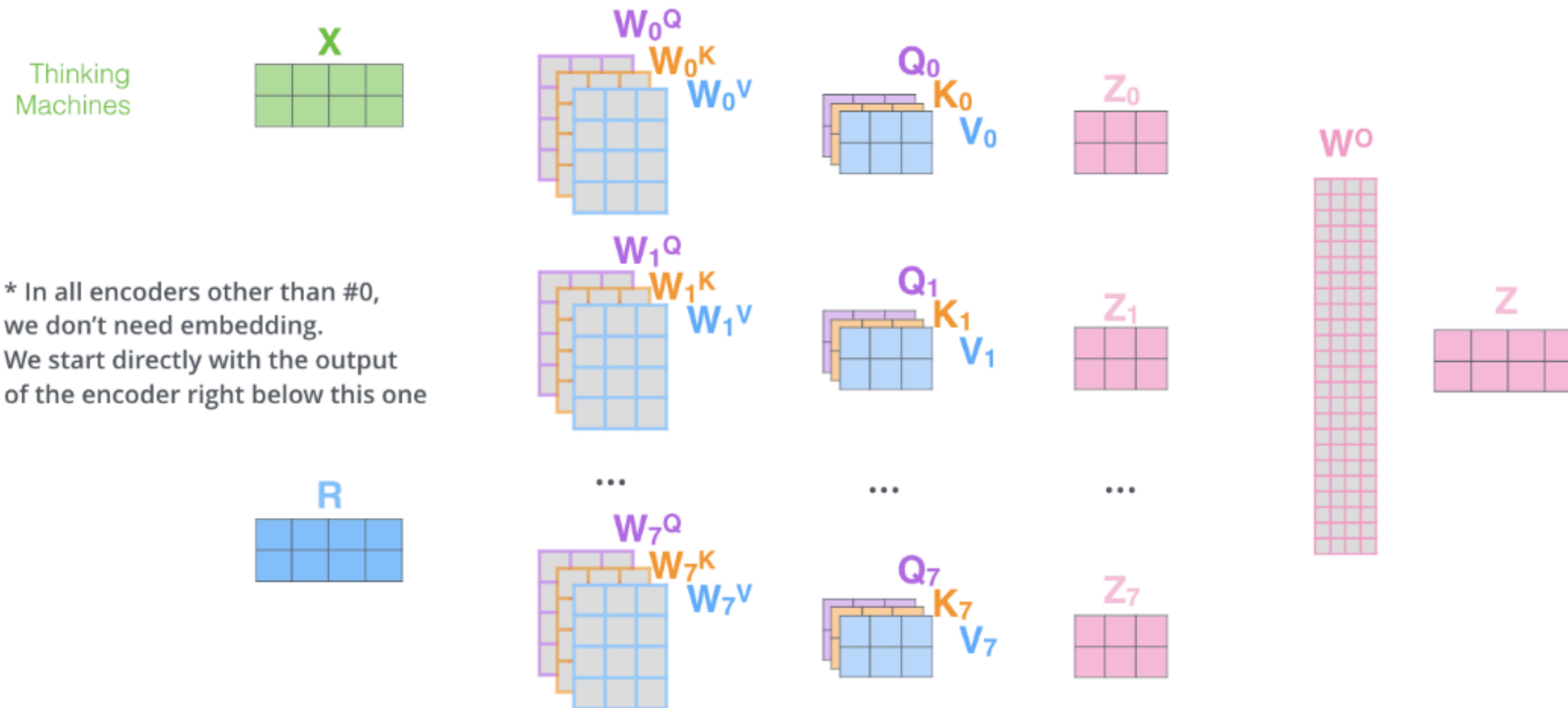
3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



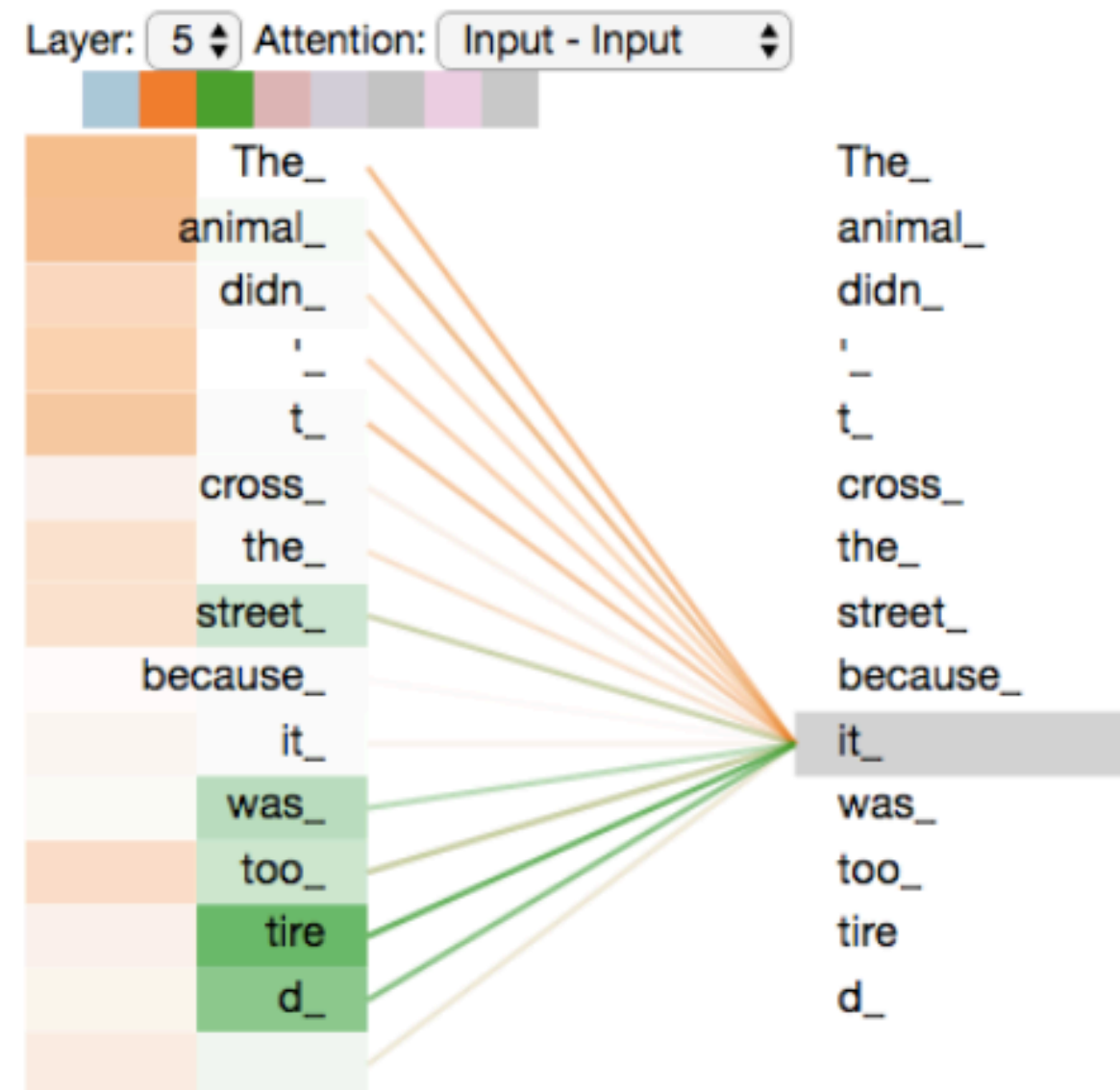
- The feed-forward layer is not expecting eight matrices – it's expecting a single matrix (a vector for each word). So we need a way to condense these eight down into a single matrix.

Multi Head Attention Overview

- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



Transformers



different attention heads are focusing as we encode the word “it” in our example sentence:

As we encode the word "it", one attention head is focusing most on "the animal", while another is focusing on "tired" -- in a sense, the model's representation of the word "it" bakes in some of the representation of both "animal" and "tired".

<https://github.com/Amr-Abdellatif>

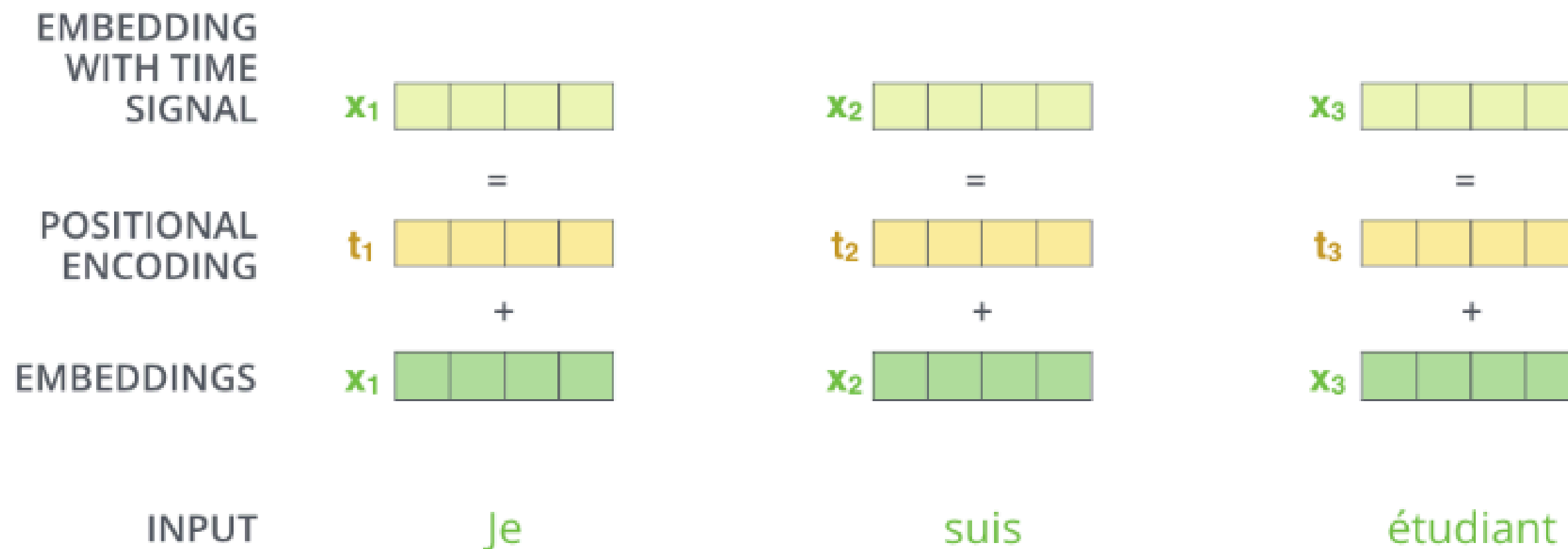
Transformers

Representing The Order of The Sequence Using Positional Encoding

<https://github.com/Amr-Abdellatif>

Transformers

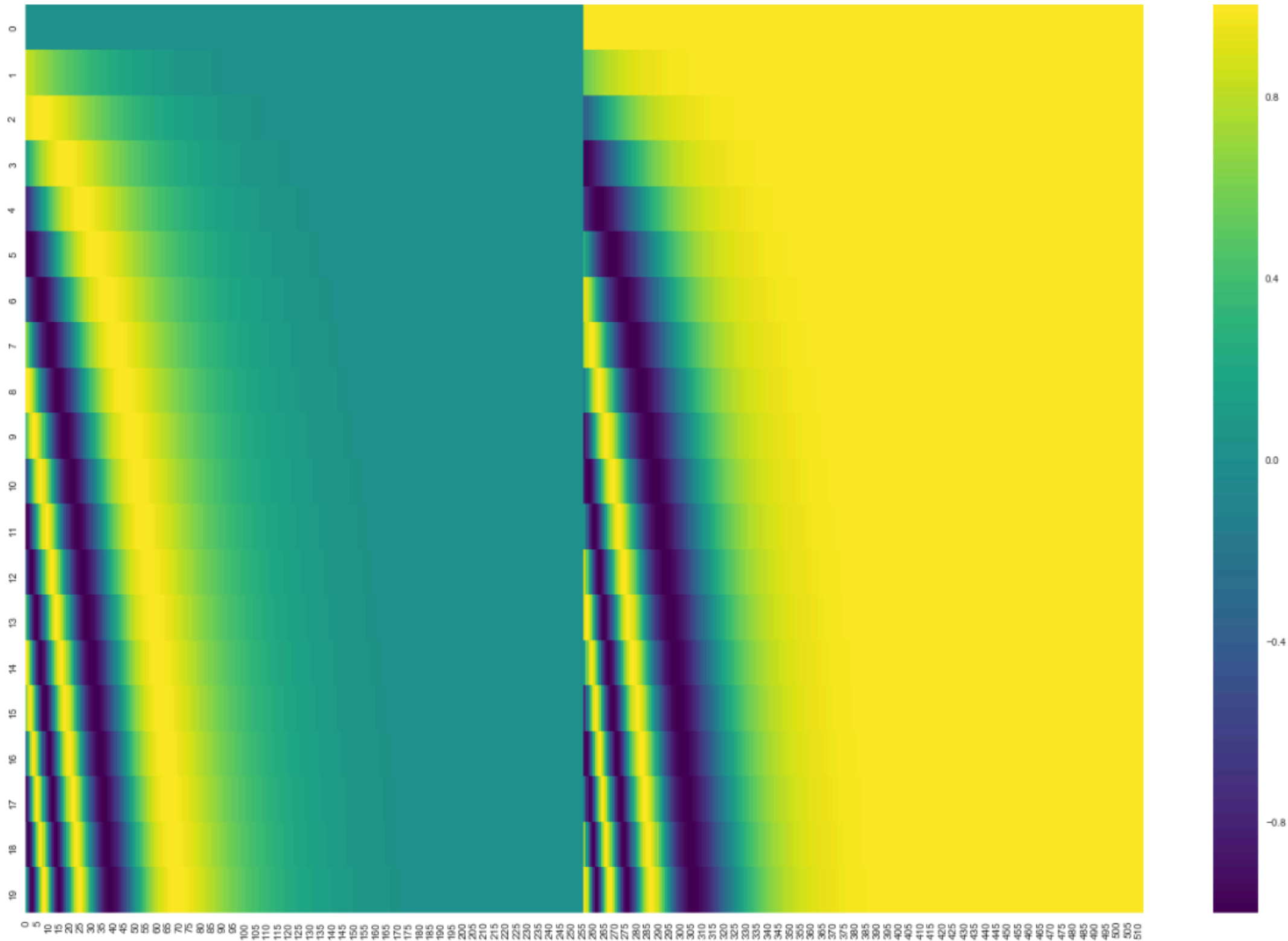
One thing that's missing from the model as we have described it so far is a way to account for the order of the words in the input sequence.



To give the model a sense of the order of the words, we add positional encoding vectors -- the values of which follow a specific pattern.

<https://github.com/Amr-Abdellatif>

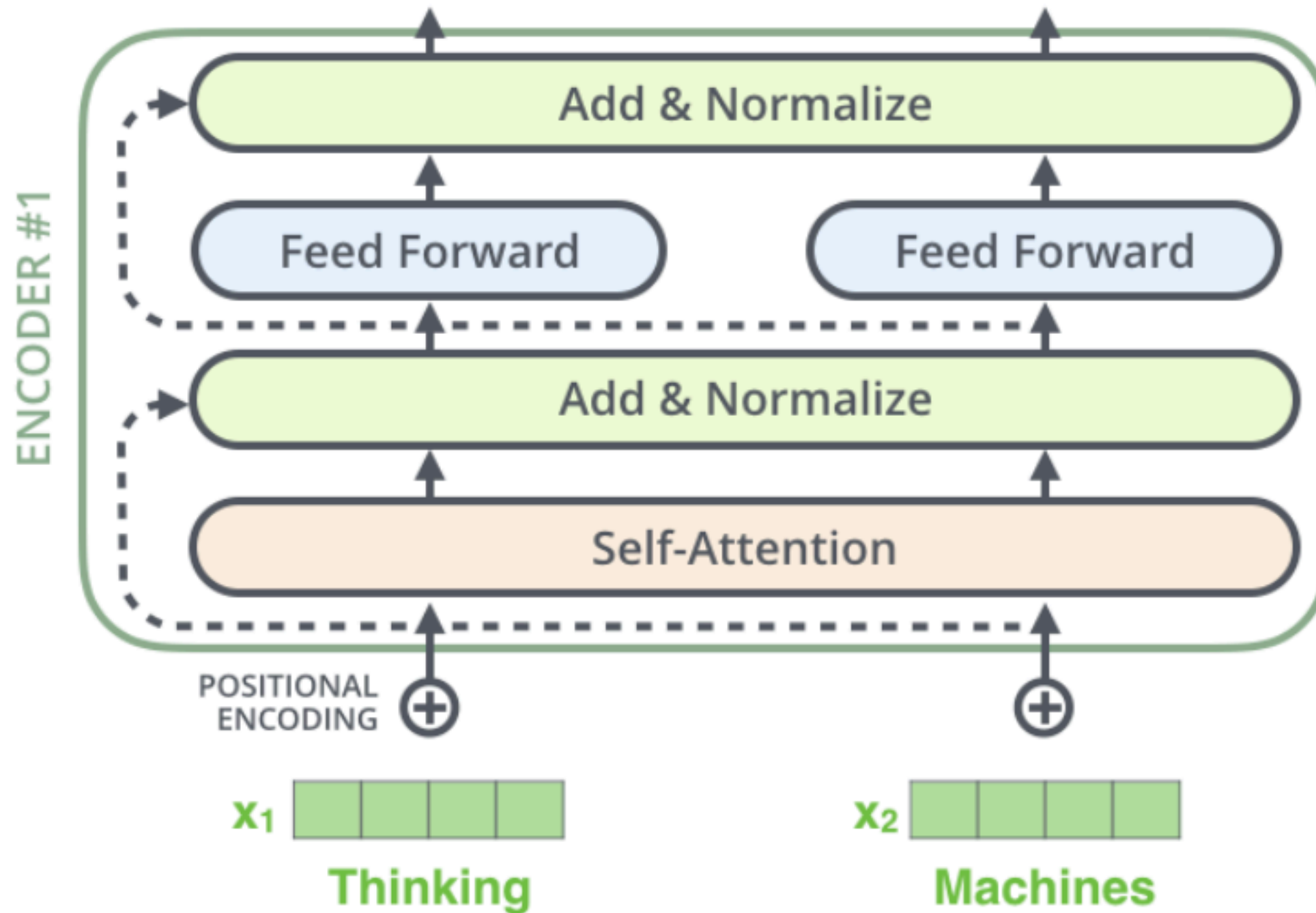
Transformers



- In the following figure, each row corresponds to a positional encoding of a vector. So the first row would be the vector we'd add to the embedding of the first word in an input sequence. Each row contains 512 values – each with a value between 1 and -1

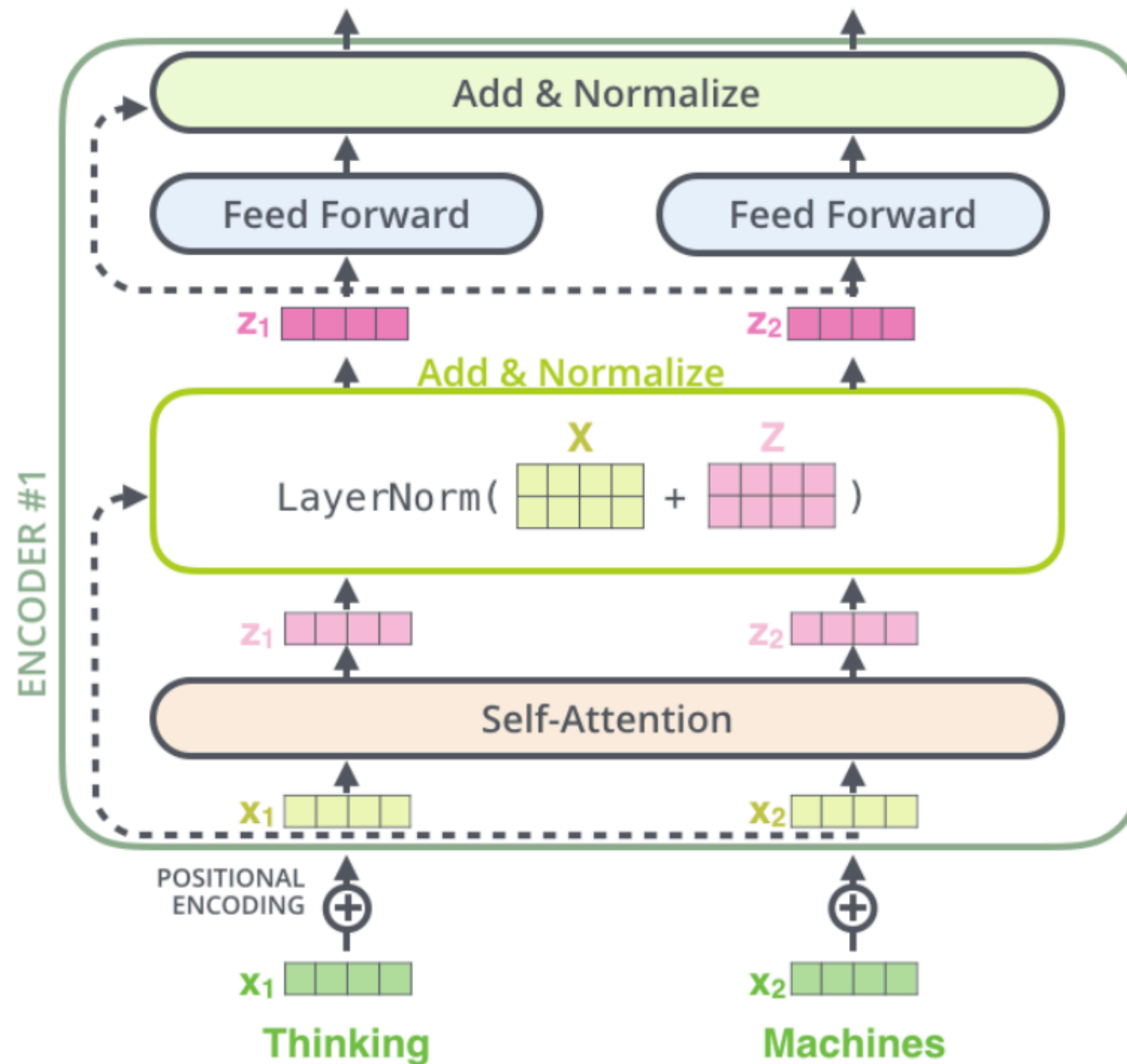
<https://github.com/Amr-Abdellatif>

Transformers



- One detail in the architecture of the encoder that we need to mention before moving on, is that each sub-layer (self-attention, ffnn) in each encoder has a residual connection around it, and is followed by a layer normalization step.

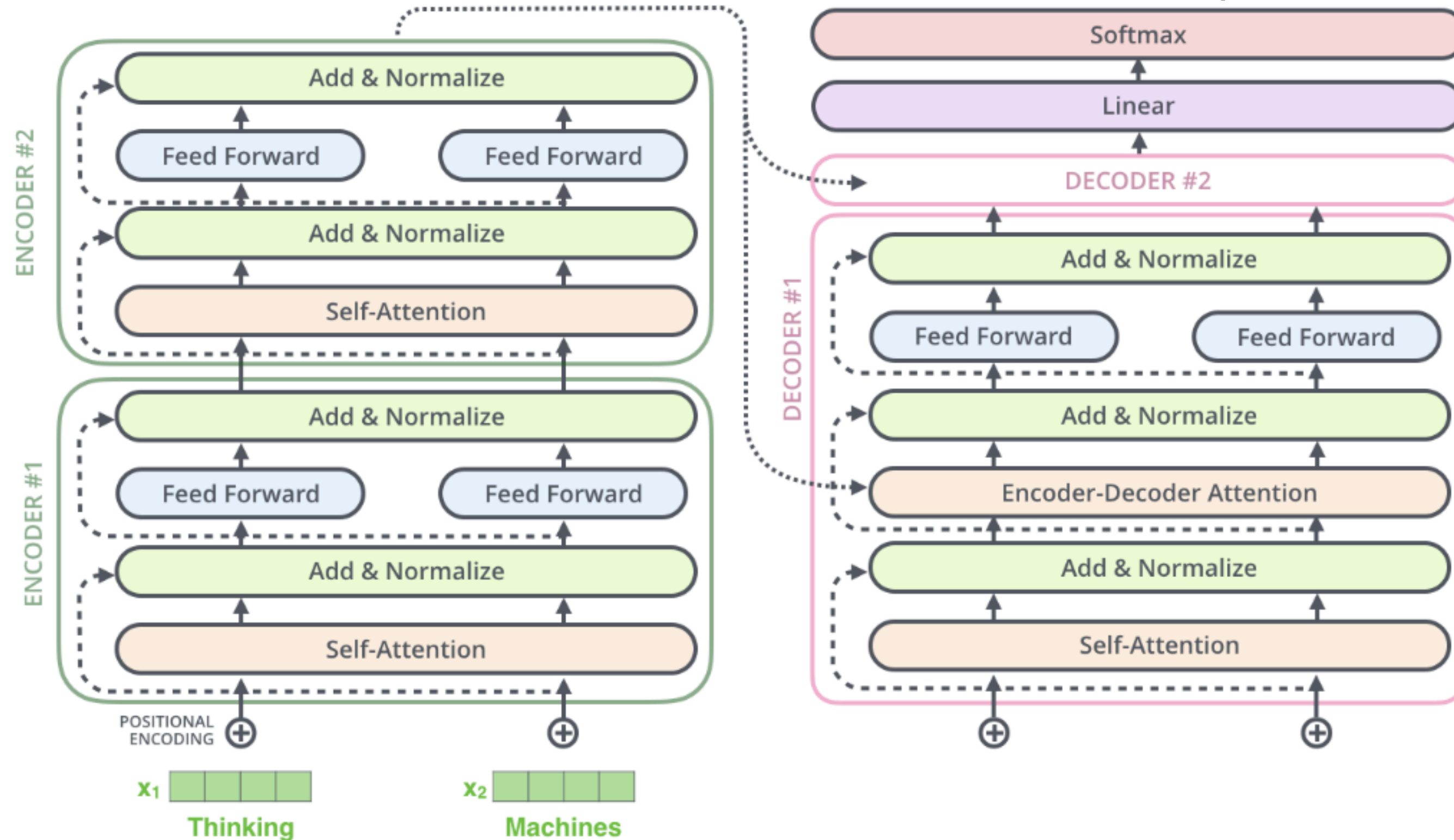
Transformers



- If we're to visualize the vectors and the layer-norm operation associated with self attention, it would look like this

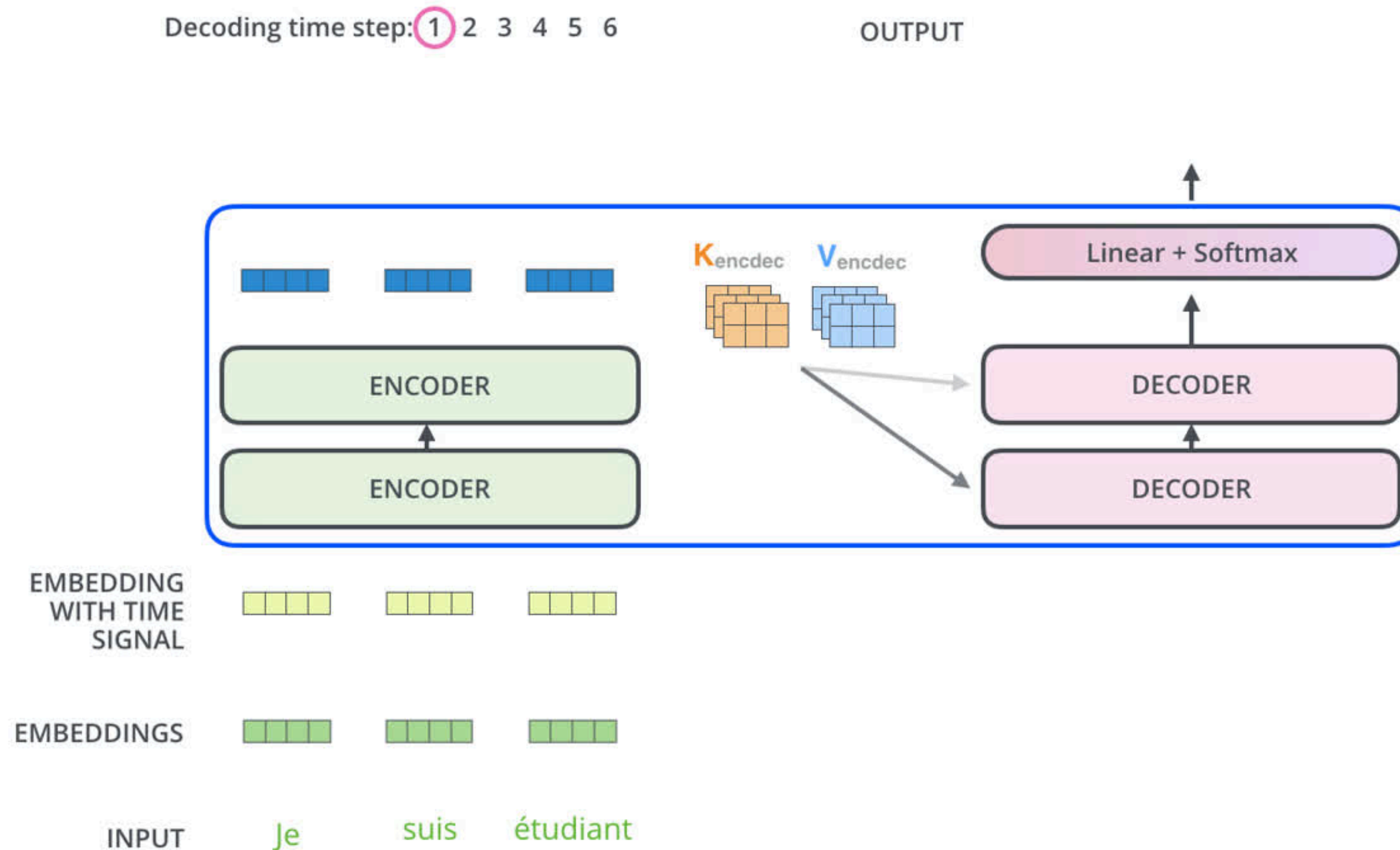
Transformers

A transformer of 2 stacked encoder - decoder per each.



<https://github.com/Amr-Abdellatif>

Transformers



- The encoder start by processing the input sequence. The output of the top encoder is then transformed into a set of attention vectors K and V. These are to be used by each decoder in its “encoder-decoder attention” layer which helps the decoder focus on appropriate places in the input sequence

Transformers

Imagine we have a very simple encoder processing the sentence: "The cat sits"

Step 1: Input Embedding

Each word is converted to an embedding vector

"The" → [0.1, 0.2, 0.3, 0.4]

"cat" → [0.5, 0.6, 0.7, 0.8]

"sits" → [0.9, 1.0, 1.1, 1.2]

Transformers

Step 2: Add Positional Encoding

Position information is added to each embedding

Let's say after adding position info we get:

"The" (pos 1) $\rightarrow$ [0.15, 0.25, 0.35, 0.45]

"cat" (pos 2) $\rightarrow$ [0.55, 0.65, 0.75, 0.85]

"sits" (pos 3) $\rightarrow$ [0.95, 1.05, 1.15, 1.25]

Transformers

Step 3: Linear Projections to create Q, K, V

The encoder applies three different weight matrices to create Q, K, V vectors for each word

For "The":

- Q_The = [0.11, 0.21, 0.31, 0.41]
- K_The = [0.12, 0.22, 0.32, 0.42]
- V_The = [0.13, 0.23, 0.33, 0.43]

Similar projections for "cat" and "sits"

Transformers

Step 4: Self-Attention and Processing

The encoder applies self-attention and feed-forward networks

After all encoder processing, we get final representations for each word

Transformers

Step 5: Final Representations

- "The" → [0.2, 0.3, 0.4, 0.5]
- "cat" → [0.6, 0.7, 0.8, 0.9]
- "sits" → [1.0, 1.1, 1.2, 1.3]

Transformers

Step 6: Creating K and V matrices

The encoder applies two separate learned linear transformations (weight matrices) to these hidden states

For K: We multiply each hidden state by a "key projection" weight matrix

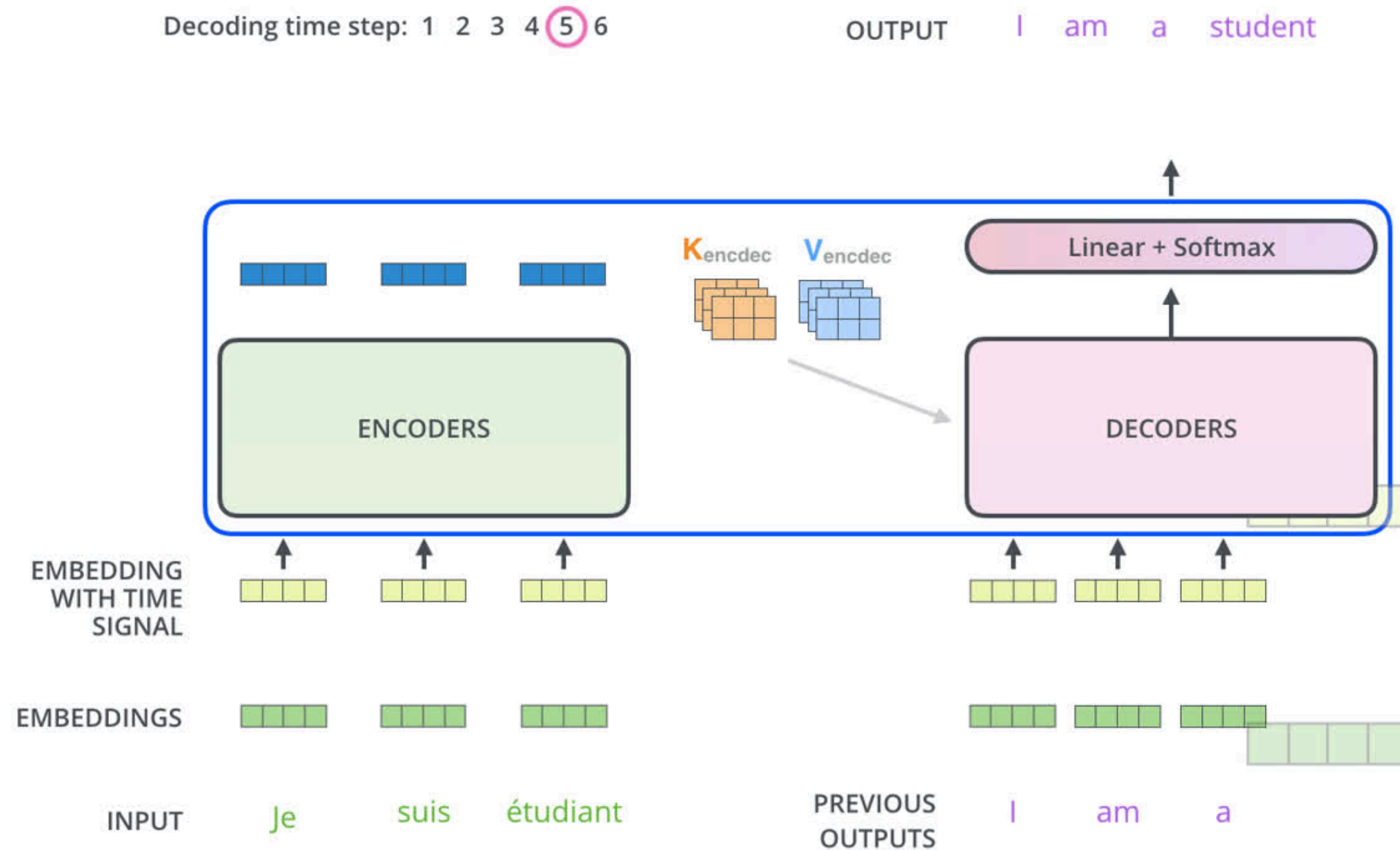
For V: We multiply each hidden state by a "value projection" weight matrix

Result :

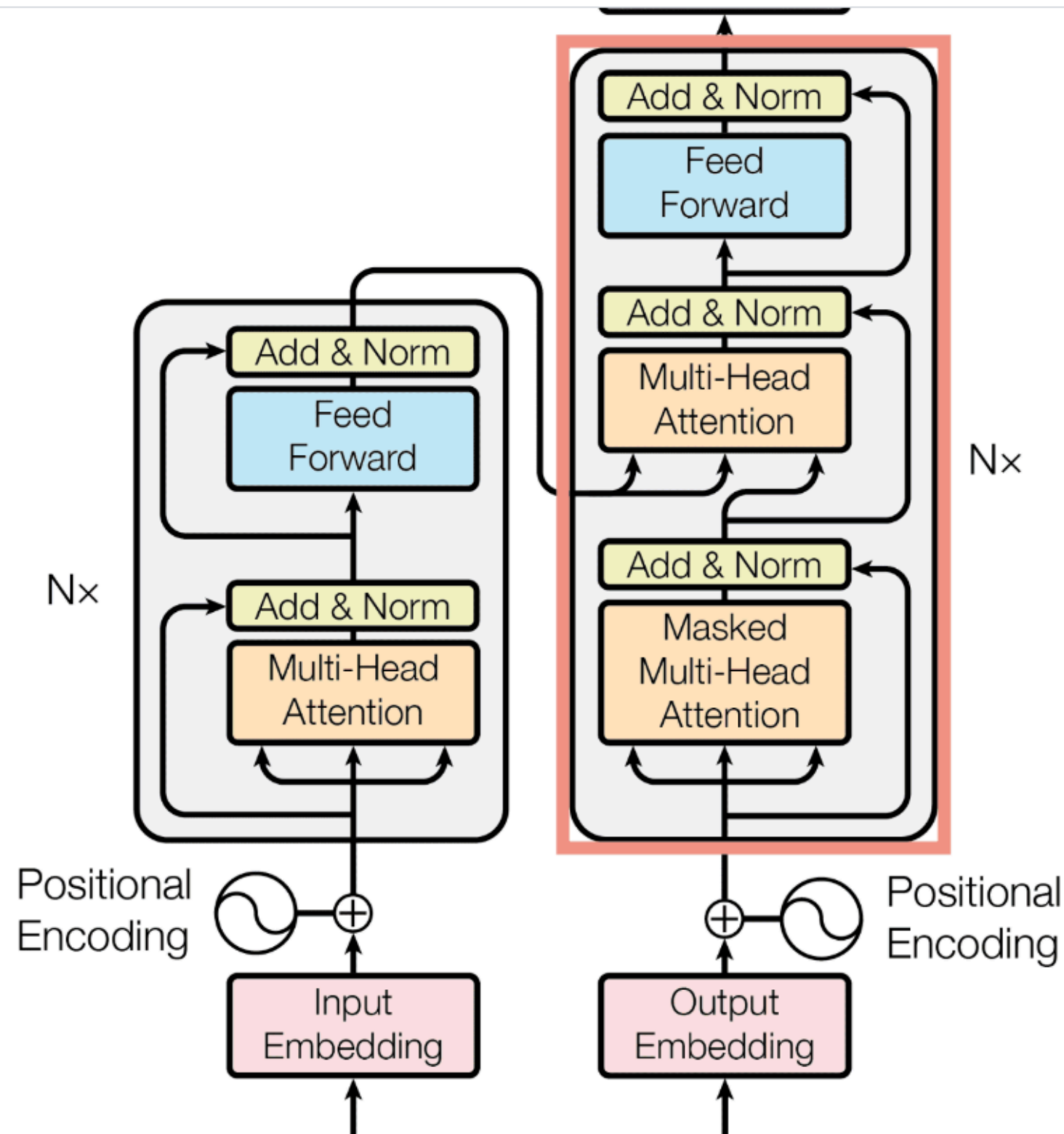
1. The K matrix contains all the key vectors, one for each input token
2. The V matrix contains all the value vectors, one for each input token

Transformers

- The following steps repeat the process until a special symbol is reached indicating the transformer decoder has completed its output. The output of each step is fed to the bottom decoder in the next time step, and the decoders bubble up their decoding results just like the encoders did.



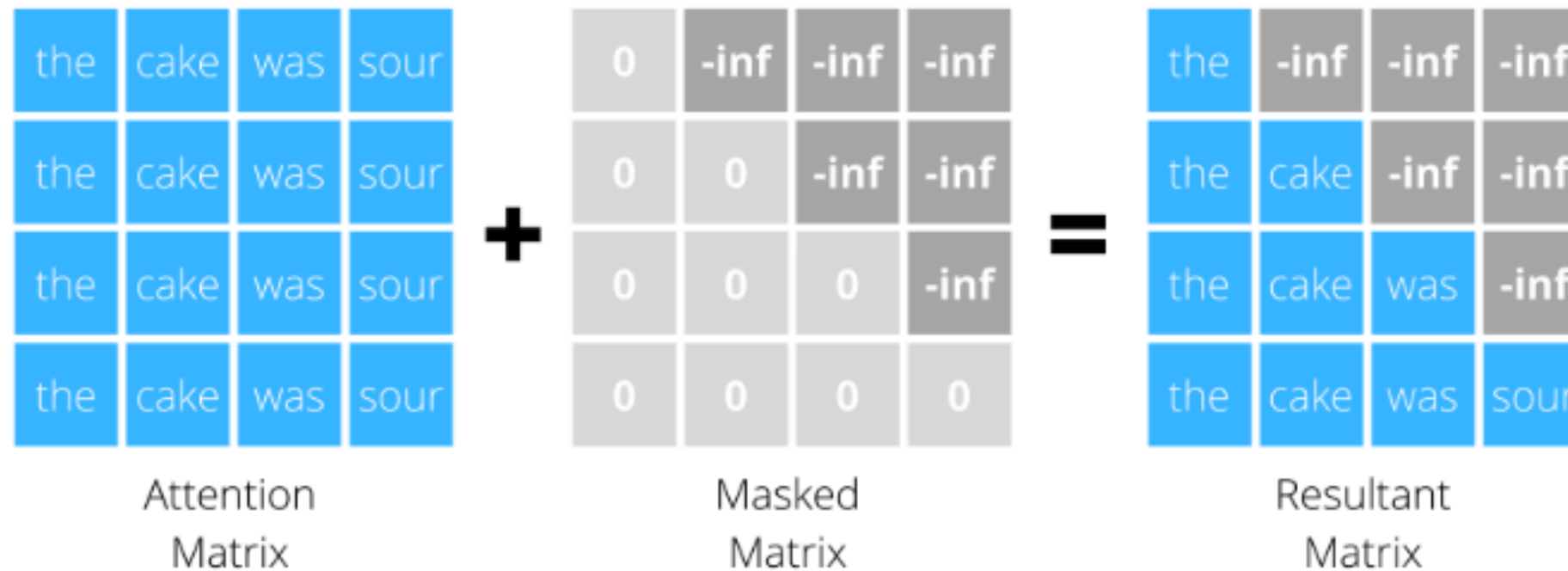
Transformers



- In the decoder, the self-attention layer is only allowed to attend to earlier positions in the output sequence. This is done by masking future positions (setting them to $-\infty$) before the softmax step in the self-attention calculation.

Transformers

Masked Attention



Transformers

The Final Linear and Softmax Layer

The Linear layer is a simple fully connected neural network that projects the vector produced by the stack of decoders, into a much, much larger vector called a logits vector.

Let's assume that our model knows 10,000 unique English words (our model's "output vocabulary") that it's learned from its training dataset. This would make the logits vector 10,000 cells wide – each cell corresponding to the score of a unique word. That is how we interpret the output of the model followed by the Linear layer.

The softmax layer then turns those scores into probabilities (all positive, all add up to 1.0). The cell with the highest probability is chosen, and the word associated with it is produced as the output for this time step.

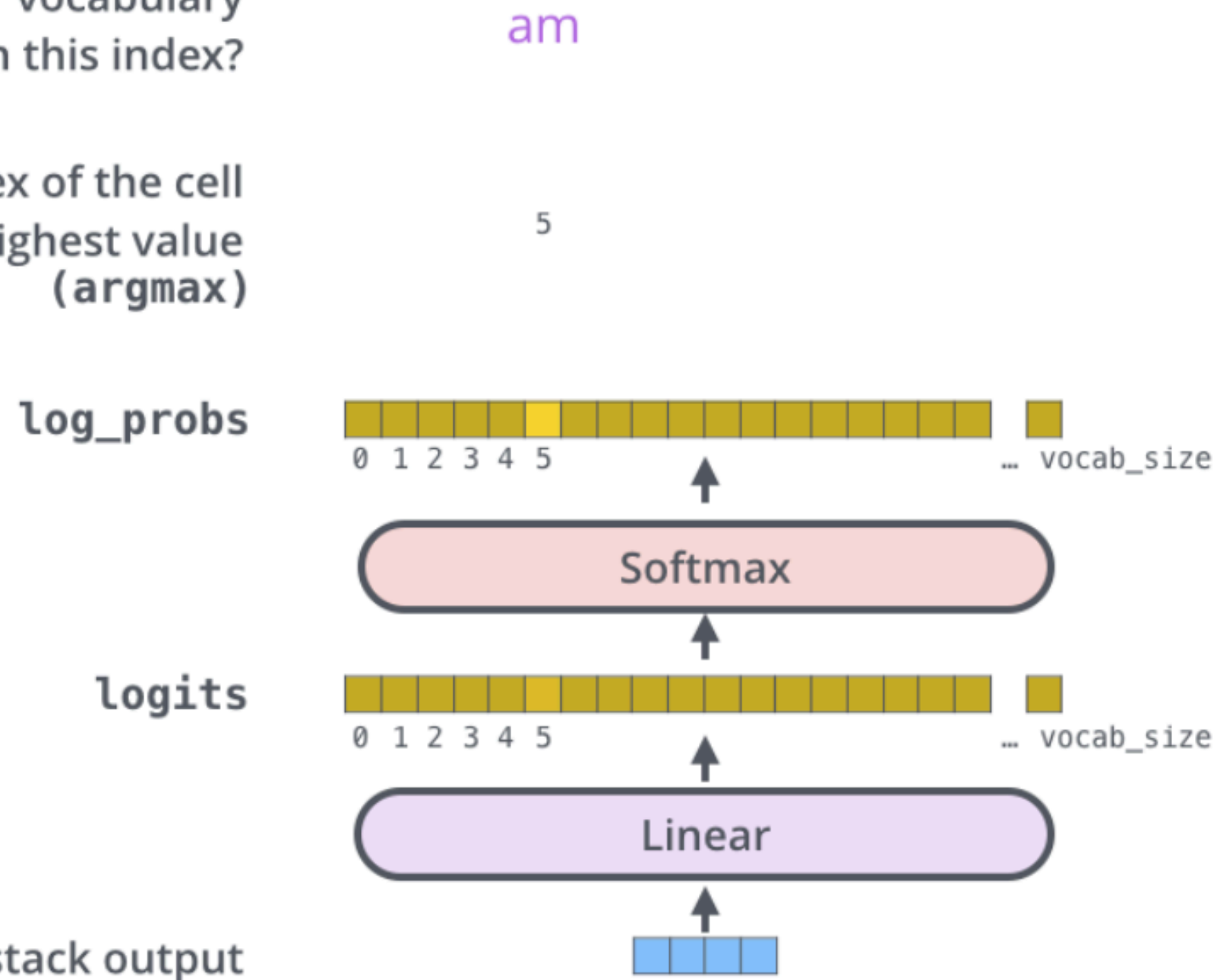
<https://github.com/Amr-Abdellatif>

Transformers

The Final Linear and Softmax Layer

Which word in our vocabulary
is associated with this index?

Get the index of the cell
with the highest value
(argmax)



This figure starts from the bottom with the vector produced as the output of the decoder stack. It is then turned into an output word.

<https://github.com/Amr-Abdellatif>

Transformers

Reading The Paper

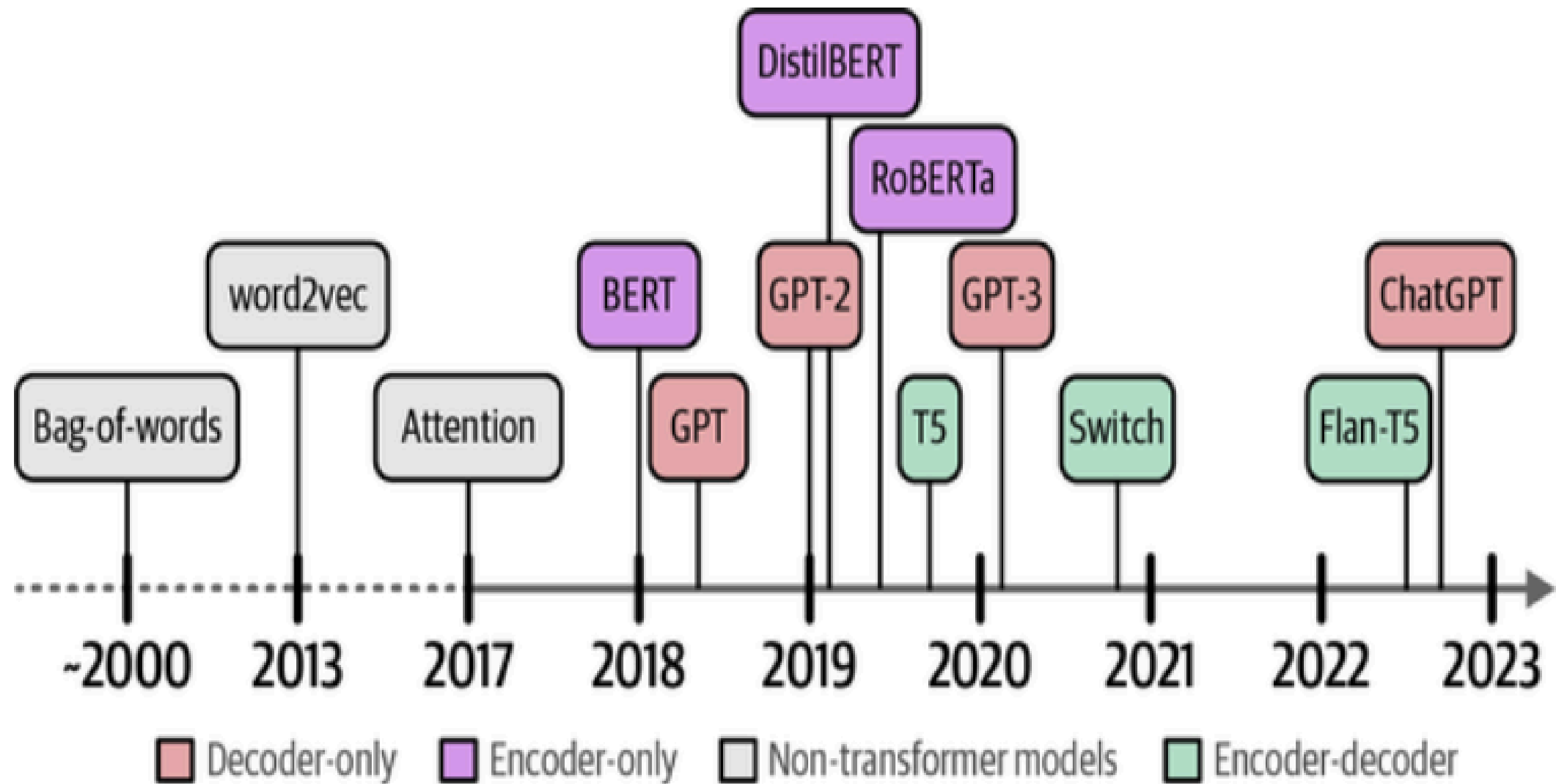
<https://arxiv.org/pdf/1706.03762>

<https://github.com/Amr-Abdellatif>

LLM's

<https://github.com/Amr-Abdellatif>

LLM's



<https://github.com/Amr-Abdellatif>

LLM's

Generative vs Representative models

<https://github.com/Amr-Abdellatif>

LLM's

Representation models mainly focus on representing language, for instance, by creating embeddings, and typically do not generate text. In contrast, generative models focus primarily on generating text and typically are not trained to generate embeddings.

<https://github.com/Amr-Abdellatif>

LLM's

Specialization vs Generalization

<https://github.com/Amr-Abdellatif>

LLM's

Representation Models: Encoder-Only Models

<https://github.com/Amr-Abdellatif>

LLM's

Representation Models: Encoder-Only Models

<https://github.com/Amr-Abdellatif>

Representation Models: Encoder-Only Models

The original Transformer model is an encoder-decoder architecture that serves translation tasks well but cannot easily be used for other tasks, like text classification.

<https://github.com/Amr-Abdellatif>

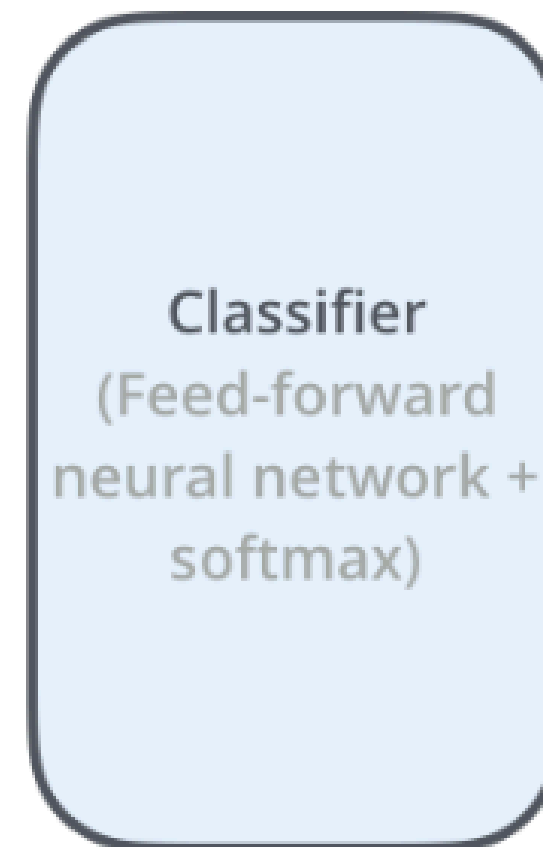
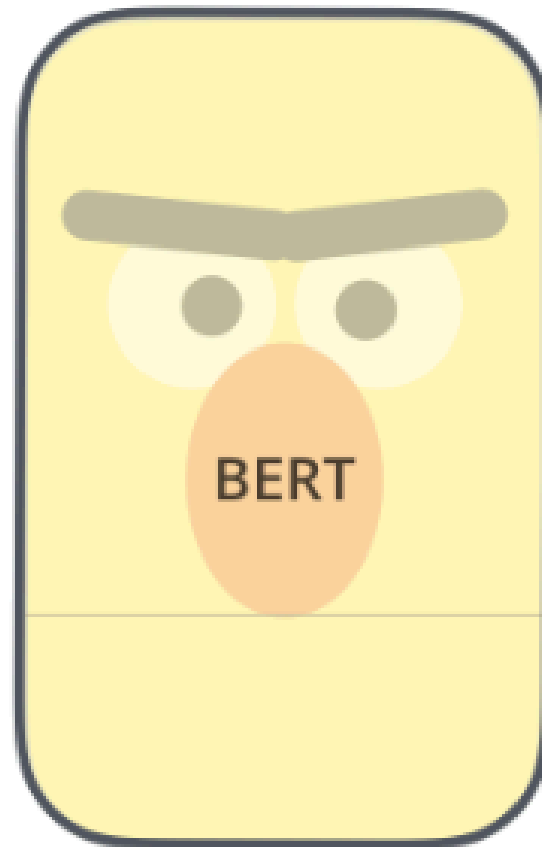
BERT

<https://github.com/Amr-Abdellatif>

BERT

Input
Features

Help Prince Mayuko Transfer
Huge Inheritance

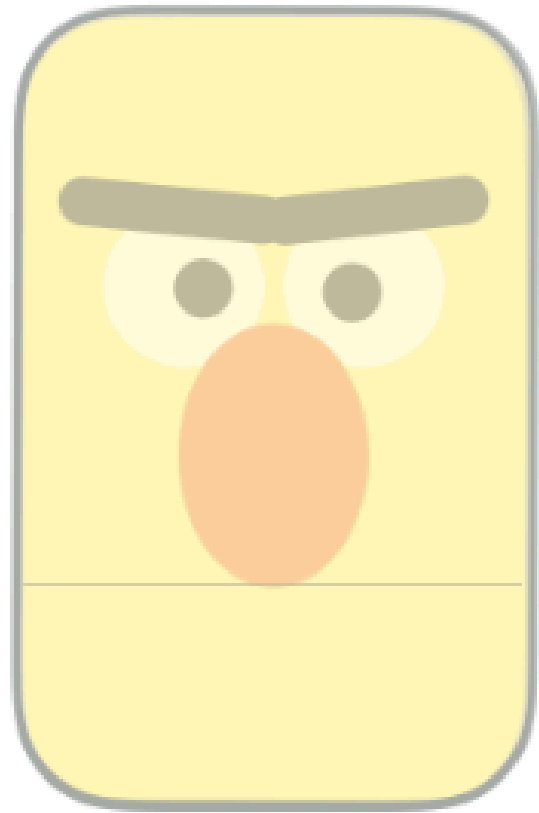


Output
Prediction

85%	Spam
15%	Not Spam

<https://github.com/Amr-Abdellatif>

BERT



BERT_{BASE}



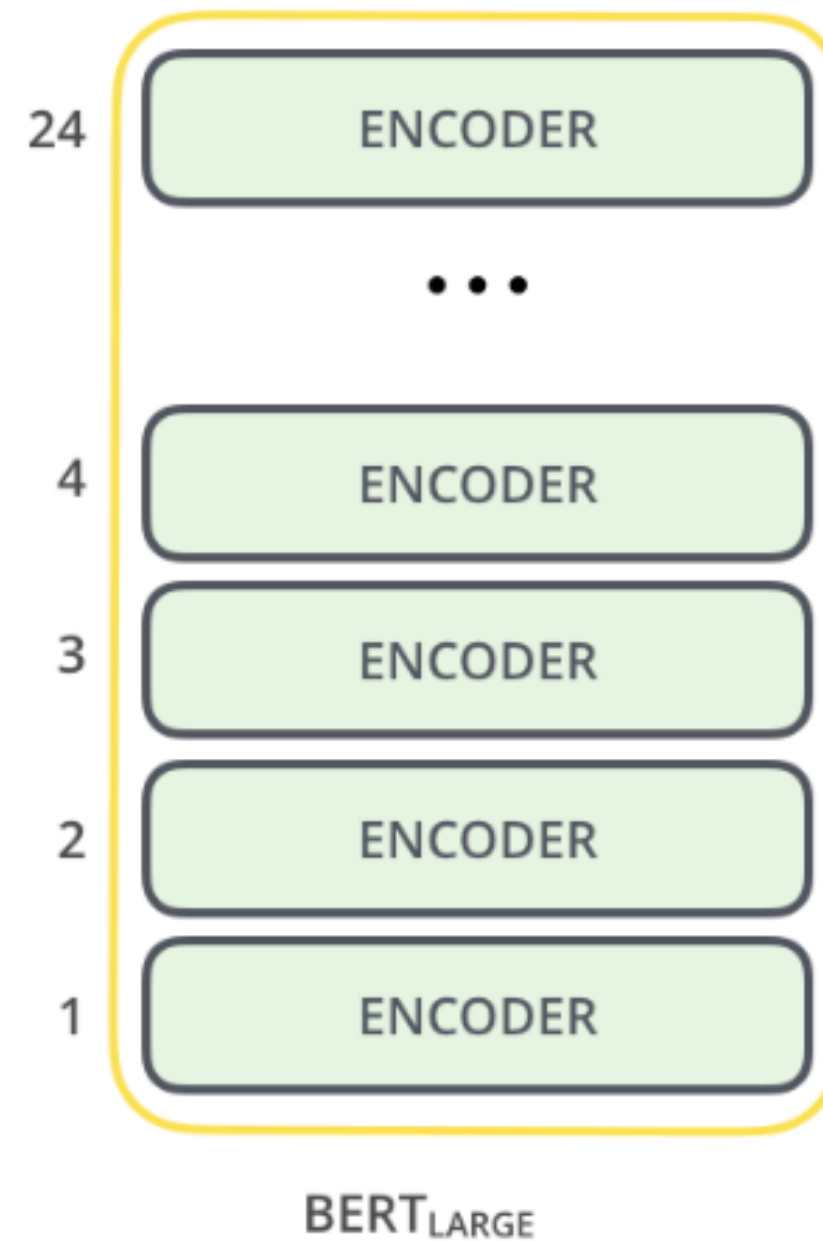
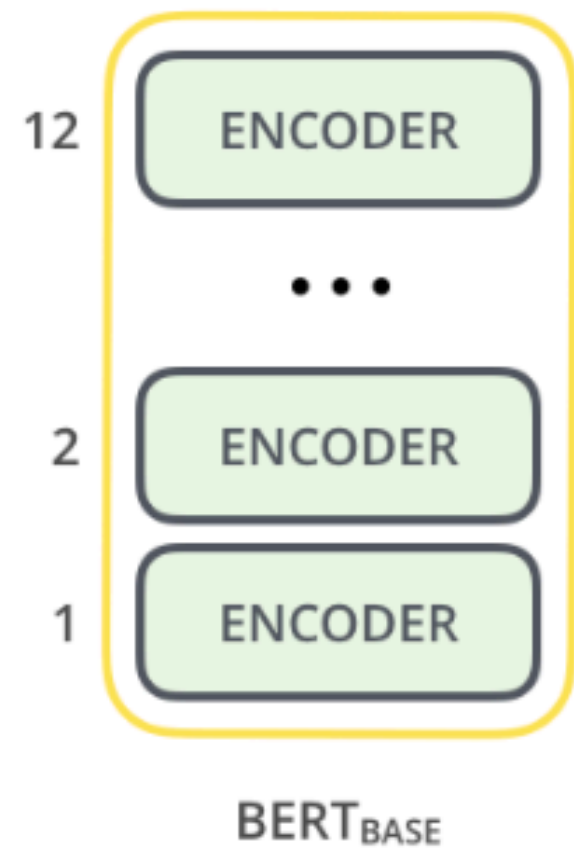
BERT_{LARGE}

The paper presents two model sizes for BERT:

- BERT BASE – Comparable in size to the OpenAI Transformer in order to compare performance
- BERT LARGE – A ridiculously huge model which achieved the state of the art results reported in the paper

<https://github.com/Amr-Abdellatif>

BERT

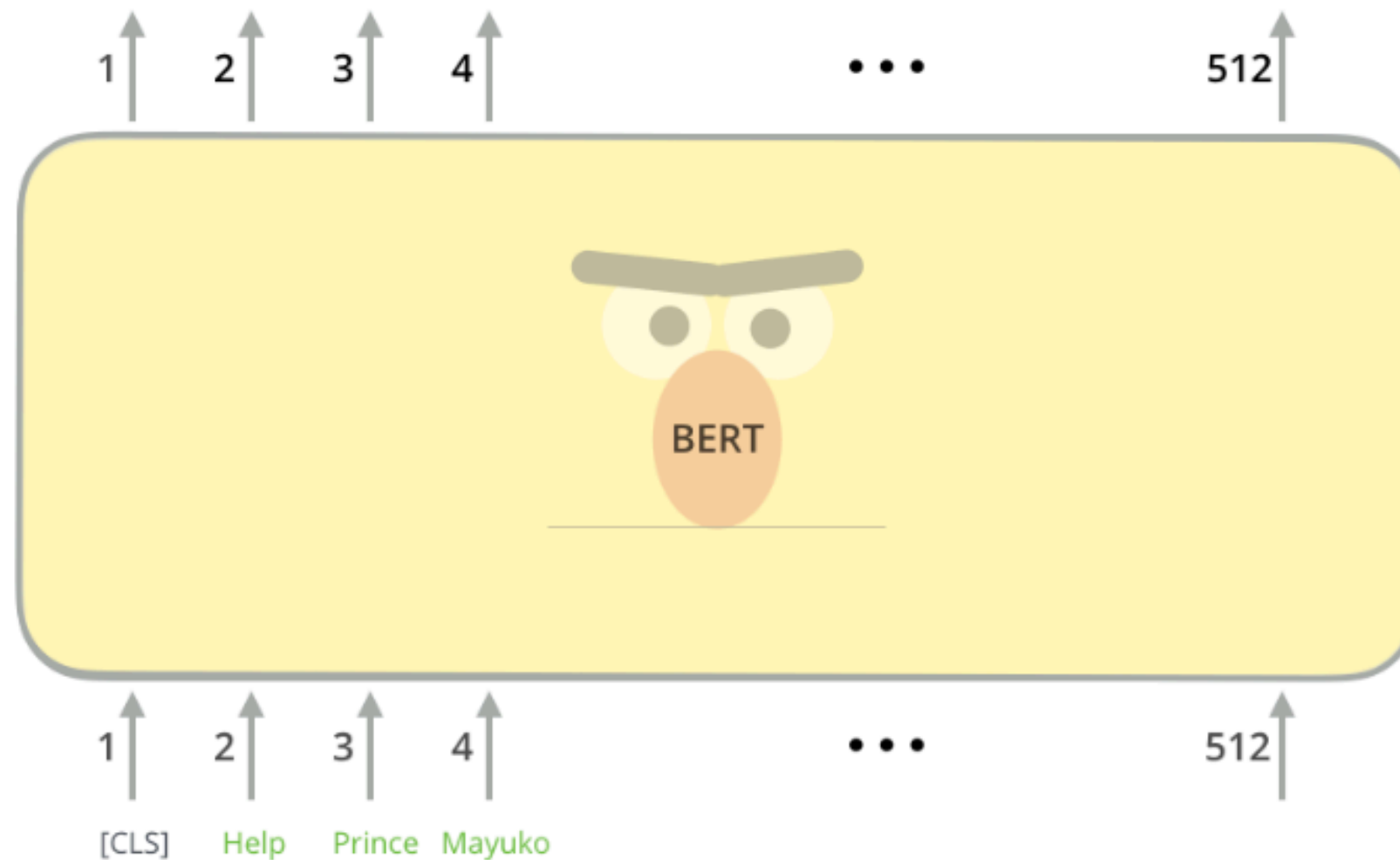


- BERT BASE – 12 encoder layers
- BERT LARGE – 24 encoder layers

<https://github.com/Amr-Abdellatif>

BERT

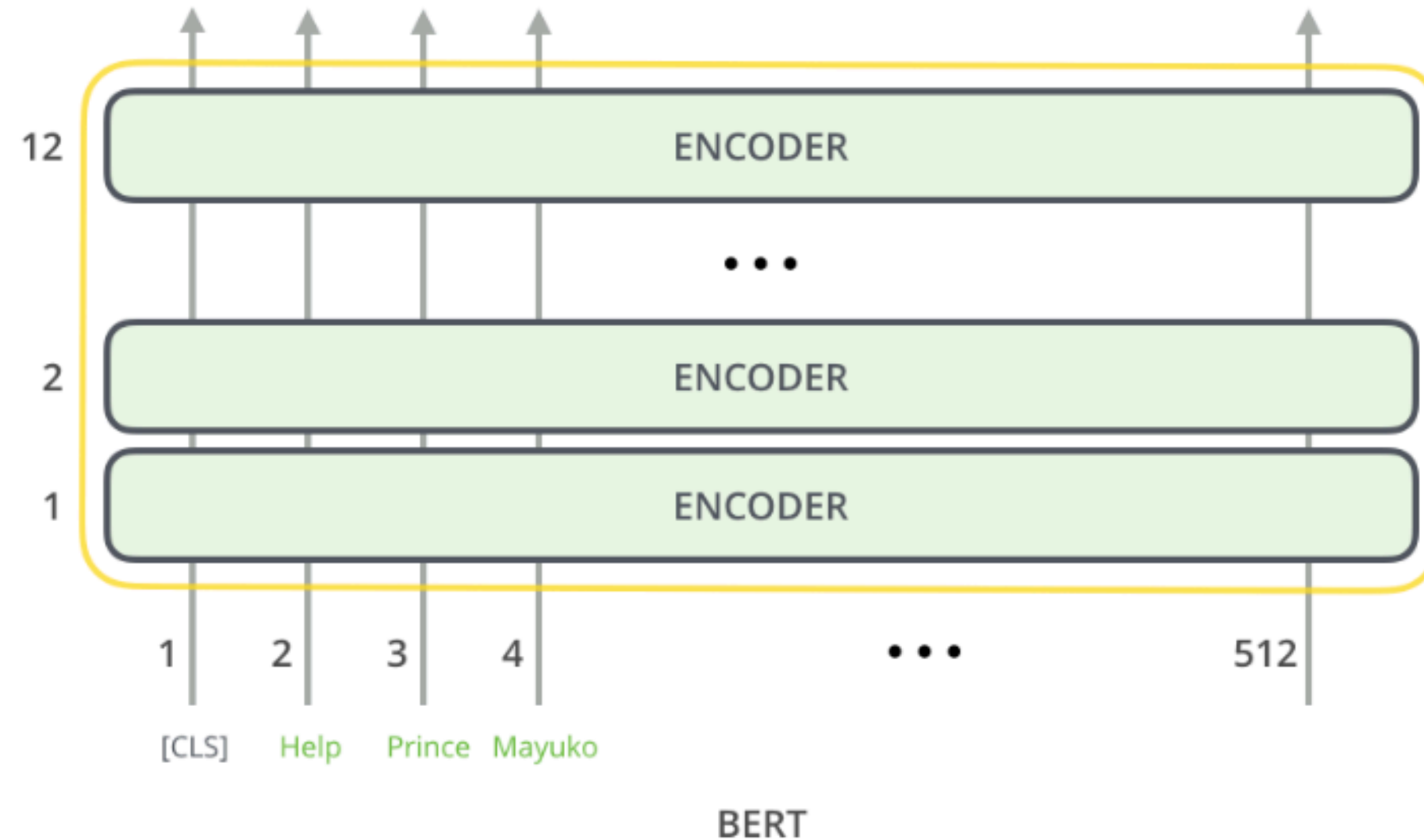
Model Inputs



The first input token is supplied with a special [CLS] token for reasons that will become apparent later on. CLS here stands for Classification.

<https://github.com/Amr-Abdellatif>

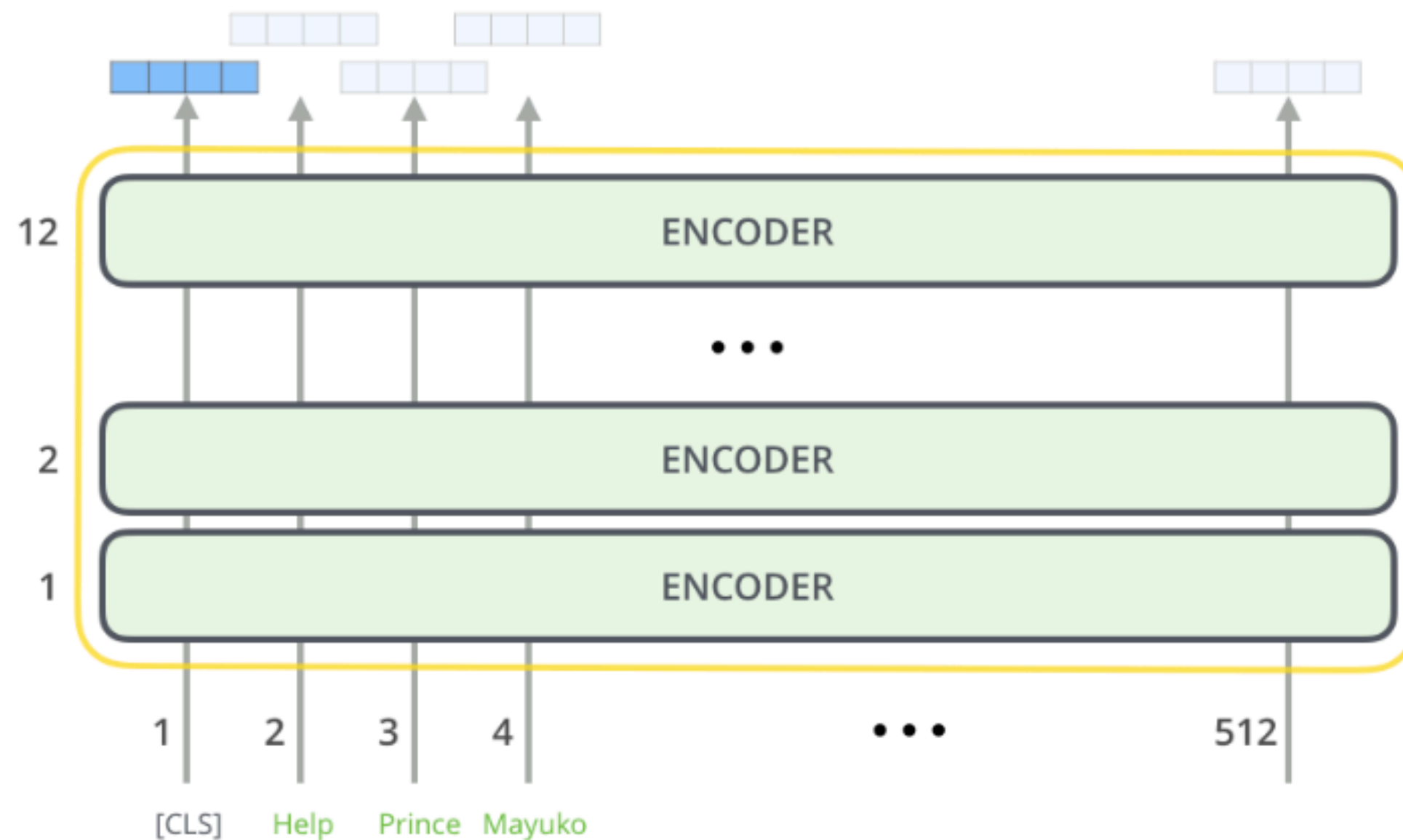
BERT



Just like the vanilla encoder of the transformer, BERT takes a sequence of words as input which keep flowing up the stack. Each layer applies self-attention, and passes its results through a feed-forward network, and then hands it off to the next encoder.

<https://github.com/Amr-Abdellatif>

BERT

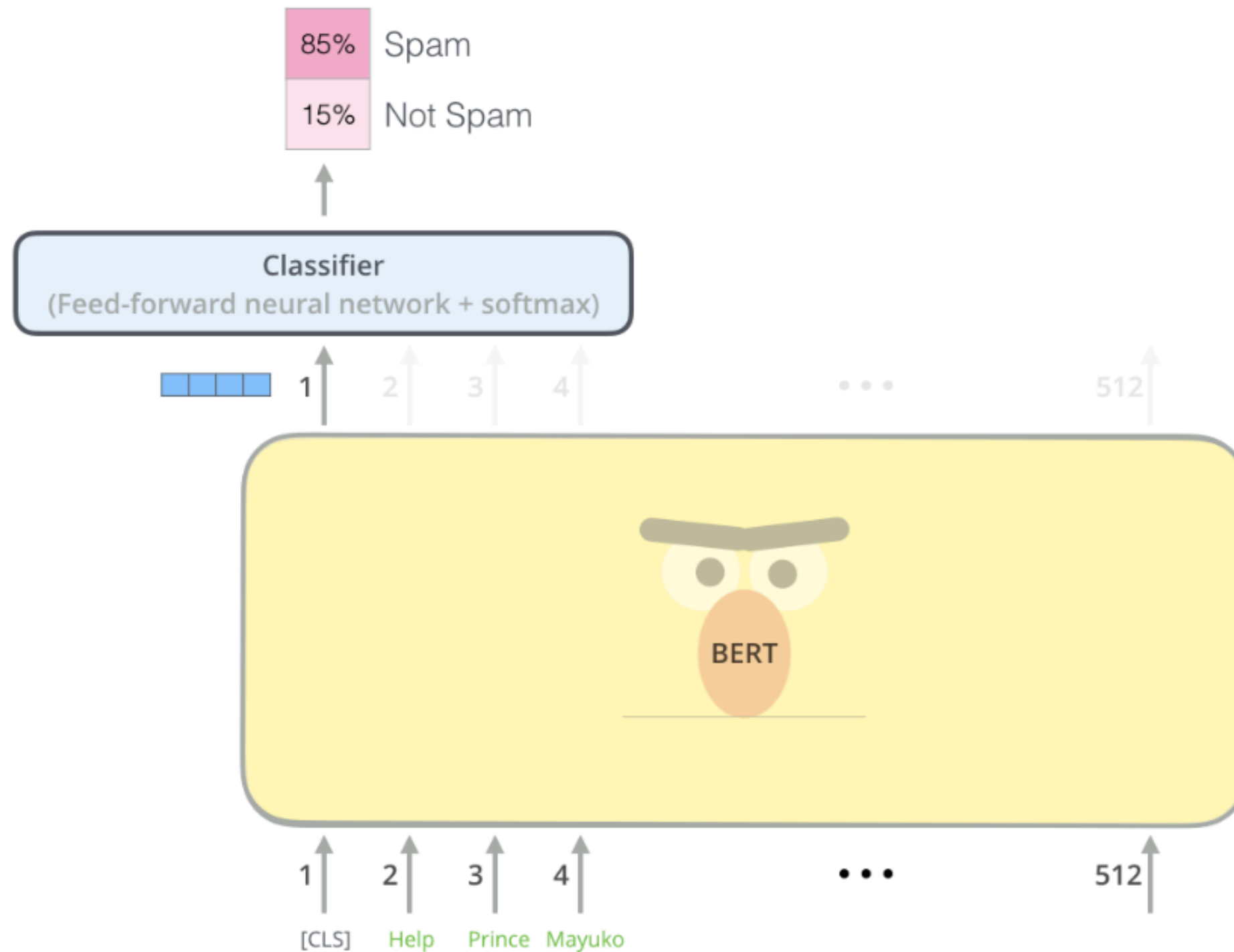


BERT

Each position outputs a vector of size *hidden_size* (768 in BERT Base). For the sentence classification example we've looked at above, we focus on the output of only the first position (that we passed the special [CLS] token to).

<https://github.com/Amr-Abdellatif>

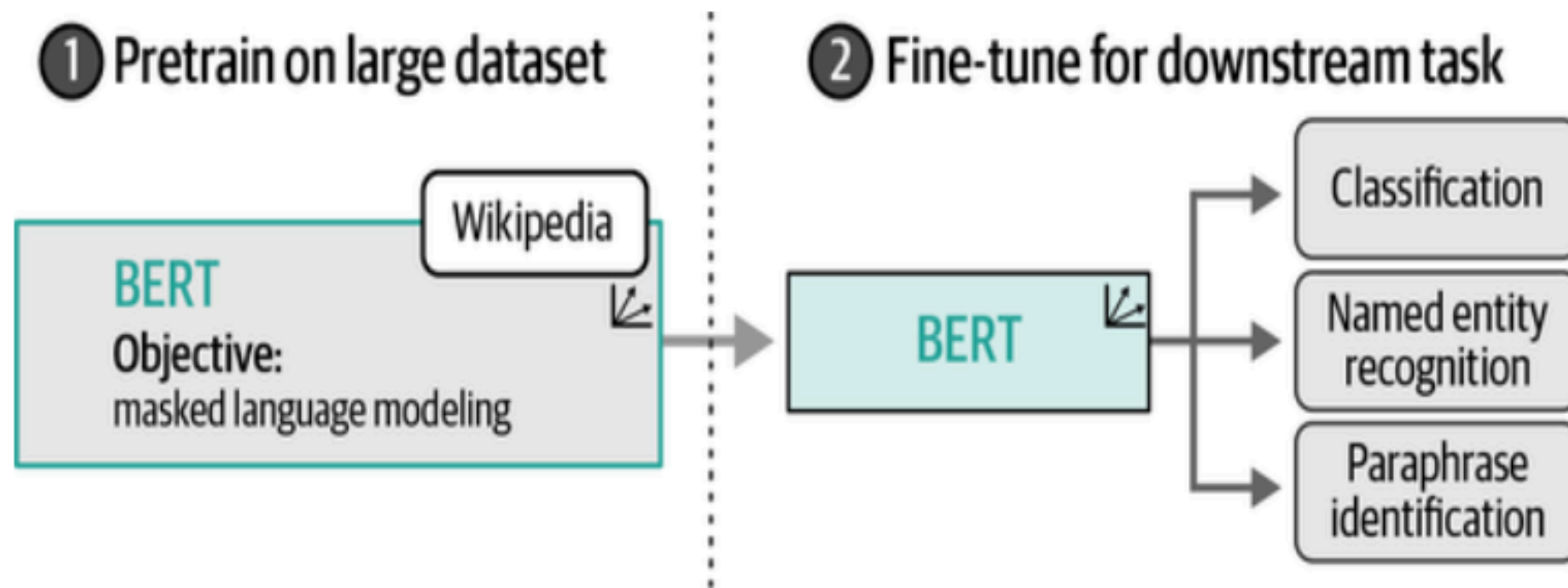
BERT



That vector can now be used as the input for a classifier of our choosing. The paper achieves great results by just using a single-layer neural network as the classifier.

<https://github.com/Amr-Abdellatif>

BERT



<https://github.com/Amr-Abdellatif>

BERT

code

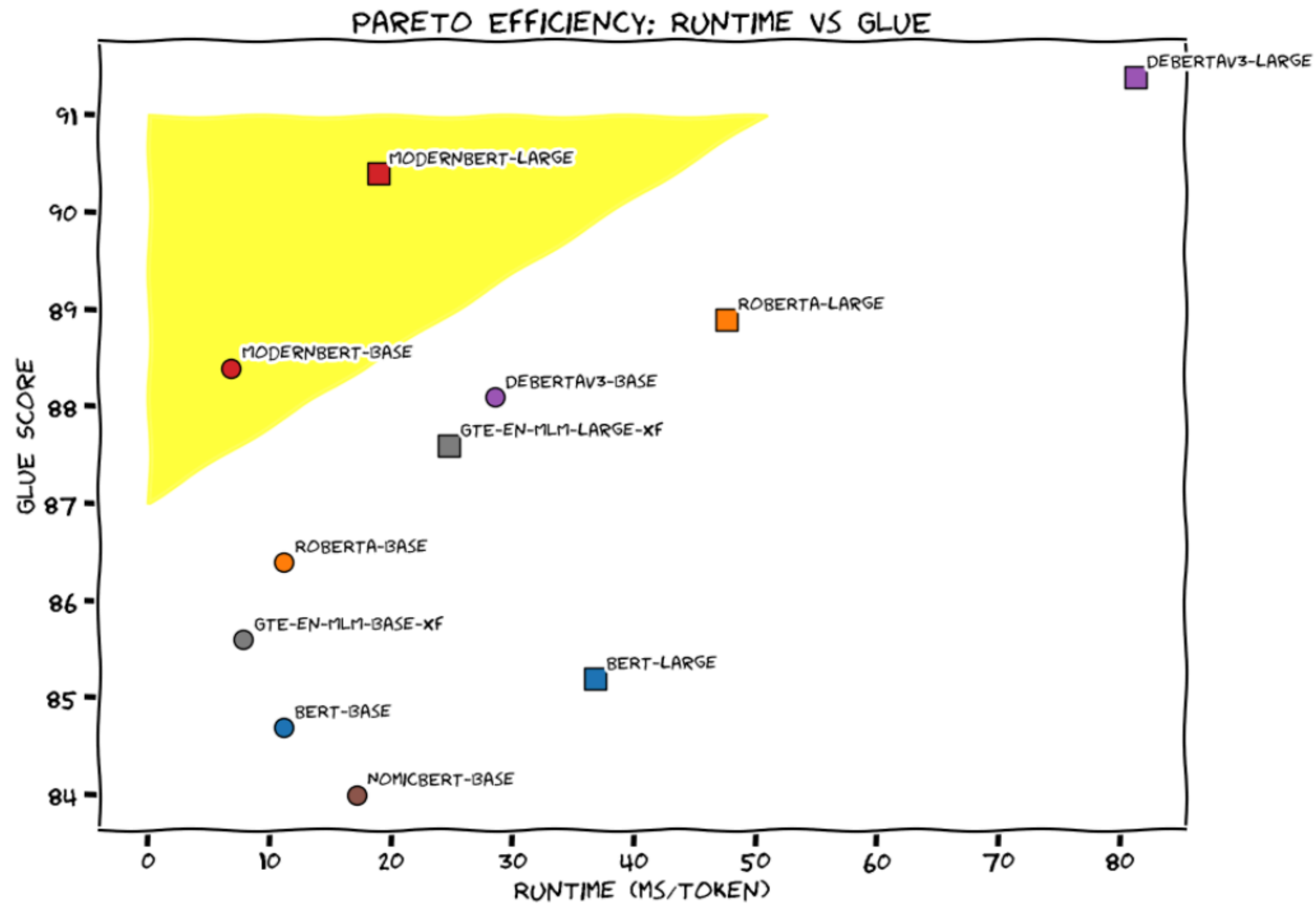
<https://github.com/Amr-Abdellatif>

BERT

Do another 2 downstream task

<https://github.com/Amr-Abdellatif>

BERT



<https://github.com/Amr-Abdellatif>

Can encoder only be used for something else?

<https://github.com/Amr-Abdellatif>

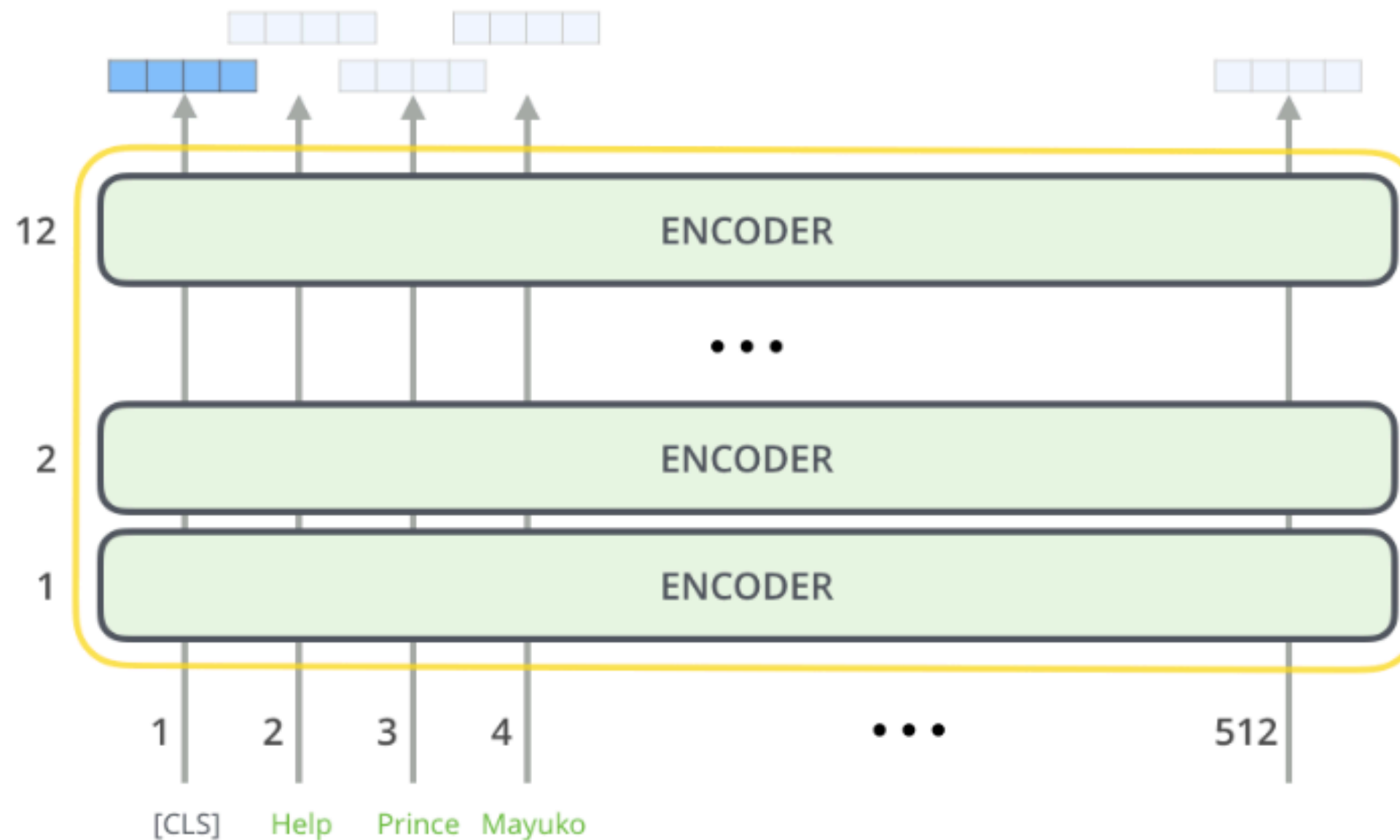
Embedding models

sentence-transformers/all-MiniLM-L6-v2

most downloaded model on the hub

<https://github.com/Amr-Abdellatif>

Embedding models



BERT

Text embeddings - Extract vector representations from the encoder's output layers. These dense vectors capture semantic meaning that's useful for Semantic search

<https://github.com/Amr-Abdellatif>

Cost

Inference costs: What the above suggests, is that on an inference-per-inference basis, there are many times more inferences performed per year on encoder-only models than on decoder-only or generative models. An interesting example is FineWeb-Edu, where model-based quality filtering had to be performed over 15 trillion tokens. The FineWeb-Edu team chose to generate annotations with a decoder-only model, Llama-3-70b-Instruct, and perform the bulk of the filtering with a fine-tuned BERT-based model. This filtering took 6,000 H100 hours, which, at HuggingFace Inference Endpoints' pricing of \$10/hour, comes to a total of \$60,000. On the other hand, feeding 15 trillion tokens to popular decoder-only models, even with the lowest-cost option of using Google's Gemini Flash and its low inference cost of \$0.075/million tokens, would cost over one million dollars!

<https://huggingface.co/blog/modernbert>

<https://github.com/Amr-Abdellatif>

LLM's

Generative AI

<https://github.com/Amr-Abdellatif>

Decoder only - LLM's



<https://github.com/Amr-Abdellatif>

LLM's

The nature of LLM's is complicated and they can generalize to untrained tasks

<https://github.com/Amr-Abdellatif>

LLM's

IN Context Learning

<https://github.com/Amr-Abdellatif>

LLM's

Zero shot learning

<https://github.com/Amr-Abdellatif>

LLM's

One shot learning

<https://github.com/Amr-Abdellatif>

LLM's

Few shot learning

<https://github.com/Amr-Abdellatif>

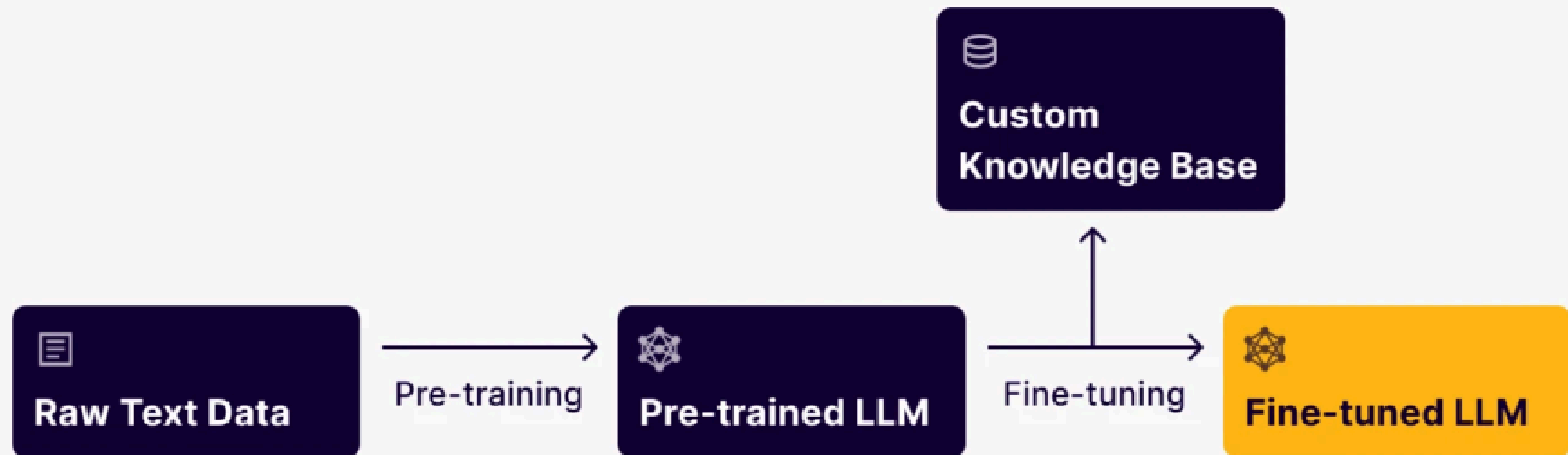
LLM's

Fine - Tuning

<https://github.com/Amr-Abdellatif>

LLM's

SFT



<https://github.com/Amr-Abdellatif>

LLM's

SFT

{"prompt": "Explain quantum computing in simple terms.", "completion": "Quantum computing uses quantum bits or qubits which, unlike regular bits, can exist in multiple states at once. This allows quantum computers to process complex problems much faster than traditional computers for certain types of calculations."}

{"prompt": "Write a short poem about the moon.", "completion": "Silver orb in night's dark sea,\nWatching over you and me.\nAncient light from distant past,\nSilent witness, ever vast."}

{"prompt": "What are three ways to improve code readability?", "completion": "1. Use consistent and descriptive naming conventions for variables and functions.\n2. Break complex operations into smaller, well-named functions.\n3. Add meaningful comments that explain why code exists rather than what it does."}

<https://github.com/Amr-Abdellatif>

LLM's

SFT

```
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "Explain photosynthesis briefly."}, {"role": "assistant", "content": "Photosynthesis is the process plants use to convert sunlight, water, and carbon dioxide into oxygen and energy in the form of sugar (glucose). The plant captures light energy using chlorophyll in its leaves, uses it to split water molecules, and combines the resulting hydrogen with CO2 to create glucose while releasing oxygen as a byproduct."}]}
```

```
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "Write a function in Python to check if a string is a palindrome."}, {"role": "assistant", "content": "```python\ndef is_palindrome(text):\n    # Remove non-alphanumeric characters and convert to lowercase\n    clean_text = ''.join(char.lower() for char in text if char.isalnum())\n    # Check if the string equals its reverse\n    return clean_text == clean_text[::-1]\n\n# Example usage\nprint(is_palindrome(\"A man, a plan, a canal: Panama\")) # True\nprint(is_palindrome(\"hello\")) # False\n```"}]
```

<https://github.com/Amr-Abdellatif>

LLM's

RLHF

- Pre-training: Start with a foundation model that's been pre-trained on a large corpus of text.
- Supervised Fine-Tuning (SFT): Fine-tune the pre-trained model on high-quality demonstrations of desired behavior.
- Input: Prompt-completion pairs created by humans
- Output: A model that can generate good responses but isn't yet optimized for human preferences
- Reward Model Training:
 - Collect human preference data (comparisons between responses)
 - Train a reward model to predict which responses humans prefer
 - Input: Prompt and multiple completions with preference labels
 - Output: A model that can score how good a completion is according to human preferences
- RL Optimization (often using PPO - Proximal Policy Optimization):
 - Use the reward model as a feedback mechanism
 - Optimize the SFT model to maximize the predicted reward
 - Input: Prompts for generation + reward model
 - Output: A model that generates responses optimized for human preferences

<https://github.com/Amr-Abdellatif>

LLM's

RLHF

```
{
  "prompt": "Explain why the sky is blue in scientific terms.",
  "responses": [
    {
      "text": "The sky appears blue due to a phenomenon called Rayleigh scattering. Sunlight contains all colors of the visible spectrum, but blue light has a shorter wavelength and scatters more in our atmosphere than other colors. This scattered blue light comes to our eyes from all directions, making the sky appear blue.",
      "rank": 1
    },
    {
      "text": "The sky is blue because blue light from the sun gets scattered in all directions by the gases and particles in the atmosphere. This process is called Rayleigh scattering.",
      "rank": 2
    },
    {
      "text": "The sky looks blue because of light from the sun hitting our atmosphere.",
      "rank": 3
    }
  ]
}
```

<https://github.com/Amr-Abdellatif>

Blog Post on Training an LLM

<https://huggingface.co/blog/bloom-megatron-deepspeed>

<https://github.com/Amr-Abdellatif>

LLM's

Hallucinations

<https://github.com/Amr-Abdellatif>

what is Rag ?

<https://github.com/Amr-Abdellatif>

who is the current president of America ?

<https://github.com/Amr-Abdellatif>

Knowledge cutoff problem

Hallucinations

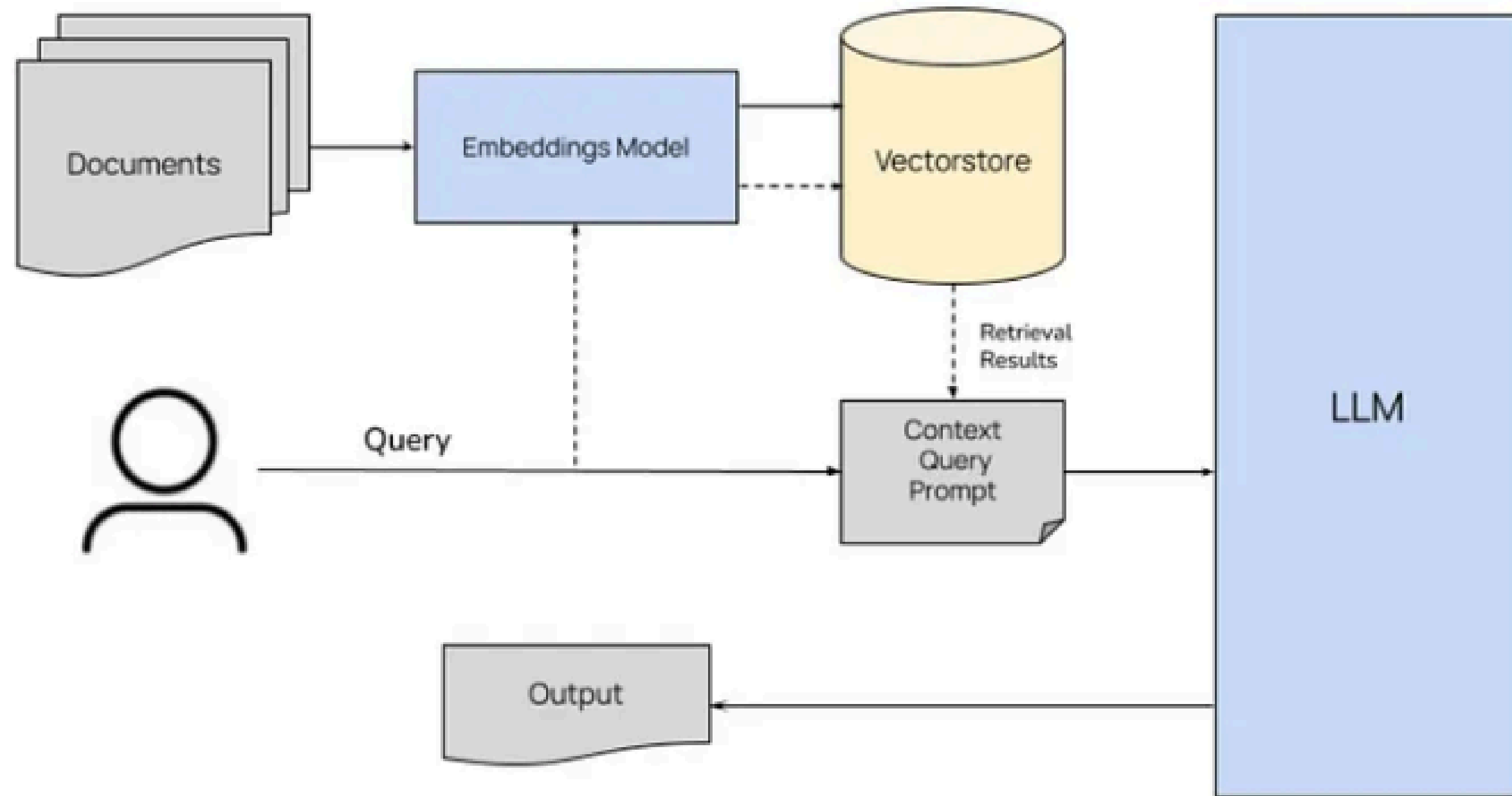
Domain-specific expertise

Source attribution

Private data handling

<https://github.com/Amr-Abdellatif>

Rag



<https://github.com/Amr-Abdellatif>

Rag

Step 1 : Chunking

splitting the documents into chunks/parts

<https://github.com/Amr-Abdellatif>

Rag

Step 2 : Embedding

Embedding is taking chunked text and changing it into Vectors

<https://github.com/Amr-Abdellatif>

Rag

Step 3 : Indexing

storing these vectors in a Vector DB

<https://github.com/Amr-Abdellatif>

Rag

Step 4 : retrieval

we ask a question and retrieve chunks from the Vector DB based on that question through a similarity metric like cosine similarity

<https://github.com/Amr-Abdellatif>

Rag

Step 5 : question answering by LLM

we pass the chunks with the user question to an LLM to answer

<https://github.com/Amr-Abdellatif>

If i have a alot of chunks of texts, how can i know which one has the info related to my question ?



<https://github.com/Amr-Abdellatif>

Retrieval

what are the ways to retrieve relevant context ?

<https://github.com/Amr-Abdellatif>

Retrieval

1. Dense Vector Retrieval
2. Sparse Retrieval
3. Hybrid Retrieval
4. Multi-Vector Retrieval
5. Hierarchical Retrieval
6. Reranking
7. Query Expansion & Transformation

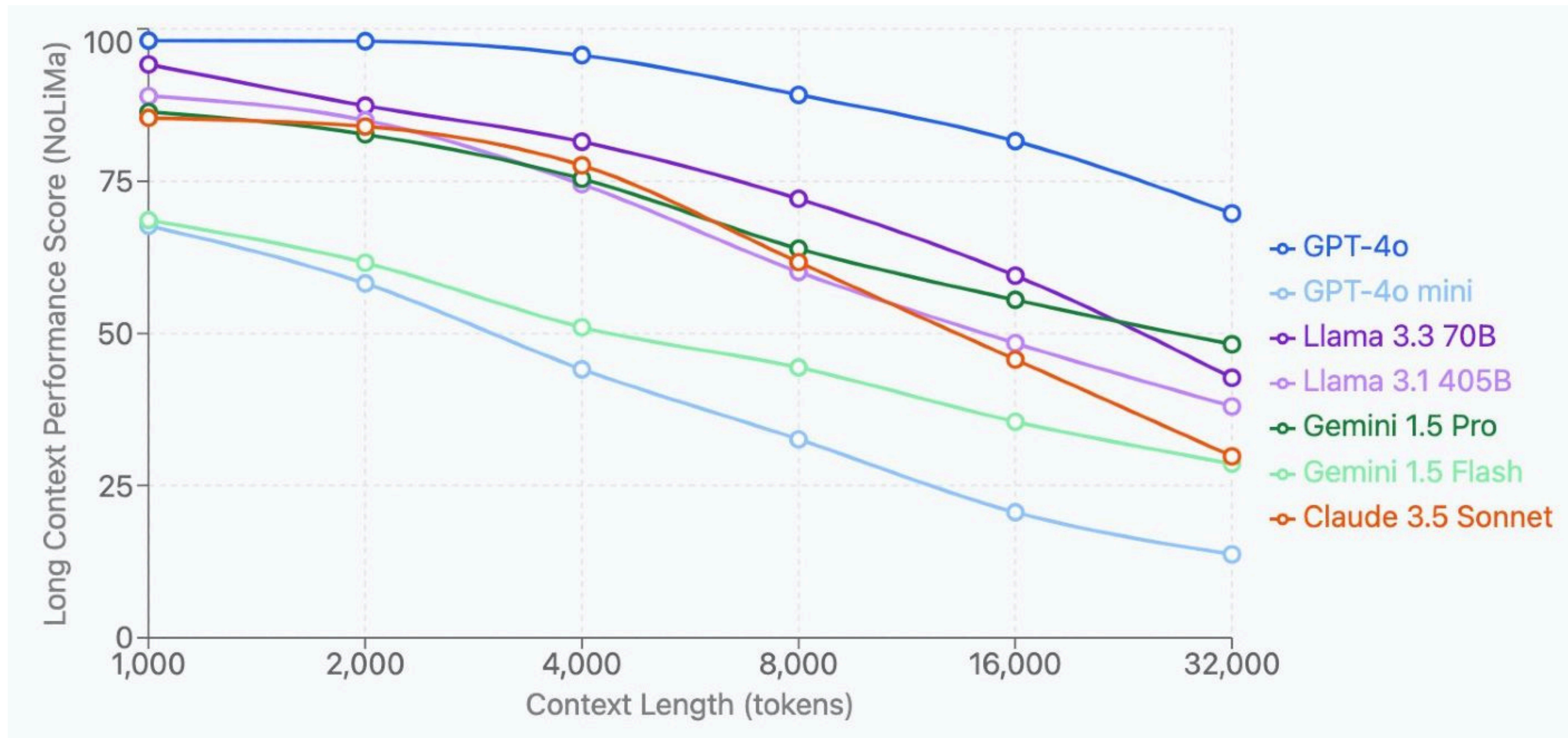
<https://github.com/Amr-Abdellatif>

Retrieval

are we limited by the context window ? how many K chunks can we provide ?

<https://github.com/Amr-Abdellatif>

the longer the context the more degradation you get



<https://github.com/Amr-Abdellatif>

Rag

Evaluation

<https://github.com/Amr-Abdellatif>

Rag

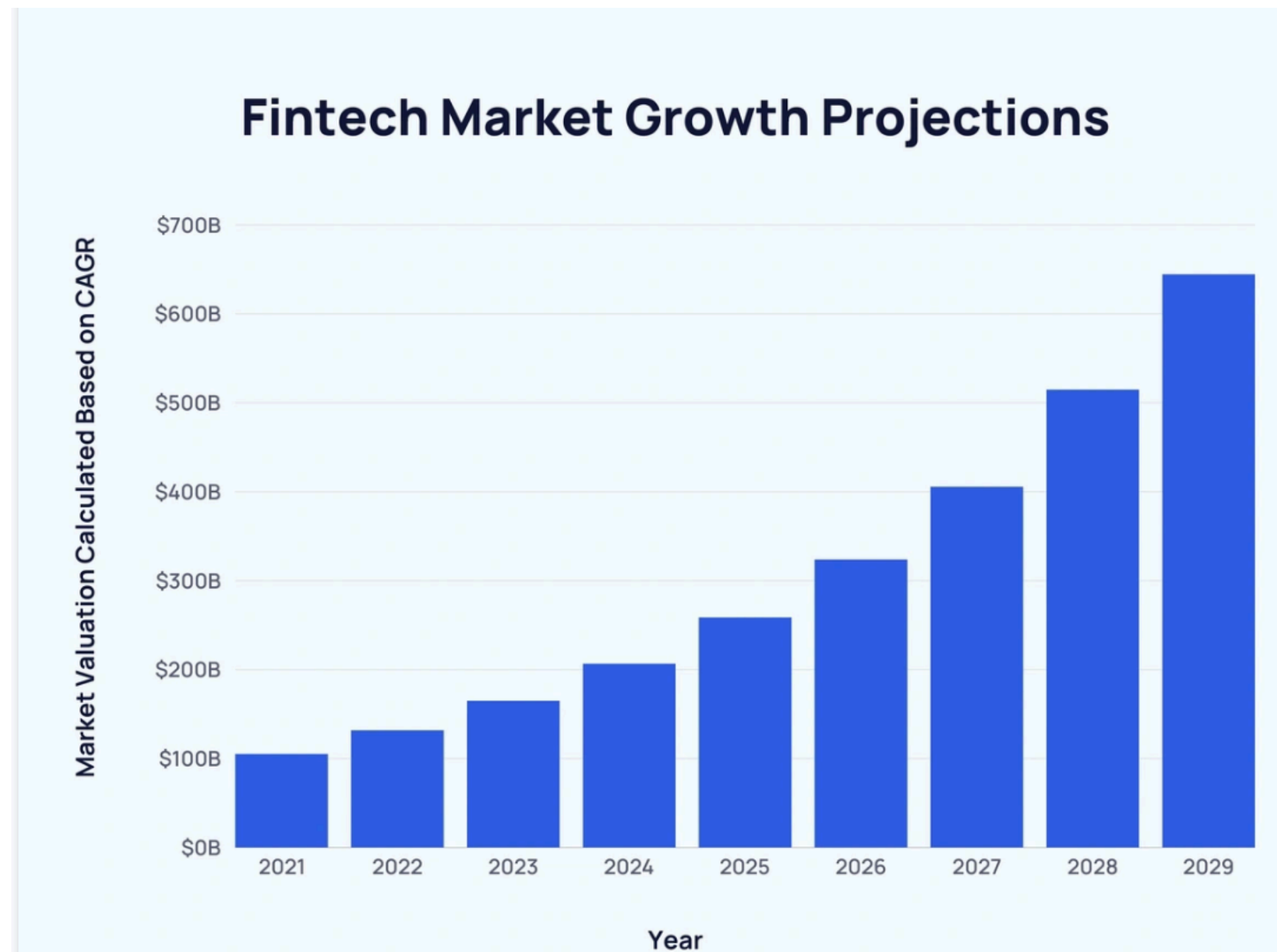
Business evaluation is the point

<https://github.com/Amr-Abdellatif>

Rag problems in multimodality

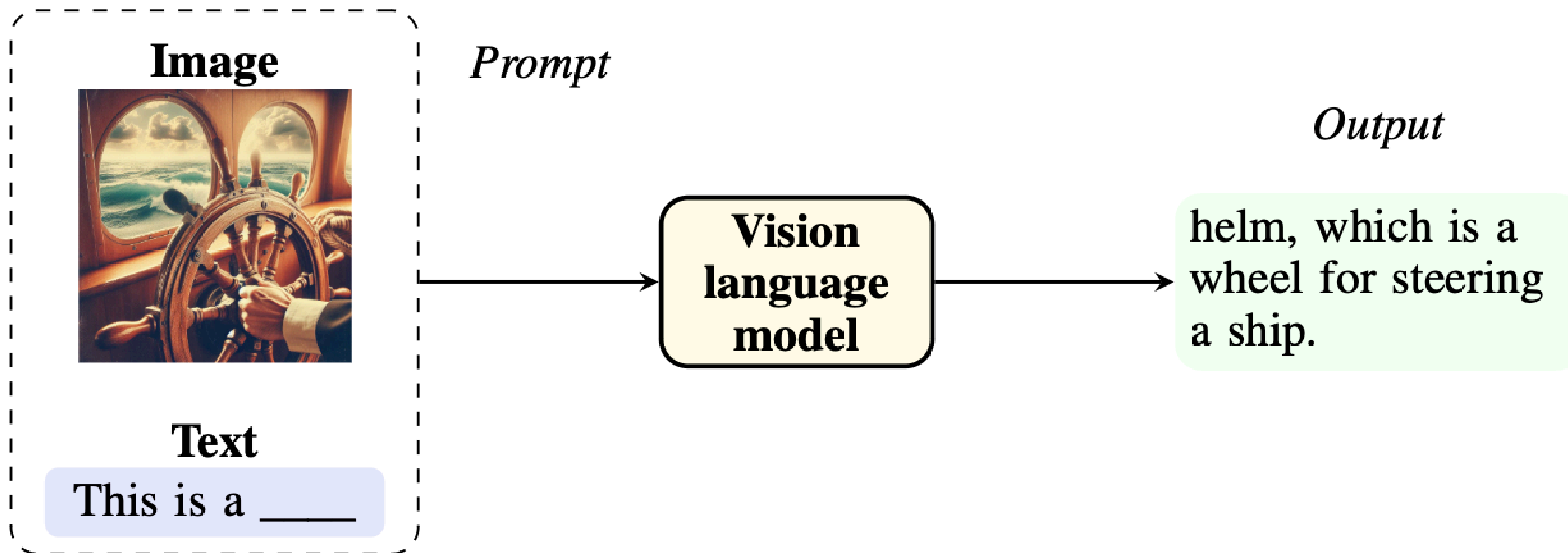
<https://github.com/Amr-Abdellatif>

Can we interpret a visualization ?



<https://github.com/Amr-Abdellatif>

VLM's

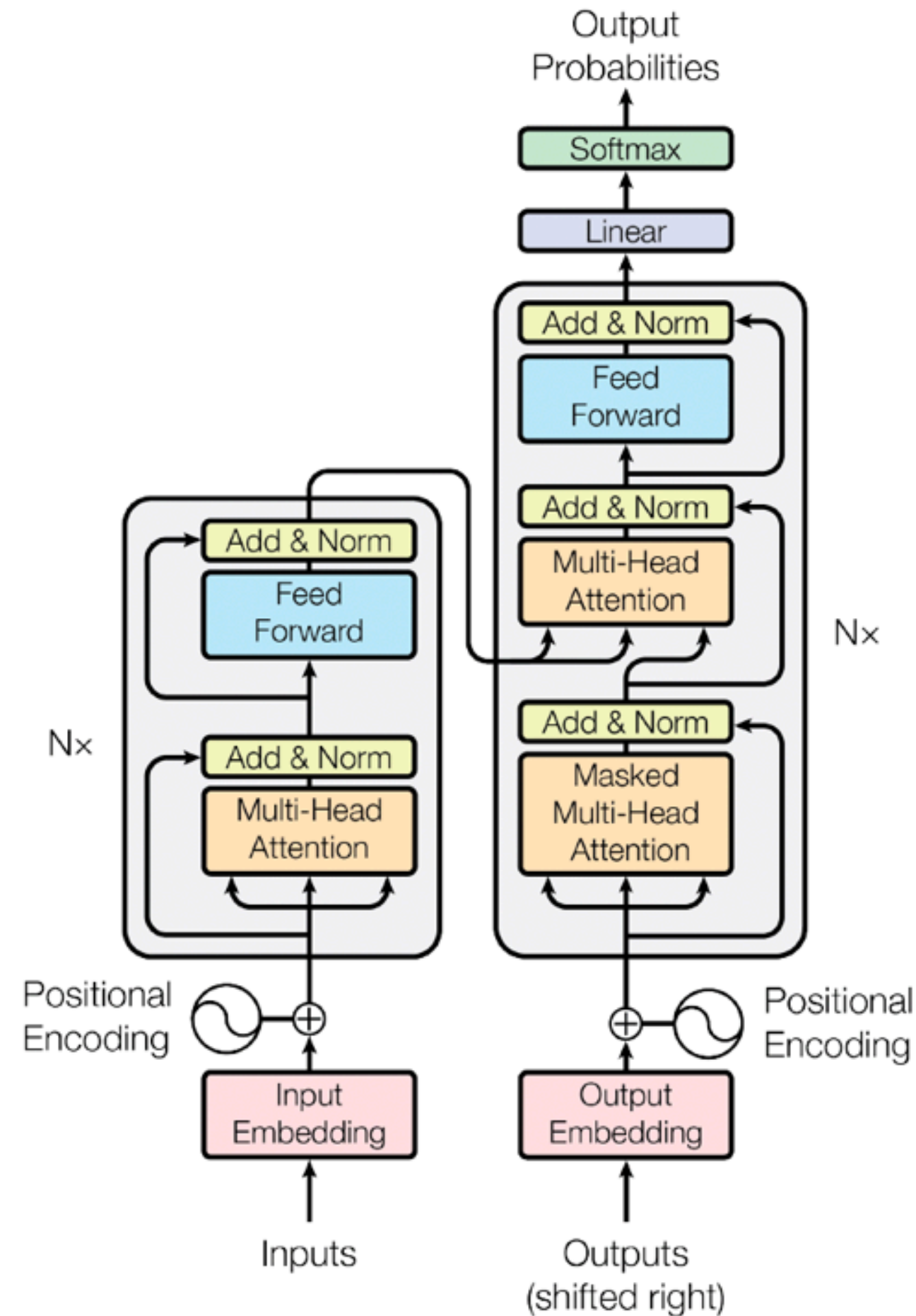


<https://github.com/Amr-Abdellatif>

How VLM's work ?

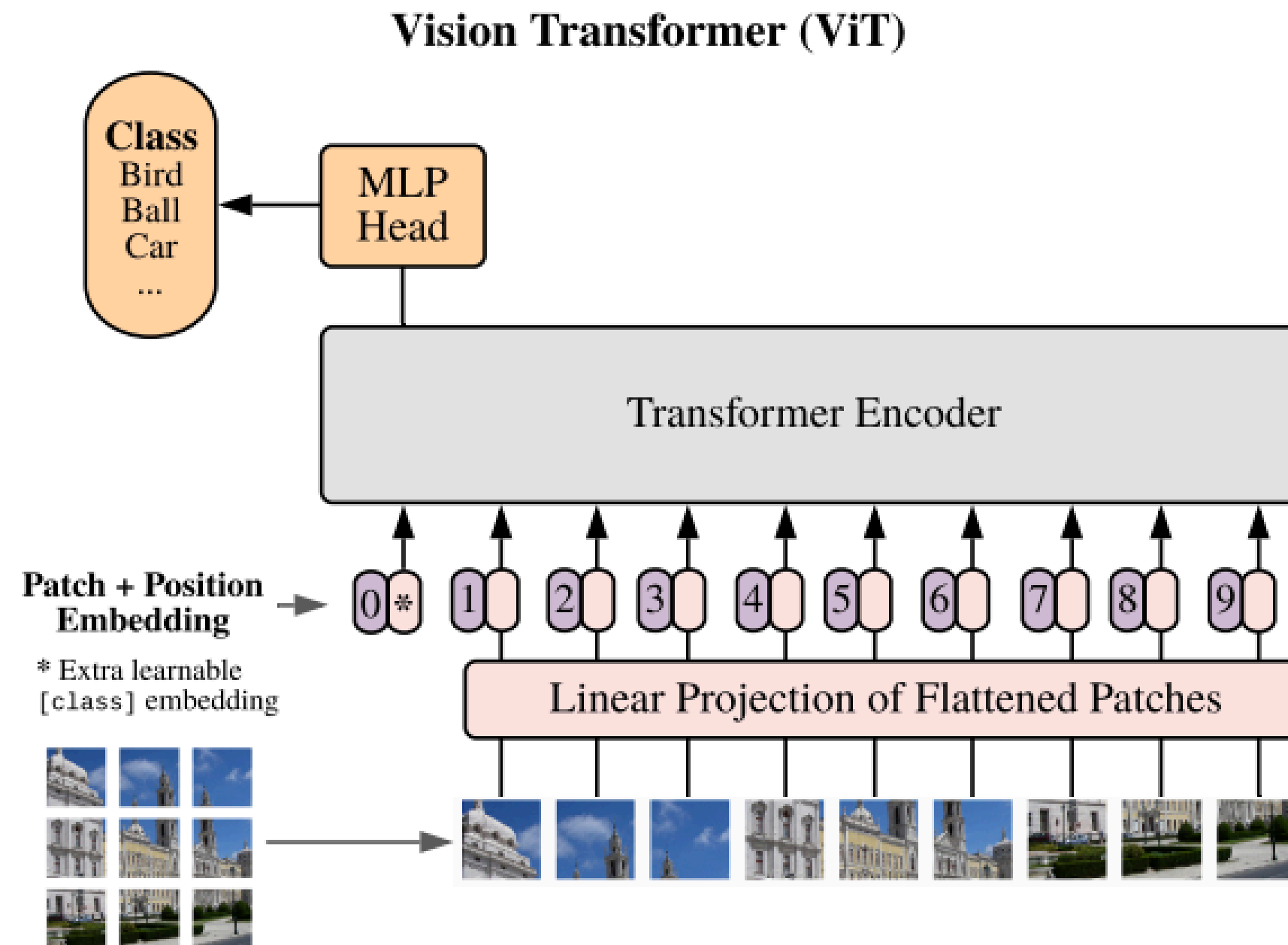
<https://github.com/Amr-Abdellatif>

To answer this lets go back to Transformers 2017



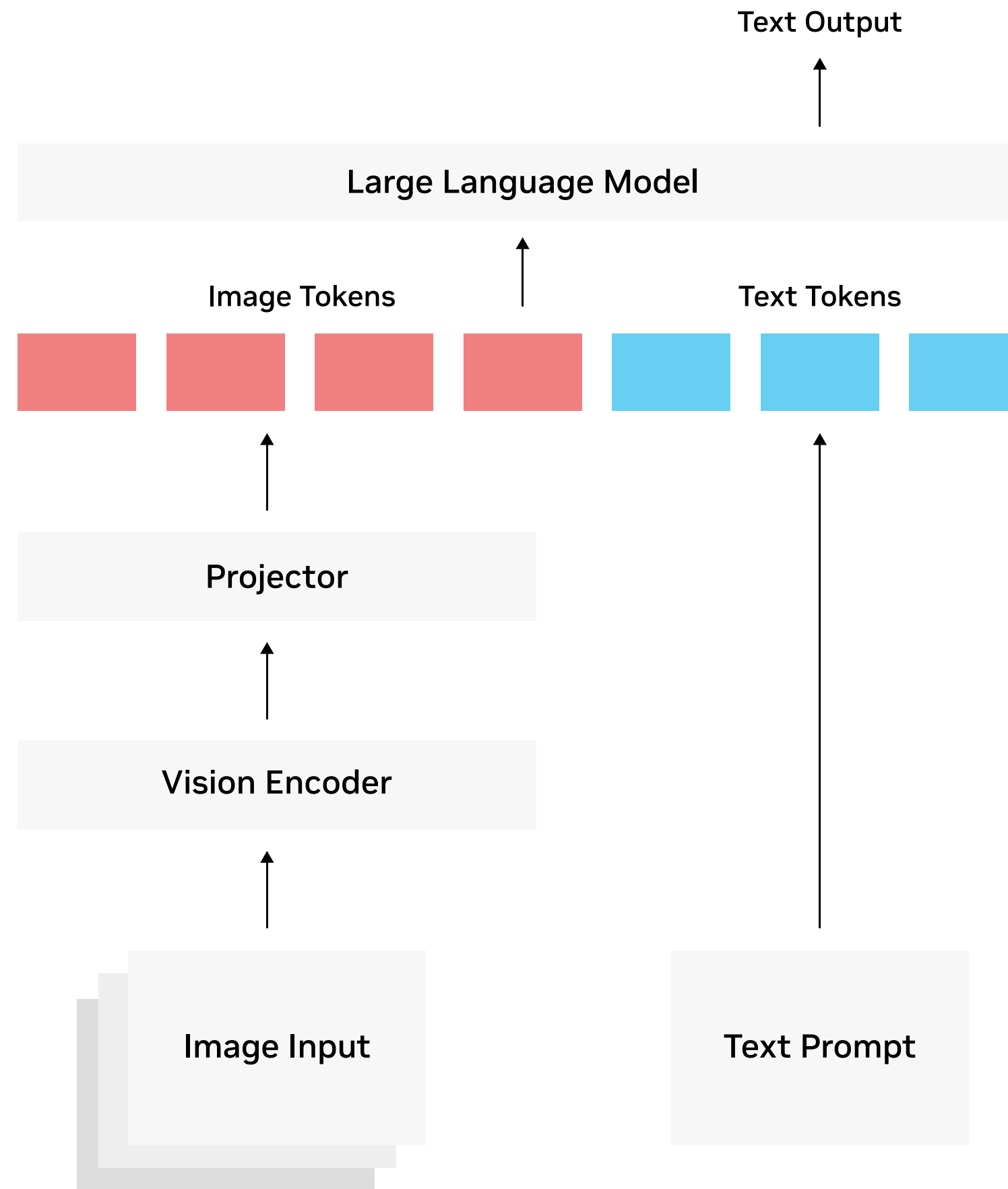
A simple task is Translation

2021 ViT Paper



<https://github.com/Amr-Abdellatif>

Encoder - Decoder - Projection Layer



Add-Ons

اكتب الكود بايدك

Vision Transformer paper

<https://huggingface.co/blog/AviSoori1x/seemore-vision-language-model>

<https://github.com/Amr-Abdellatif>

SOTA

Qwen-VL 2.5

<https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct>

<https://github.com/Amr-Abdellatif>

can we extract Arabic text from a visuals as well ?

<https://github.com/Amr-Abdellatif>

Example

<https://github.com/Amr-Abdellatif>

Next Problem !

<https://github.com/Amr-Abdellatif>

Multi-Modal Retriever's

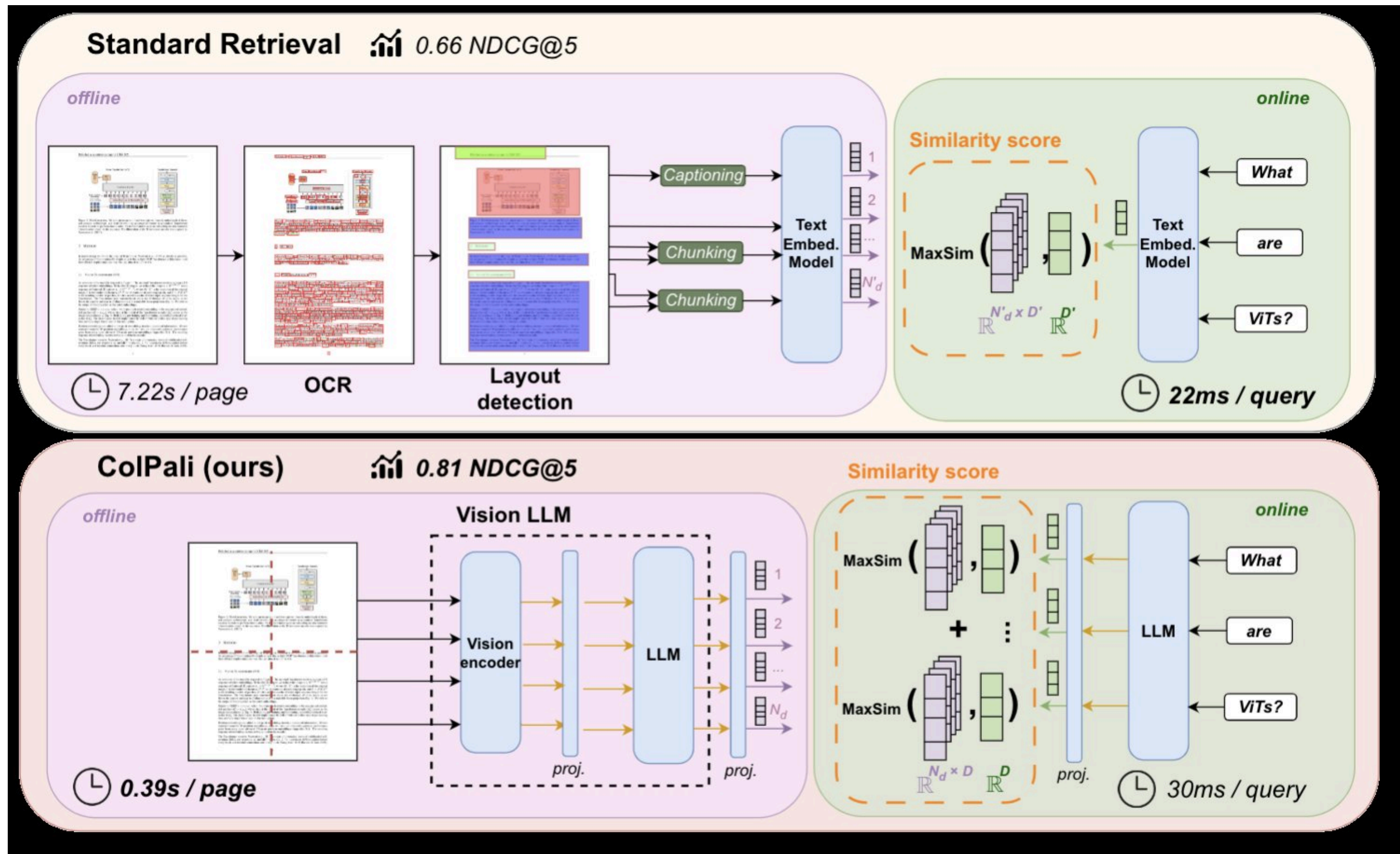
<https://github.com/Amr-Abdellatif>

If i have a big chunk of images, how can i know which one has the info related to my question ?



<https://github.com/Amr-Abdellatif>

The Old way vs Colpali



<https://github.com/Amr-Abdellatif>

Add Ons

<https://huggingface.co/blog/manu/colpali>

<https://github.com/Amr-Abdellatif>

**What if i have a charts/ visualizations pdf and i want to
chat with this pdf ?**

<https://github.com/Amr-Abdellatif>

VisRag- Ret

<https://github.com/Amr-Abdellatif>

Col-qwen 2.5

<https://github.com/Amr-Abdellatif>

Rag ADD ONS Overview

<https://github.com/Amr-Abdellatif>

Another problem !
What if i have multilingual pdf's ?

<https://github.com/Amr-Abdellatif>

if i have text in English & Arabic how to embedd ?

<https://github.com/Amr-Abdellatif>

multilingual text embeddings

<https://github.com/Amr-Abdellatif>

Example

<https://github.com/Amr-Abdellatif>

END

<https://github.com/Amr-Abdellatif>